

Мастер рад

1. Увод.....	3
2. Пројектовање и израда система за балансирање лопте на шини	5
2.1. Пројектовање система за балансирање лопте на шини	6
2.2. Израда пројектованих делова.....	9
3. Математички модел система за балансирање лопте на шини	12
3.1. Једначина кретања лопте на шини.....	16
3.1.1. Једначина промене нагибног угла шине	16
3.1.2. Једначине кретања лопте на шини и преносна функција објекта	17
3.2. Математичко модел серво мотора једносмерне струје.....	19
3.2.1. Прикупљање података помоћу експеримента	19
3.2.1.1. Пројектовање и израда платформе за испитивање.....	19
3.2.1.2. Поставка експеримента и прикупљање података	20
3.2.1.3. Обрада података.....	32
3.2.2. Идентификација система методом сиве кутије.....	39
3.2.2.1. Општи математички модел електро мотора једносмерне струје са редуктором	39
3.2.2.2. Одређивање математичког модела	42
3.2.3. Идентификација система методом црне кутије	50
3.2.3.1. Вишеслојна каскадна неуронска мрежа без повратне спреге	50
3.2.3.2. Дефиниције ауторегресивних модела.....	52
3.2.3.3. Идентификација система	54
4. Пројектовање и имплементација регулатора.....	57
4.1. Анализа и испитивање система	57
4.2. Одабир регулатора	62
4.3. Пројектовање и подешавање регулатора.....	64
4.4. Имплементација регулатора	71

4.5.	Поређење математичких модела са стварним системом	76
5.	Закључак.....	80
6.	Додатак.....	83
6.1.	Додатак А – Техничка документација стандардних компоненти	83
6.1.1.	Микроконтролер – Arduino Uno	83
6.1.2.	Сензор растојања – VL53L0X.....	85
6.1.3.	Серво мотор	86
6.1.4.	Микропроцесор - Raspberry Pi 4.....	87
6.1.5.	Ротациони енкодер – LDP3806–600BM–5G –24C.....	88
6.2.	Додатак Б – Помоћне скрипте.....	90
6.2.1.	data_processing_01.py	90
6.2.2.	data_processing_02.py.....	91
6.2.3.	data_visualization.py.....	93
7.	Литература	95

1. Увод

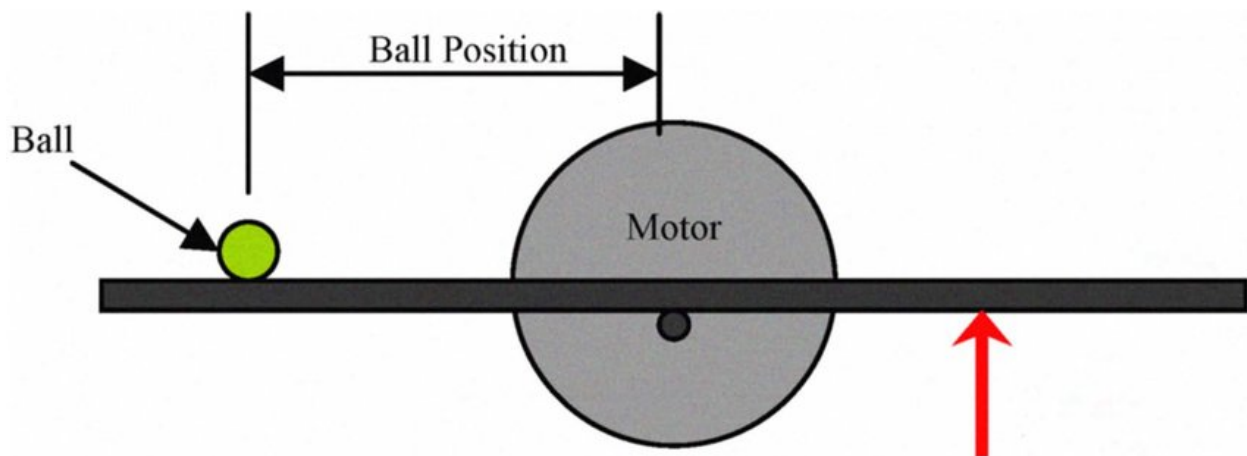
Управљање нестабилним системима је веома важан задатак у аутоматском управљању, а како се ови системи сматрају веома опасним они се испитују у лабораторијама. Систем за балансирање лопте на шини јесте нестабилан систем и може се поистоветити са реалним проблемима као што су хоризонтална стабилизација авиона током слетања или у средини турбулентног струјања.

Систем за балансирања лопте на шини је веома лак за разумевање и многе технике управљања се могу изучавати на њему, па се због тога често може наћи у лабораторијама инжињера за аутоматско управљање.

Овај систем се састоји од лопте и шине чији се угао заогретања контролише мотором, а циљ оваког система јесте да се лопта заустави на тачно одређеној позицији на шини. Ако је управљање лоше, лопта ће пасти са шине.

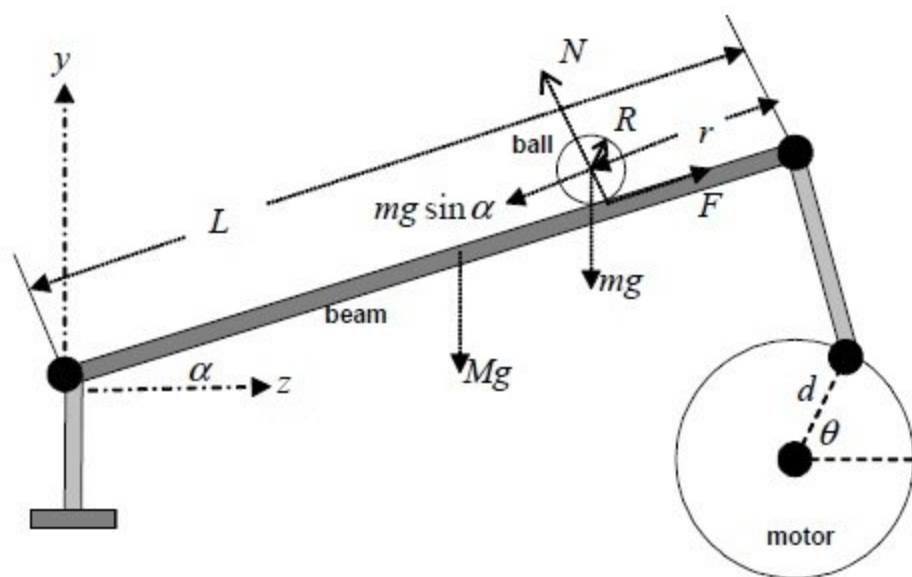
Систем балансирања лопте на шини се може видети у две конфигурације.

Прва конфигурација је приказана на слици 1.1. Са слике се види да је греда ослоњена у средини и да се ротира око своје централне осе. Оваква конфигурација поседује две предности. Прва предност је та, што је лака за израду, а друга јесте то што јој је математички модел релативно једноставан.



Конфигурација 1 – Слика 1.1

Друга конфигурација је нешто сложенија, а приказана је на слици 1.2. У овој конфигурацији греда има две тачке ослоња. Један крај је непокретан и греда се ротира око његове тачке, док је други крај преко система полуга спојен на мотор. Оваква конфигурација је теже за израду и има мало сложенији математички модел.



Конфигурација 2 – Слика 1.2

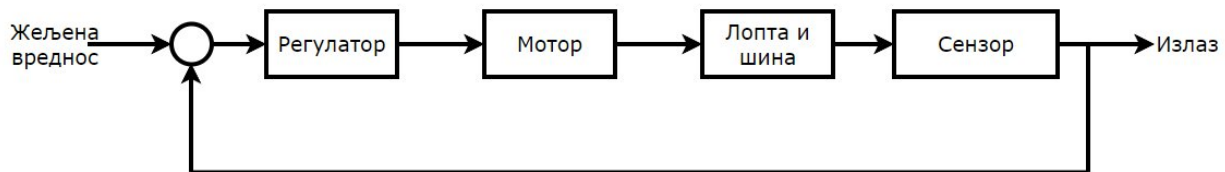
У овом раду се користи друга конфигурација, без обзира на сложеност. Такође, напомену бих да су следеће теме обрађене:

- Пројектовање и израда система за балансер лопте на шини,
- Одређивање математичког модела система за балансирање лопте на шини,
- Пројектовање и имплементација регулатора и
- Испитивање у реалном времену.

2. Пројектовање и израда система за балансирање лопте на шини

Први пробелми и задаци овог мастер рада јесу пројектовање и израда система за балансирање лопте на шини. А да би се у опште могло испројектовати систем за балансирање лопте на шини потребно је знати одговоре на питања “Шта је циљ овог система? И како у опште он ради?”.

Циљ овог система јесте да доведе лопту на одређену позицију тако што ће сам себе избацити из равнотежног стања. Да би се то боље разумело, претпоставимо да имамо шину дугачку 50cm, да се лопта налази и мирује на десном крају те шине и да је жељена позиција те исте лопте средина шине. У следећем кораку, на основу жељене вредности излаза, мотор треба да заротира излазно вратило за угао α и нагне шину на лево, односно за угао β , сензор чита нову позицију лопте, која сада креће према среди, а затим регулатор на основу разлике жељене и стварне позиције лопте формирати нови управљачки сигнал за мотор. У другом кораку мотор ће поново заротирати излазно вратило за угао α и кориговаће угао шине, а сензор чита нову позицију лопте и помоћу регулатора формирати нови сигнал за мотор. Ови кораци ће се понављати све док се лопта не доведе на жељену позицију. Све претходно објашњено може се представити и једним алгоритмом који је приказан на слици 2.1.



Уопштени дизајн система – Слика 2.1

На основу уопштеног дизајна и изабране конфигурације система могу се формирати и додатна под питања.

Прво питање на које треба одговорити је “Шта се пројектује?”, а одговор на то је: “Систем за балансирање лопте на шини који се гони мотором преко система полуа”. Следећа питања на која треба одговорити јесу:

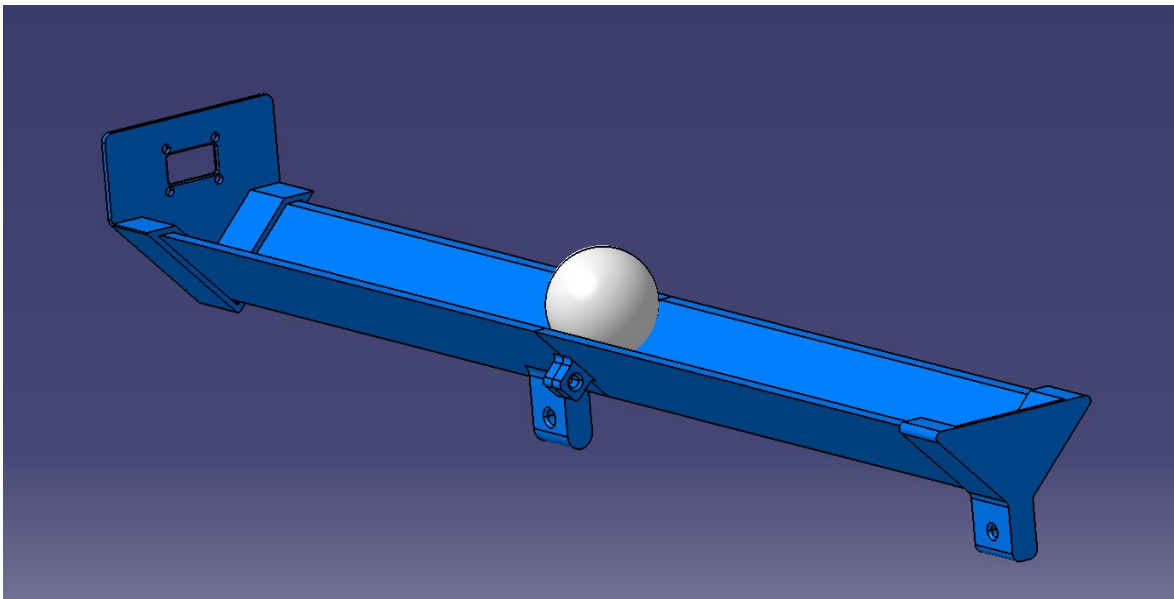
- Којим сензором се мерити позиција лопте на шини? Којим мотором се гонити цео систем? Којим контролером се управља системом?
 - За мерење позиције изабран је ласерски сензор удаљености VL530LX, за извршни орган серво мотор непознатог произвођача и за формирање и прослеђивање управљачког сигнала микроконтролер Arduino Uno. Техничка документација и више информација о овим компонента, као и осталим компонентама које су коришћене приликом израде овог мастер рада, може се пронаћи на крају овог рада у додатку А.

- Како ће се сви испројектовани, нестандартни, делови израдити?
 - Сви делови који су испројектовани за овај мастер рад су израђени адитивном технологијом израде, тј. на 3D штампачу.

2.1. Пројектовање система за балансирање лопте на шини

Након што су дати одговори на сва претходна питања, слика самог система постаје јаснија и може се приступи пројектовању и моделирању система.

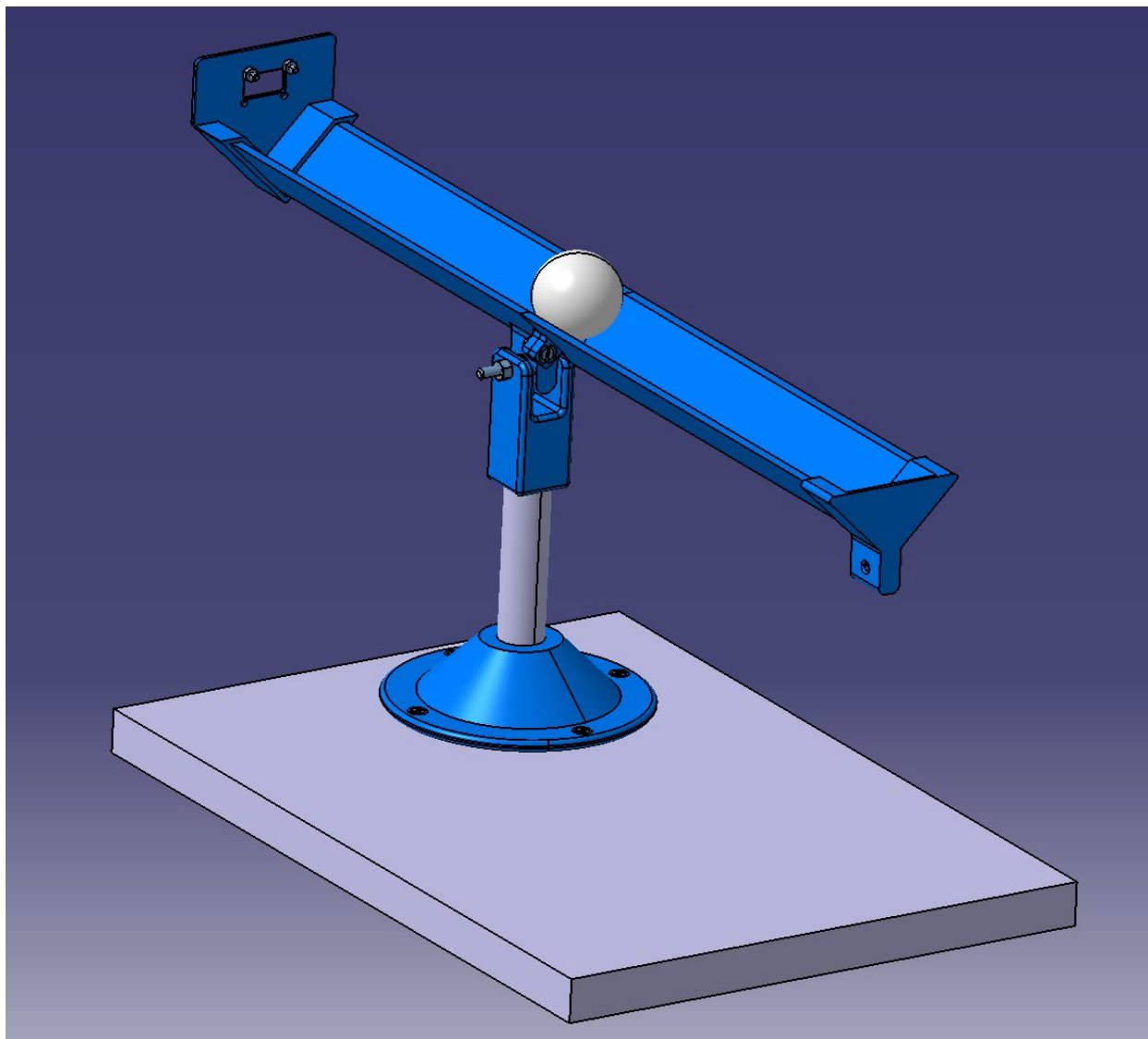
Први део који је испројектован јесте шина и она је кроз пар итерација добила следећи облик:



Шина са лоптицом – Слика 2.2

Са слике се види да се шина састоји из четири дела. Саме шине која се састоји из два дела, који се вијчаном везом спајају на среди, и два граничника, који су залепљена епоксидним лепком на крајевима шине. Једна половина шине поседује ушицу преко које се шина спаја са ослонцем, као и један граничник чија ушица служи за везу са мотором преко система полуга. Док други граничник је уједно и носач за ласерског сензор удаљености.

Следећи испројектовани део јесте носач шине. Носач се састоји из три дела; ушице, навојне шипке и базе. Делови су спојени навијањем на навојну шипку. На једном крају шипке навијена је ушица, а на другом база. Цео носач је затим зашрафљен за постоље од иверице.

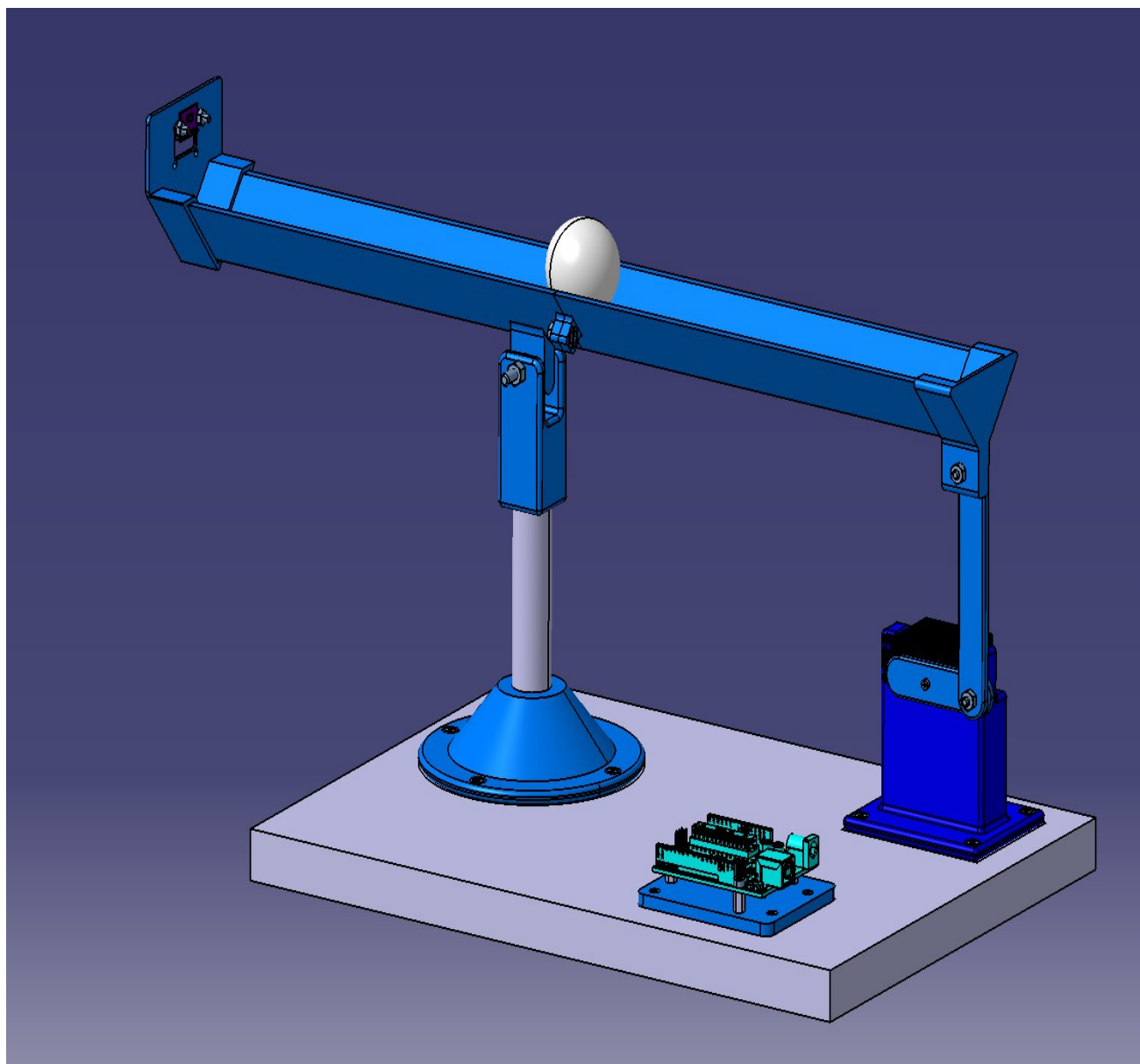


Шина са носачем – Слика 2.3

Након шине и носача, било је потребно испројектовати и додати везу између шине и мотора, сензор, микроконтролер и носаче за микроконтролер и серво мотор. Веза између шине и мотора се састоји из две полуге различитих дужина, где је дужа полука повезана са шином на једном крају, а на другом крају са мањом полугом, која је повезана са серво мотором.

Финални модел система за балансирање лопте на шини приказан је на слици 2.4

Цео овај систем је измоделиран у софтверу за Компјутерски Адитивни Дизајн (CAD) CATIA и може се пронаћи на следећем линку:
https://github.com/zelja182/Ball_And_Beam/tree/main/Ball_and_Beam_CAD_model



Систем за балансирање лопте на шини – Слика 2.4

2.2. Израда пројектованих делова

Аддитивна производња, позната и као 3D штампа или аддитивна израда, је процес израде тродимензионалних објеката додавањем материјала у слојевима, уместо сечења или обликовања материјала.

У адитивној производњи, машине користе податке о CAD моделу да би додавале материјал у слојевима да би се израдио објекат, за разлику од традиционалних начина производње где машине уклањају материјал из блока или компоненте да би се добио жељени део.

Аддитивна производња, у даљем тексту 3D штампа, има многе предности у односу на традиционалну производњу, укључујући већу сложеност дизајна, брзину производње, ниског трошкова производње малих серија или јединичних производа, могућност производње на захтев и низ других предности.

Због свих ових предности 3D штампач је нашао велику примену у различитим индустријама као што су авио и аутомовлиска, медицинска и многе друге индустрије, али поред индустрије 3D штампач се је постао и стандардна опрема многих лабораторија, као и радионица разних хобиста.

Постоје различите врсте 3D штампања, а неке од њих јесу фузија ласера, слој по слој (FDM) и селективна ласерска синтеризација (SLS). Која ће се врста 3D штампе користити зависи од жељеног материјала, величине објекта и других фактора.

За потребе мастер рада, сви пројектовани, не стандардни, делови су израђен на 3D штампачу Ender 3 Pro (слика 2.5) компаније Creality. Овај штампач користи технологију слој по слој (FDM), а његове основне карактеристике можете видети у табели 2.1.



Ender 3 Pro - Слика 2.5

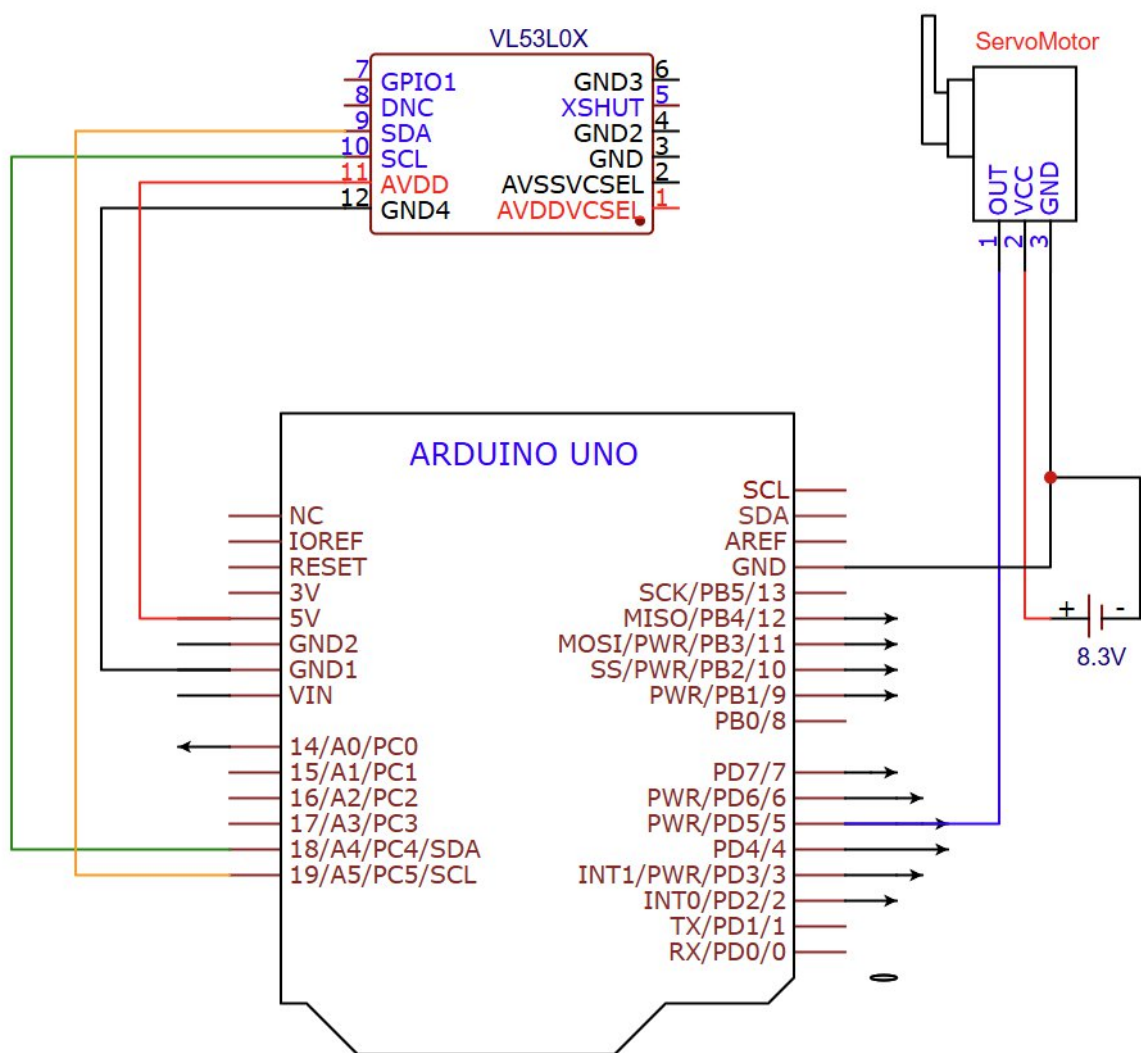
Назив штампача	Ender 3 Pro
Запремина штампања	220x220x250mm
Габарити штампача	440x410x465mm
Пречник млазнице	0.4mm
Максимална температура млазнице	250°C
Максимална температура подлоге за штампање	110°C
Висина слоја	0.1–0.4mm
Повезвање	USB, SD картица
Материјали за штампање	PLA, ABS, PETG, TPU

Техничке карактеристике штампача Ender 3 Pro – Табела 2.1

Припрема делова за штампу је урађена у софтверу UltiMaker Cura. Сви делови су израђени од PLA пластике и са истом прецизношћу и за ту потребу температура подлоге за штампу је подешена на 55°C, температура млазнице на 215° и висина слоја на 0.2mm. Параметри попут дебљине зида, типа и густине испуне разликују се од дела до дела.

За израду свих делова, узимајући у обзир и шкартове, било је потребно 96 сати. Док је сам систем је склопљен следећим редом:

1. Састављање носача шине
2. Састављање шине и монтирање сензора на исту
3. Позиционирање и фиксирање носача шине и носача серво мотора
4. Спајање шине и носача шине
5. Монтирање серво мотора на носач истог
6. Спајање шине и серво мотора полугом
7. Позиционирање и фиксирање носача за микро контролер
8. Монтирање микроконтролер на носач за исти
9. Ожичавање система пратећи шему са слике 2.6



Шема повезивања система за балансирање лопте на шини – Слика 2.6

3. Математички модел система за балансирање лопте на шини

Дефиниција 1 Аутоматско управљање је мултидисциплинарна грана инжењерства усмерена на пројектовање, анализу и имплементацију система који контролишу понашање динамичких процеса. Главни задаци ове гране јесу идентификација система, која обухвата моделирање система, процену параметара и валидацију модела, и управљање системом, чији је циљ да систем учини стабилнијим, одзивнијим и прецизнијим.

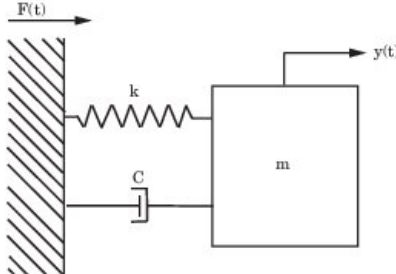
На основу дефиниције 1 лако је закључити да су наредна два корака идентификација система и конструисање алгоритма за управљање. С обзиром да је познавање математичког модела система предуслов за конструисање алгоритма за управљање, онда је и јасно да ће у овом поглављу главни циљ бити идентификација самог система, односно одређивање његовог математичког модела.

Дефиниција 2 Модел физичког система је идеализовани, замишљени, систем који задржава особине стварног система битне за његову анализу. Модел физичког система често буде

Дефиниција 3 Математичко модел система је формални математички опис модела физичког система који успоставља једнозначну везу између излазних и улазних величина за произвољне промене улазних величина и за произвољне почетне услове, а исказан је помоћу математичких симбола, операција и релација.

Ако се претпостави да математички модел довољно тачно описује физички систем, тј. модел физичког система, и да представља његов веродостојан опис, онда он садржи све информације о физичким особинама система. Тада се проучавање тог физичког система могу извршити на његовом математичком моделу.

У колико су динамика система и његови параметри познати, до математичког модела система може се доћи користећи законе физике. Претпоставимо да је систем представљен на следећој слици:



Систем пригушеног осцилаторног кретања тела – Слика 3.1

и да су познати сви параметри овог система C , k и m . Користећи знање Механике може се закључити да овај систем преставља пригушено осцилаторно кретање тела и да се он може описати следећом једначином:

$$(3.1) \quad m \frac{d^2 y}{dt^2} + c \frac{dy}{dt} + ky(t) = F(t).$$

Претходна једначина јесте математички модел система са слике која се даље може користити за анализу система. Овакав систем се у жаргону назва белом кутијом, јер су сви његови параметри познати, као и његова динамика.

Супротно од беле кутије, јесте црна кутија, односно модели чија динамика је непозната. У оваквим случајевима идентификација система, као засебна област аутоматског управљања, долази до изражаја.

Приликом одређивања математичког модела система непознате динамике, први корак јесте прикупљање информација о систему. Овај корак подразумева извођење експеримената над системом и то тако што се на систем побуђује одређеним улазним величинама, а сам излаз система мери.

Следећи корак би био одабир структуре модела. За структуру система може се одабрати преносна функција са n бројем полова и m бројем нула, диференцијална једначина, ауто-регресивни модел, неуронска мрежа или било који други модел.

Након што су прикупљене информације о систему и одабрана структура модела, потребно је естимирати параметре и валидирати модел. Приликом валидације система треба користити нов сет података, где ће се улази тог сета искористити за пободу модела, а затим излази модела и стварног система поредити.

Када се све сумира, идентификација система се састоји од експеримента и прикупљање улазно-излазних података система, одабира структуре модела, естимације параметара и валидацију модела; али *“Како изабрати праву структуру система?”* и *“Шта радити ако након валидације критеријуми нису задовољени?”*. Ствар јесте у томе што правила приликом одабира структуре модела нема и ако након валидације модела критеријуми нису задовољени, потребно је вратити се на “почетак” и изабрати нову структуру или модификовати претходно изабрану, а за тим поново естимирати параметара и валидирати модел. Идентификација система јесте процес који захтева више итерација да би се постигли жељени резултати, али и процес у којем искуство може бити од великог значаја.

Дефиниција 4 Идентификација система је методологија за конструисање математичког модела динамичког система користећи мерене улазне и излазне сигнале система.

Дефиниција 5 Процес идентификације система састоји се из:

- Мерење улазних и излазних сигнала система у временском или фреквентном домену,
- Одабира структуре система,
- Естимације параметара за изабрану структуру модела и
- Валидације самог модела.



Алгоритам за идентификације система – Слика 3.2

Поред, такозваних, белих и црних кутија, постоје и сиве кутије. То су системи чија је динамика добрим делом позната, али не и параметри, или барем не сви. У оваквим случајевима се, такође, користи, претходно објашњена, метод за идентификацију система.

Претпоставимо поново да је задатак одређивање математичког модела са слике 3.1 и да је једини познати параметар маса тела. Пратећи алгоритам за идентификацију система први корак јесте прикупљање информација о систему. Након тога потребно је одабрати конструкцију модела. Узимајући у обзир да је динамика система за конструкцију модела може се изабрати једначина (3.1). Затим би се неком од метода минимизације естимирали непознати коефицијенти s и k и на крају, користећи нови сет улазно-излазних података, валидирао модел.

За валидирање модела често се упоређују стварни одзив и одзив модела, али се за боље разумевање самог система користи и анализа резидуала.

Дефиниција 6 Анализа резидуала је статистичка техника која се користи у идентификацији система како би се проценила тачност и квалитет модела анализом разлике између стварног излаза система и излаза предвиђеног моделом (3.2.29).

$$(3.2) \quad r(t) = y_{\text{измерено}}(t) - y_{\text{предвиђено}}(t)$$

Ове разлике, познате као резидуи, представљају необјашњени део излаза који модел не хвата. Анализа резидуа помаже да се утврди да ли модел добро захвата основну динамику система или има систематске грешке.

Дефиниција 7 Аутокорелација резидуала се односи на корелацију резидуа са самим собом на различитим временским интервалима. Ако постоји значајна аутокорелација, то сугерише да модел не хвата у потпуности понашање система.

Дефиниција 8 Унакрсна корелација резидуала проверава да ли су резидуи у корелацији са прошлим улазима. Ако су резидуи у корелацији са улазом, то сугерише да модел не успева да ухвати неку важну динамику система под утицајем улаза.

Посматрајући систем за балансирање лопте на шини се састоји од лопте на шини која је системом полуга спојена на серво мотор, где је серво мотор извршни орган, а лопта на шини објект система. Математички модел објекта може се извести користећи законе кинематике и динамике, док је математички модел извршног органа, узимајући у обзир да ова информација није дата у техничког документацији, потребно наћи користећи један од метода за идентификацију система.

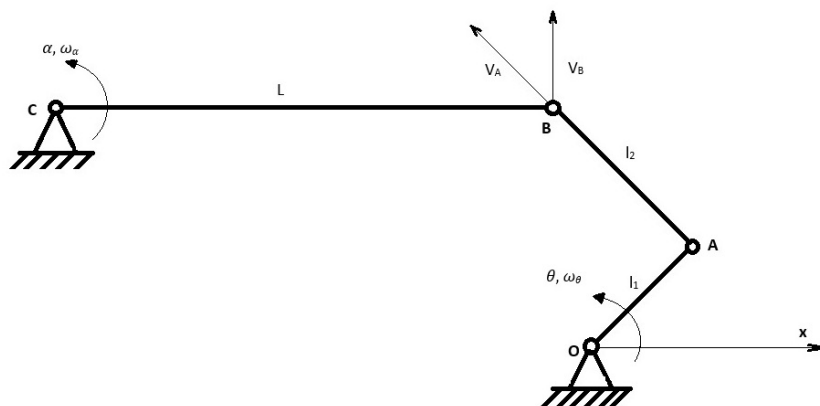
3.1. Једначина кретања лопте на шини

Тумачећи конфигурацију система са слике 1.2 може се доћи до закључка да брзина и смер кретања лопте на шини зависи од промене нагибног угла шине, односно промене угла излазног вратила серво мотора.

3.1.1. Једначина промене нагибног угла шине

Једначина промене нагибног угла шине може се наћи решавањем проблема приказаног на слици 3.3, где су:

- l_1 – дужина краће полуге,
- l_2 – дужина дуге полуге,
- L – растојање између ослоња шине и гоњеног краја шине,
- α – нагибни угао шине,
- ω_α – угаона брзина шине,
- θ – угао излазног вратила серво мотора и
- ω_θ – угаона брзина серво мотора.



Систем полуга система за балансирање лопте на шини – Слика 3.3
Брзине V_A и V_B су представљене у следећем облику:

$$(3.1.1) V_A = \omega_\theta l_1,$$

$$(3.1.2) V_B = \omega_\alpha L.$$

Користећи теорију сличности троуглова може се закључити следеће:

$$(3.1.3) V_B = V_A \cos \theta.$$

Комбиновањем једначина (3.1.1), (3.1.2) и (3.1.3) добија се:

$$(3.1.4) \omega_\alpha L = \omega_\theta l_1 \cos \theta$$

и ако се настави са даљим разлагањем претходне једначине, може се доћи до следеће једначине:

$$(3.1.5) \frac{d\alpha}{dt} = \frac{l_1}{L} \frac{d\theta}{dt} \cos\theta.$$

Интеграљењем једначе (3.1.5) довија се:

$$(3.1.6) \alpha = \frac{l_1}{L} \sin\theta + C.$$

Ако важе следеће претпоставке:

$$(3.1.7) \sin\theta \approx \theta \text{ за веома мале вредности угла } \theta \text{ и}$$

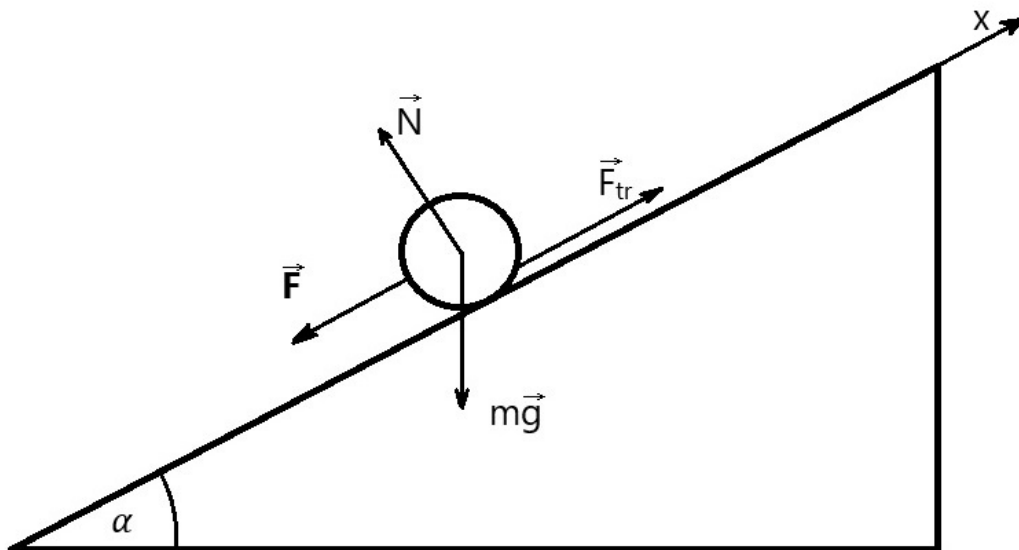
$$(3.1.8) \alpha_0 = 0 \rightarrow C = 0.$$

Онда из (3.1.8) и (3.1.9) следи једначина промене нагибног угла шине у зависности од промене угла вратила на серво мотору:

$$(3.1.9) \alpha = \frac{l_1}{L} \theta.$$

3.1.2. Једначине кретања лопте на шини и преносна функција објекта

За потребе налажења једначине кретање лопте на шини потребно је посматрати случај када је нагибни угао шине различит од нуле ($\alpha \neq 0$). Узимајући претходну претпоставку у обзир, проблем се може поистоветити са проблемом кретање лопте по стрмој равни (слика 3.4).



Кретање лопте по стрмој равни – Слика 3. 4

За решавање овог проблема потребно је кренути од једначине суме свих сила на тело:

$$(3.1.10) \quad \vec{F} = m\vec{g} + \vec{N} + \vec{F}_{tr}.$$

Узимајући у обзир да се за овај пројекат користити лоптицу за стони тенис, због њене мале масе може се занемарити силу потиска, а са њом и силу трења. Стога једначина (3.1.11), пројектована на x осу, имаће следећи облик:

$$(3.1.11) \quad F(t) = mg \sin\alpha(t),$$

односно, та исту једначина може се записати и у следећем облику:

$$(3.1.12) \quad \left(\frac{I}{R^2} + m\right)\ddot{x}(t) = -mg \sin\alpha(t).$$

Где су:

- I – момент инерције лопте,
- R – полупречник лопте,
- m – маса лопте,
- $\ddot{x}(t)$ – убрзање лопте,
- g – гравитационо убрзање,
- α – нагибни угао шине.

Ако претпоставимо да је:

$$(3.1.13) \quad \sin\alpha \approx \alpha \text{ за веома мале вредности угла } \alpha,$$

тада једначину (3.1.12) можемо написати у следећем облику:

$$(3.1.14) \quad \left(\frac{I}{R^2} + m\right)\ddot{x}(t) = -mg \alpha(t).$$

Односно, када се угао α замени његовом релацијом из једначине (3.1.10), тада се добија:

$$(3.1.15) \quad \left(\frac{I}{R^2} + m\right)\ddot{x}(t) = -mg \frac{l_1}{L} \theta(t),$$

односно:

$$(3.1.16) \quad \ddot{x}(t) = -\frac{mgl_1}{L\left(\frac{I}{R^2} + m\right)} \theta(t).$$

До преносне функције објекта долази се пребацивањем једначине (3.1.14) из временског домена у комплексни уз помоћ Лапласове трансформације:

$$(3.1.17) \quad s^2 x(t) = -\frac{mgl_1}{L\left(\frac{I}{R^2} + m\right)} \theta(s).$$

Након сређивања једначине (3.1.15) добија се следећа преносна функција:

$$(3.1.18) \quad W_O(s) = \frac{x(s)}{\theta(s)} = -\frac{mgl_1}{L\left(\frac{I}{R^2} + m\right)} \frac{1}{s^2},$$

која након замене симбола њиховим вредностима, добијамо облик:

$$(3.1.19) \quad W_O(s) = -\frac{0.80541872}{s^2}.$$

3.2. Математичко модел серво мотора једносмерне струје

Одређивању математичког модела серво мотора приступљено је на два начина. Први начин јесте идентификацијом методом сиве кутије, где је на основу општих познатих информација о серво мотору формирана структура модела. Док за други начин искоришћена идентификација система методом црне кутије. Пратећи алгоритам за идентификацију система у овом поглављу су обрађене следеће тебе:

- прикупљање података помоћу експеримента,
- одабир конструкције модела,
- естимација параметара модела и
- валидацију модела.

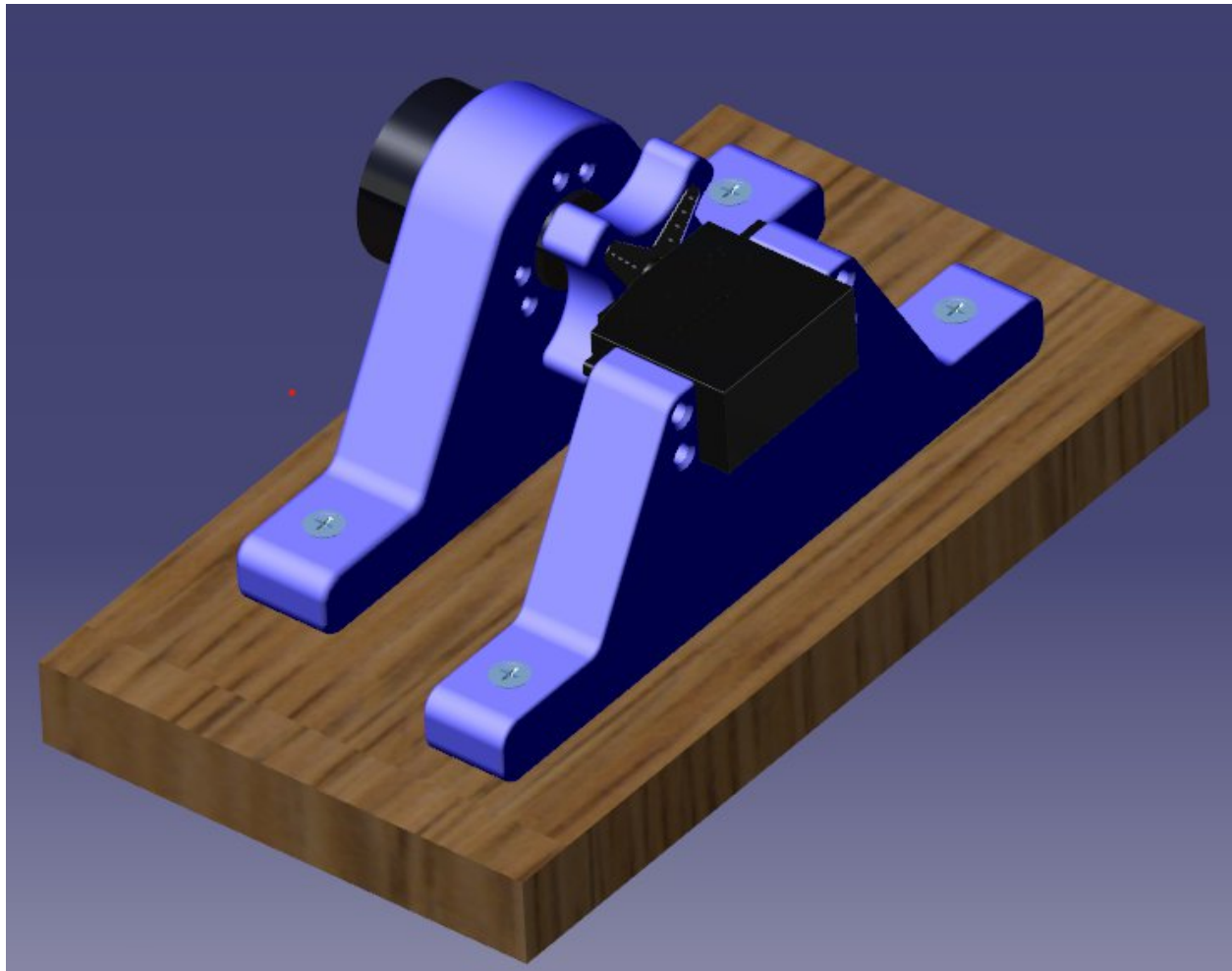
3.2.1. Прикупљање података помоћу експеримента

3.2.1.1. Пројектовање и израда платформе за испитивање

За потребе експеримента било је потребно конструисати платформу за испитивање. До решења је дођено одговорима на следећа питања:

- Шта је циљ експеримента?
 - Циљ експеримента јесте прикупљање улазних и излазних величина система.
- Шта је улазни величина серво мотора, а шта излазна?
 - За улазну величину серво мотора је изабран жељени угао излазног вратила мотора, који је уз помоћ једноставне формуле трансформисан у ширинско модуларни пулс (стварна улазна величина), док је излаз мотора промена угла излазног вратила серво мотора.
- Како и чиме мерити промену излазне величине серво мотора?
 - Промена излазне величине серво мотора, односно промена угла излазног вратила серво мотора, мерена је ротационим енкодером LPD3806. Нешто више о овом енкодеру може се пронаћи у додатку А на крају овог рада.

Водећи се одговорима на претходна питања испројектована и израђена је платформа за испитивање која је поседује дрвено постоље, за које су вијцима причвршћени носачи серво мотора и ротационог енкодера. Док су вратила мотора и ротационог енкодера спрегнута наменски пројектованом спојницом. Оба носача и спојница су израђени на 3D штампачу, а финални производ је приказан на слици 3.5.



Платформа за испитивање серво мотора – Слика 3.5

3.2.1.2. Поставка експеримента и прикупљање података

У претходном делу је споменуто да је главни циљ овог експеримента прикупљање улазних и излазних величина серво мотора које ће се касније искористити за естимацију параметара модела. На основу циља може се закључити да експеримент треба да садржи следеће кораке:

- побуду серво мотора одређеном улазном величином,
- читавање промене излазне величине и
- чување податка о променама улазних и излазних величина.

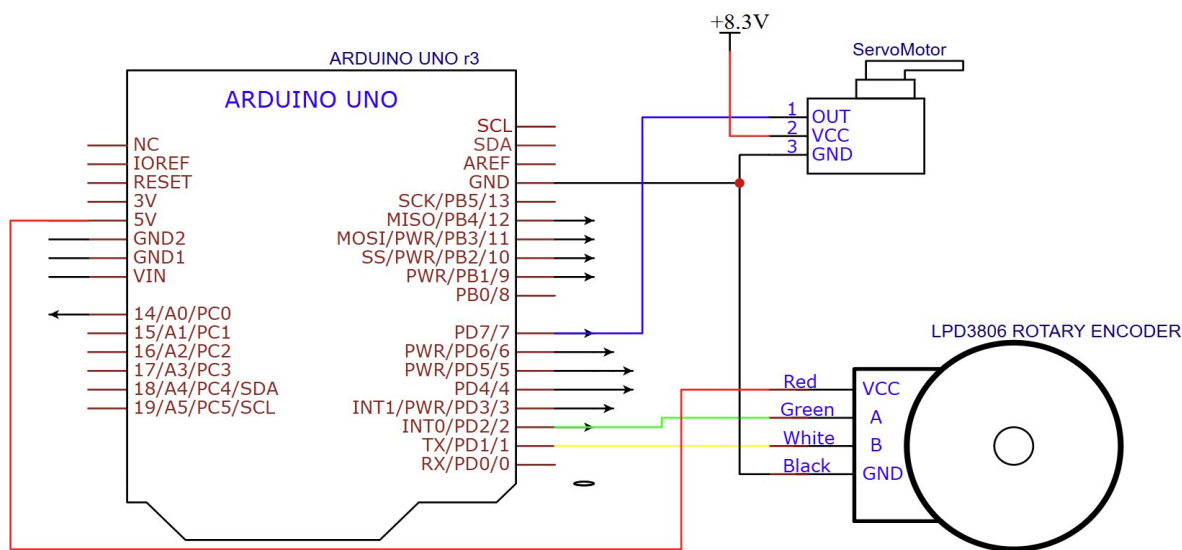
На први поглед задатак делује једноставан, међутим до финалног решење дођено је тек у трећој итерацији.

Итерација 1

У првој итерацији, идеја је била да се и за побуду серво мотора и за читавање промене угла излазног вратила серво мотора помоћу ротационог енкодера, користи један микроконтролер (Arduino Uno). У главној петљи се налазио код који је слао одређени ширински модуларни пулс серво мотору, док се у прекидној петљи налазио код који би читавао вредности са енкодера.

Главна петља у је петља која ради непрекидно, извршавајући редовне задатке у секвенцијалном редоследу. Насупрот томе, прекидна петља се активира одређеним догађајима, омогућавајући контролеру да привремено заустави главну петљу и брзо одговори на хитне, временски осетљиве задатке пре него што се врати у главну петљу.

На основу претходно објашњене идеје и објашњене разлике између главне и прекидне петље, може се закључити да је систем често улазио у прекидну петљу, а да су последице тога испрекидан ширинско модуларни сигнал који је слат мотору, као и јако вибрирање мотора.



Шема за повезивања у првој итерацији – Слика 3.6

Итерација 2

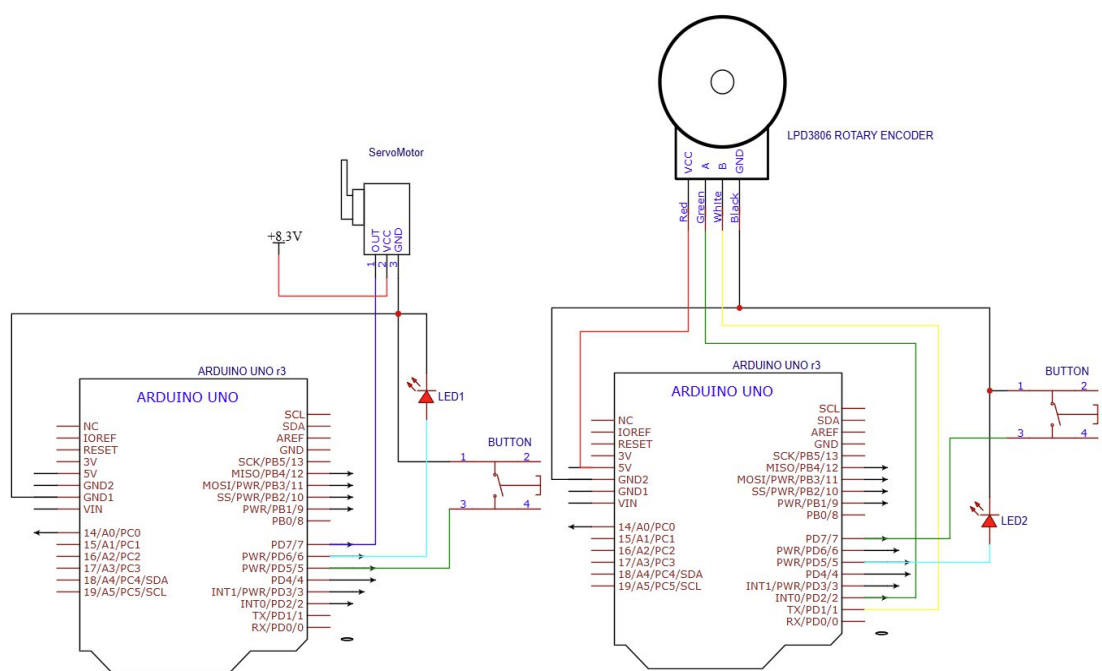
На основу искуства из прве итерације, у другој итерацији решено је да се користе два микроконтролера (Arduino Uno). Где би један служио за побуђивање серво мотора, а други за читавање ротационог енкодера. Поред ротационог енкодера и серво мотора,

оба микроконтролера су поседовала по тастер и лед диоду (шема повезивања приказана је на слици 3.7)

Лед диода на контролеру, који је служио за покретање серво мотора, је светлела све до притиска тастера. Након притиска тастера, лед диода би се угасила, затим би био извршен експеримент, где мотор ротира излазно вратило за одређени угао и након кратке паузе га враћа у почетни положај. Након извршеног експеримента лед диода би поново засијала.

Сличан алгоритам је био и на другом микроконтролеру, али је битно напоменути да је део алгоритма за читавање и чување вредности са енкодера и даље био у прекидној петљи. Што се тиче алгоритма у главној петљи, након притиска тастера диода би засијала, затим би се сви прикупљени подаци слали преко серијског порта на рачунар и након завршеног слања, диода би се угасила сигнализирајући да су сви подаци послати и да је други контролер спреман за нови експеримент.

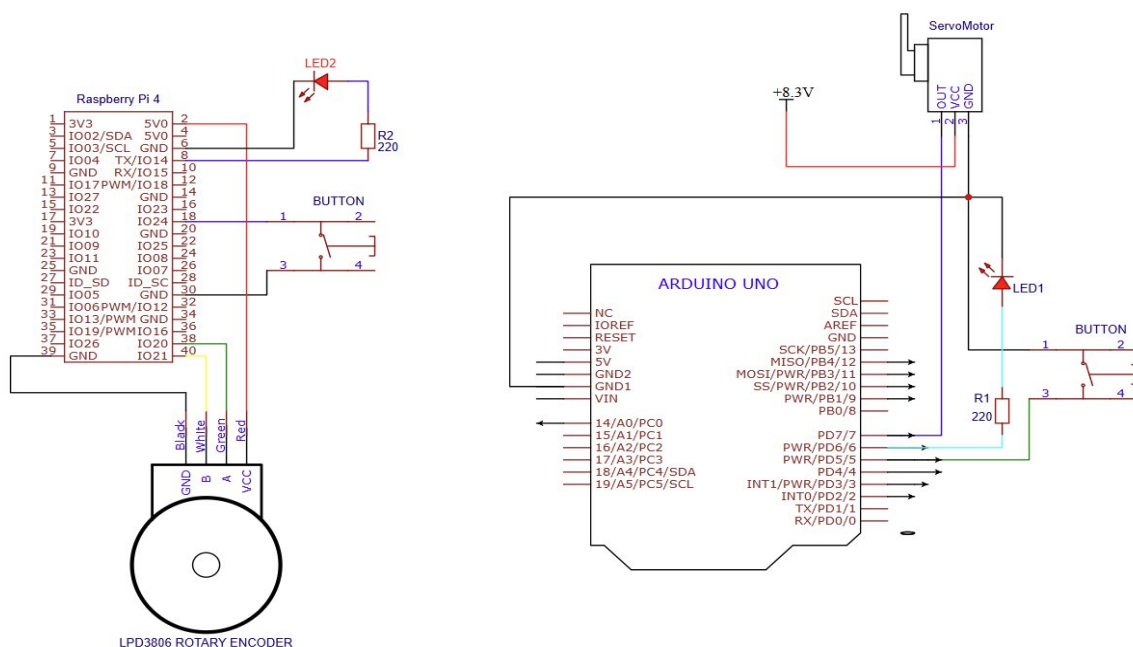
У овој итерацији пронађена су два проблема. Један од проблема јесте био нестабилан рад серво мотора услед нестабилног напајања. Док је други проблем било слање прикупљених. Због количине прикупљених података на самом почетку слања података микроконтролер би се сам од себе ресетовао.



Шема повезивања у другој итерацији – Слика 3.7

Итерација 3

У трећој итерацији, микроконтролер (Arduino Uno) који смо користили за читавање вредности са енкодера је замењен микропроцесором (Raspberry Pi) и проблем са напајањем серво мотора је решен. Док су алгоритми за покретање серво мотора и читавање вредности са ротационог енкодера добрим делом остали исти. У наставку су приложене шема за повезивање (слика 3.8), као и скрипте за прикупљање података и покретање серво мотора.



Шема повезивања у трећој итерацији – Слика 3.8

Као што је напоменуто, за мерење промене угла излазног вратила серво мотора употребљен је ротациони енкодер који је повезан на микропроцесор Raspberry Pi. Поред енкодера на овај контролер повезани су тастер и лед диода. Лед диода је индикатор која светли када је у току чување података који се чували након што се експеримент заврши и притисне тастер у датотеку json формата. Цео алгоритам је приказан на слици 3.9.

Скрипта за ове намене писана је у програмском језику Python, док су следеће библиотеке искоришћене:

- `pigpio` – библиотека која омогућава коришћење и управљање улазно-излазних пинова генералне намене микропроцесоре Raspberry Pi,
- `time` – библиотека за читавање времена,
- `json` – библиотека за рад са подацима и датотекама у json формату и

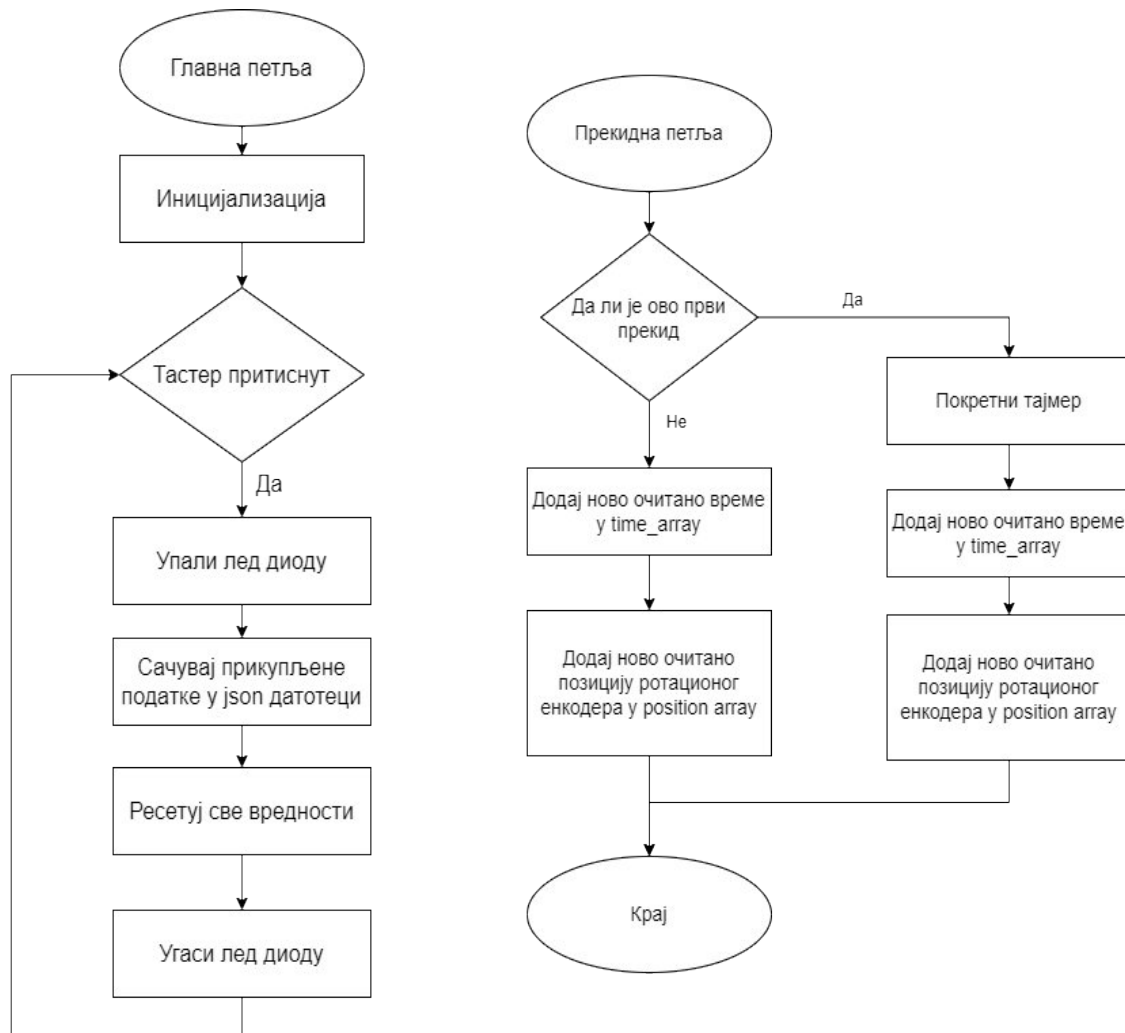
- `rotary_encoder` – библиотека која омогућава читавање вредности са ротационог енкодера.

Треба напоменути да је скрипта сачињена и од следећих функција:

- `callback` – функција прекидне петље која читава вредност ротационог енкодера и повећава, односно смањује, бројач,
- `collect_data` – функција која је у директној вези са `callback` функцијом и служи за смештање прочитаних података у привремене низове,
- `save_data` – функција за чување прочитаних података у датотеку `json` формата,
- `var_reset` – функција која служи за ресетовање и брисање свих бројача и низова које користимо за привремено смештање прочитаних вредности,
- `debounce` – функција која служи за решавање проблема са поскакивањем сигнала тастера,
- `button_press_handler` – ова функција се можда може и назвати главном функцијом главне петље, јер она у себи садржи наредбе и функције за паљење и гашење лед диоде, чување прочитаних података и ресетовање привремених податка, а притом је у директној спрези са `debounce` функцијом.

Проблем са поскакивањем сигнала (енг. `bouncing signal problem`) је карактеристичан проблем за електро механичке компоненте попут тастера.

Приликом притиска тастера два метална дела долазе у контакт како би затворили електрично коло. У идеалном случају, та два дела се спајају у једном тренутку (у трену), док у стварности та два дела долазе у контакт неколико пута пре коначног спајања. Ово узрокује вишеструка окидања која контролер поистовећује са вишеструким притиском тастера. Овај ефекат се дешава и приликом отпуштања тастера.



Алгоритам за читавање промене угла излазног вратила серво мотора и чување прочитаних података – Слика 3.9

```

import pigpio
import time as t
import json
from Scripts import rotary_encoder

```

```

BUTTON_PIN = 23 # GPIO PIN
LED_PIN = 14 # GPIO PIN
channel_A = 20 # GPIO PIN
channel_B = 21 # GPIO PIN

```

```

position = 0
position_array = []
time_array = []
time_start = 0
test_count = 0

```

```

def save_data():
    global test_count
    path = "/Master Rad/Test_data/Test_"

    data = {
        "Counts": position_array,
        "Time": time_array,
    }
    with open(path + str(test_count) + ".json", "w") as f:
        json.dump(data, f)
    test_count += 1

def var_reset():
    global position, position_array, time_array
    position = 0
    position_array = []
    time_array = []

def collect_data():
    global time_start
    if not position_array:
        time_start = t.monotonic_ns()
        time_array.append(0)
        position_array.append(position)
    else:
        time_array.append(t.monotonic_ns() - time_start)
        position_array.append(position)

def callback(way):
    global position
    position += way
    collect_data()

def debounce(func, wait_time=0.01):
    last_time = 0

    def debounced_func(*args, **kwargs):
        nonlocal last_time
        current_time = t.time()
        if current_time - last_time >= wait_time:
            last_time = current_time
            return func(*args, **kwargs)
    return debounced_func

pi = pigpio.pi()
decoder = rotary_encoder.decoder(pi, channel_A, channel_B, callback)

pi.set_mode(BUTTON_PIN, pigpio.INPUT)
pi.set_pull_up_down(BUTTON_PIN, pigpio.PUD_UP)
pi.set_mode(LED_PIN, pigpio.OUTPUT)

@debounce
def button_press_handler():
    pi.write(LED_PIN, 1)
    t.sleep(1)
    print(max(position_array))

```

```

save_data()
var_reset()
pi.write(LED_PIN, 0)

try:
    while True:
        if not pi.read(BUTTON_PIN):
            button_press_handler()
            t.sleep(0.01) # Small sleep to avoid busy-waiting

except KeyboardInterrupt:
    print("Exiting...")
finally:
    decoder.cancel()
    pi.stop()

```

На микроконтролер Arduino Uno повезани су мотор, лед диода и тастер. Лед диода сија све док се не притисне тастер који ту исту диоду гаси и прелази на извршавање експеримент из одређене групе, односно покретање test функције. Скрипта за Arduino Uno је писана у програмском језику C у наменском развојном окружењу за овај контролер (Arduino IDE). За писање ове скрипте поред стандардних функција које нуди стандардна Arduino.h библиотека, коришћене су и библиотека која нам омогућава лако управљање серво мотора (Servo.h), као и Time.h библиотека. Скрипта се састоји од следећих функција:

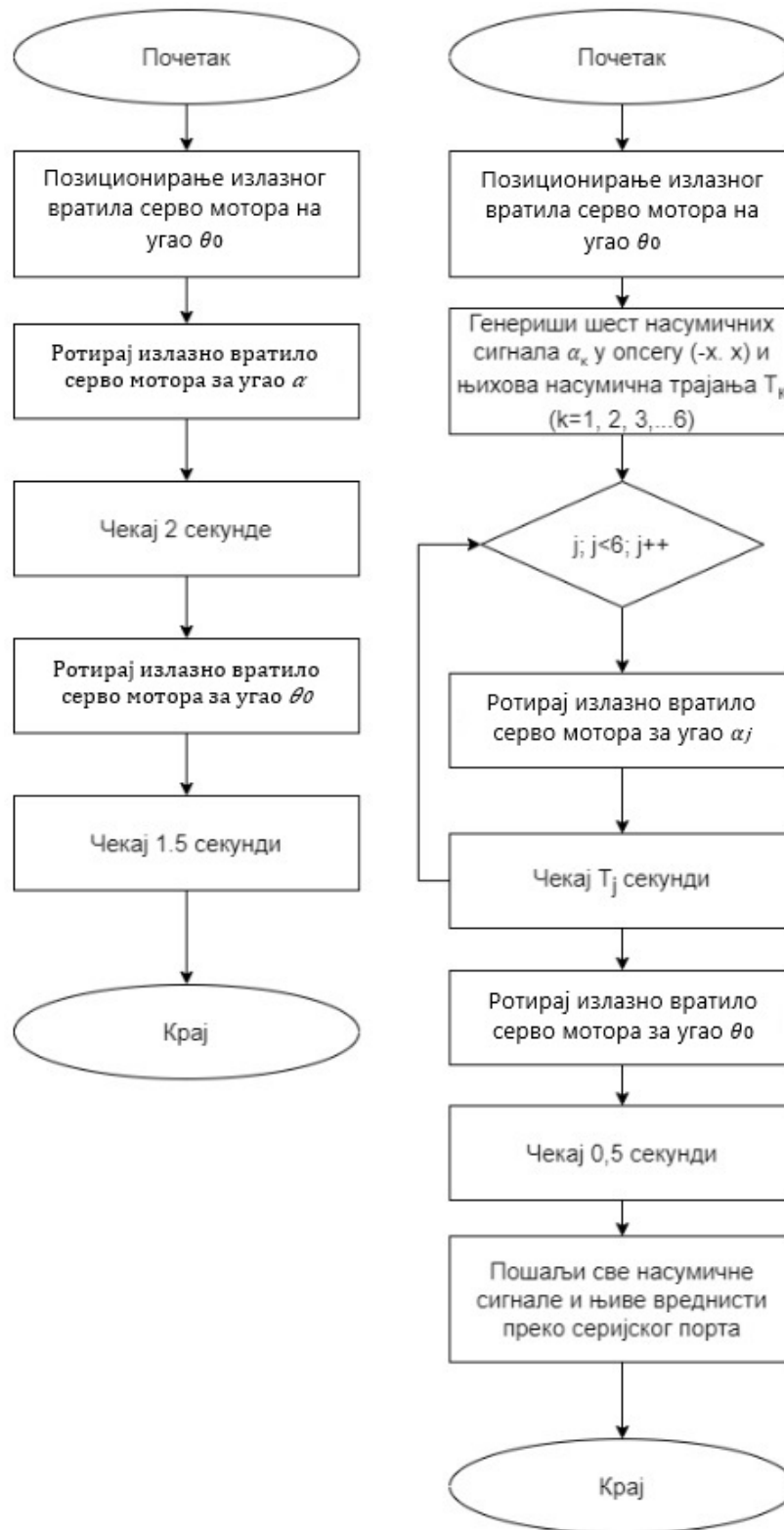
- setup – ова функција је задужена за иницијализацију пинова,
- loop – је функција главне петље која би гасила лед диоду приликом притиска на тастер и затим покретала један од тестова,
- test_1 – функција за прву групу експеримената и,
- test_2 – функција за другу групу експеримената.

Алгоритам функције test_1 приказан је на слици 3.10 (лево). Скуп углова који су коришћени за овај експеримент јесте $\alpha \in (-45^\circ, -30^\circ, -20^\circ, -10^\circ, 10^\circ, 20^\circ, 30^\circ, 45^\circ, 60^\circ)$. Обзиром да је за потребе овог мастер рада изабран хоби серво мотор, који има позициону толерирају од $\pm 3^\circ$, експеримент је поновљен пет пута за сваки угао α .

Функција test_2 је мало сложенија од претходне функције. Ова функција је давала шест насумичних сигнала (углова) са насумично одређеним паузама. Након шестог сигнала, контролер би послао седми сигнал, који је за циљ имао да врати излазно вратило серво мотора у почетни положај. Користећи функцију test_3 изведено је по петнаест експеримената за две различите конфигурације.

- Конфигурација 1
 - опсег насумичних углова – од -45° до 45° ,
 - $\theta_0 = 0^\circ$,
- Конфигурација 2
 - опсег насумичних углова – од -30° до 30° ,
 - $\theta_0 = 0^\circ$.

Алгоритам ове функције је приказан на слици 3.10 (десно).



Алгоритми за извршавање експеримената; алгоритам лево – test_1; алгоритам десно – test_3 – Слика 3.10


```

#include <Servo.h>
#include "Timer.h"

int test_angles[8] = {-45, -30, -20, -10, 10, 20, 30, 45};
int zero_pwm = map(0, -135, 135, 500, 2500);
unsigned int time_var = 0;

int i=0;
int j=0;

Servo my_servo;
Timer my_timer(MICROS);

void test_1()
{
    if(i==8)
    {
        Serial.println("The end of test");
        i++;
    }
    else if (i<8)
    {
        int angle;
        int pwm = map(test_angles[i], -135, 135, 500, 2500);
        Serial.print("Test_1 no. ");
        Serial.print(i+1);
        Serial.print(". ");
        Serial.print(j);
        Serial.println(" ");
        // Print Test Angle
        Serial.print("Test angle: ");
        Serial.println(test_angles[i]);
        Serial.print("Test pwm: ");
        Serial.println(pwm);
        // Execute Test
        my_timer.start();
        my_servo.writeMicroseconds(pwm);
        delay(2000);
        time_var = my_timer.read();
        my_servo.writeMicroseconds(zero_pwm);
        delay(1500);

        // Print Execution Time
        Serial.print("Execution time: ");
        Serial.println(time_var);
        my_timer.stop();

        // Handle Counters
        j++;
        if(j>=5)
        {
            i++;
            j = 0;
        }
    }
}

```

```

void test_2()
{
  if(i==15)
  {
    Serial.println("The end of test");
    i++;
  }
  else if (i<15)
  {
    int random_angles[6];
    int random_pwms[6];
    int random_delay[6];

    // Generate data for test
    for(j=0;j<6;j++)
    {
      random_angles[j] = random(-45, 45); // Test data 2-1
      // random_angles[j] = random(-30, 30); // Test data 2-2
      random_pwms[j] = map(random_angles[j], -135, 135, 500, 2500);
      random_delay[j] = random(10, 100) * 10;
    }

    // Execute Test
    my_timer.start();
    for(j=0;j<6;j++)
    {
      my_servo.writeMicroseconds(random_pwms[j]);
      delay(random_delay[j]);
    }
    my_servo.writeMicroseconds(zero_pwm);
    delay(500);
    time_var = my_timer.read();

    Serial.print("Test_2 no. ");
    Serial.print(i+1);
    Serial.println(" ");

    // Print Execution Time, Angles and Delays
    for(j=0;j<6;j++)
    {
      Serial.print("angle");
      Serial.print(": ");
      Serial.println(random_angles[j]);
      Serial.print("delay");
      Serial.print(": ");
      Serial.println(random_delay[j]);
    }
    Serial.print("Execution_time: ");
    Serial.println(time_var);
    my_timer.stop();

    // Handle Counters
    i++;
  }
}

```

```

void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  pinMode(2, INPUT_PULLUP); // Button Pin
  pinMode(7, OUTPUT);       // LED Pin
  my_servo.attach(5);       // Servo Pin

  my_servo.writeMicroseconds(zero_pwm);
  Serial.print("Zero pwm: ");
  Serial.println(zero_pwm);
}

void loop() {
  if(digitalRead(2))
  {
    digitalWrite(7, HIGH);
  }
  else
  {
    delay(1500);
    digitalWrite(7, LOW);
    test_2();
  }
}

```

3.2.1.3. Обрада података

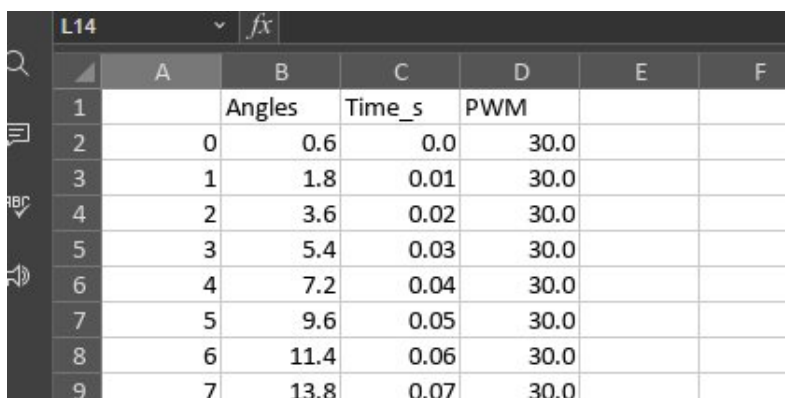
Након прикупљања података било је потребе за обрадом истих. Сви прикупљени подаци су у json формату и садрже два низа. Један низ садржи позиције у датим тренуцима, а други низ садржи временске тренутке очитаних позиција у нано секундама.

Добар део обраде података је аутоматизован скрипта написаних у Python програмском језику. Скрипта `Data_processing_1.py` је аутоматизовала обраду податка прикупљене експеримента који су покренути функцијом `test_1`. Позиције које су дате сумом очитаних пулсева ротационог енкодера су претворене у углове, време је из нано секунди претворено у секунде и додате су и вредности улазних сигнала. Узимајући у обзир:

- паузу од две секунде, која је карактеристична за прву и другу групу експеримената,
- да је излазно вратило серво мотора стизало до жељеног угла пре него што две секунде истекну и
- да су бележене само промене угла излазног вратила серво мотора.

Било је лако закључити да датотеке, у којима су сачуване мерене вредности, не садрже податке од тренутка када вратило достиже жељену позицију до почетног тренутка када се вратило враћа у почетни положај. Овај проблем је такође решен уз помоћ скрипте.

Након сређивања података помоћу скрипте, сви сетови података су прегледани и по потреби ручно исправљени.

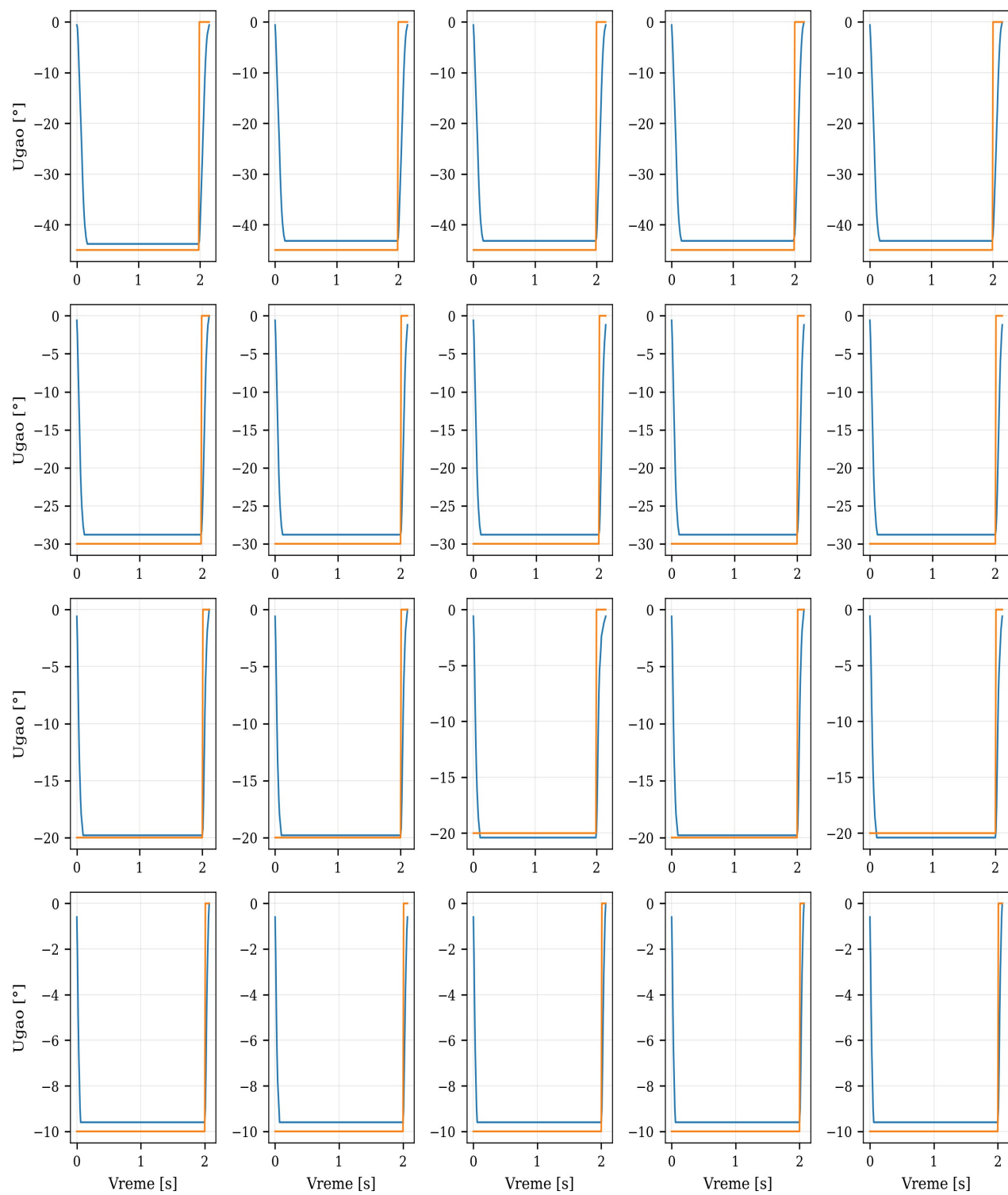


	A	B	C	D	E	F
1		Angles	Time_s	PWM		
2	0	0.6	0.0	30.0		
3	1	1.8	0.01	30.0		
4	2	3.6	0.02	30.0		
5	3	5.4	0.03	30.0		
6	4	7.2	0.04	30.0		
7	5	9.6	0.05	30.0		
8	6	11.4	0.06	30.0		
9	7	13.8	0.07	30.0		

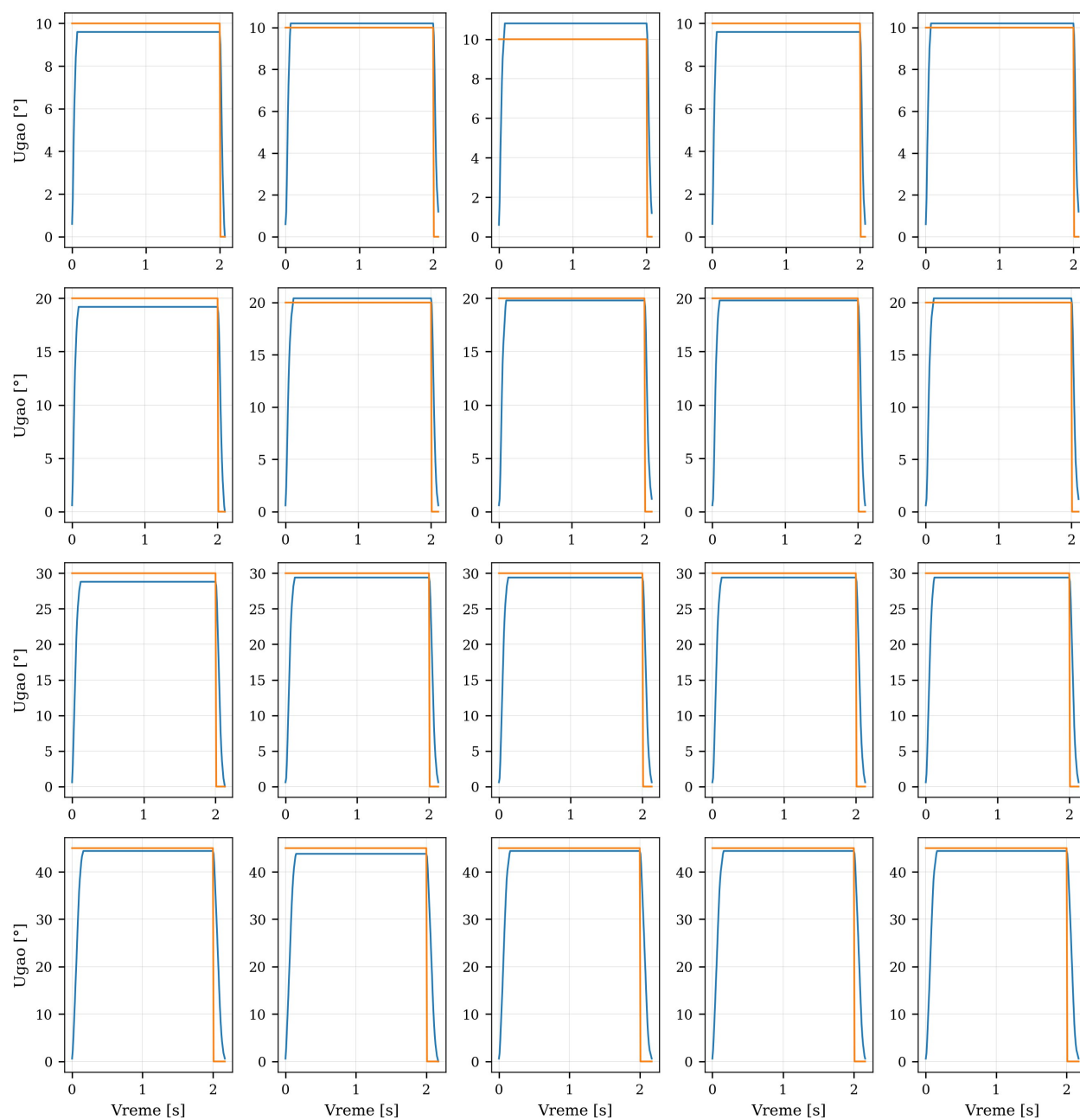
Подаци након сређивања – Слика 3.11

Претходно објашњен процес је примењен и на податке прикупљене извршавањем трећег експеримента (`test_3`). За потребе сређивање ових података коришћене су скрипта `Data_processing_2.py`.

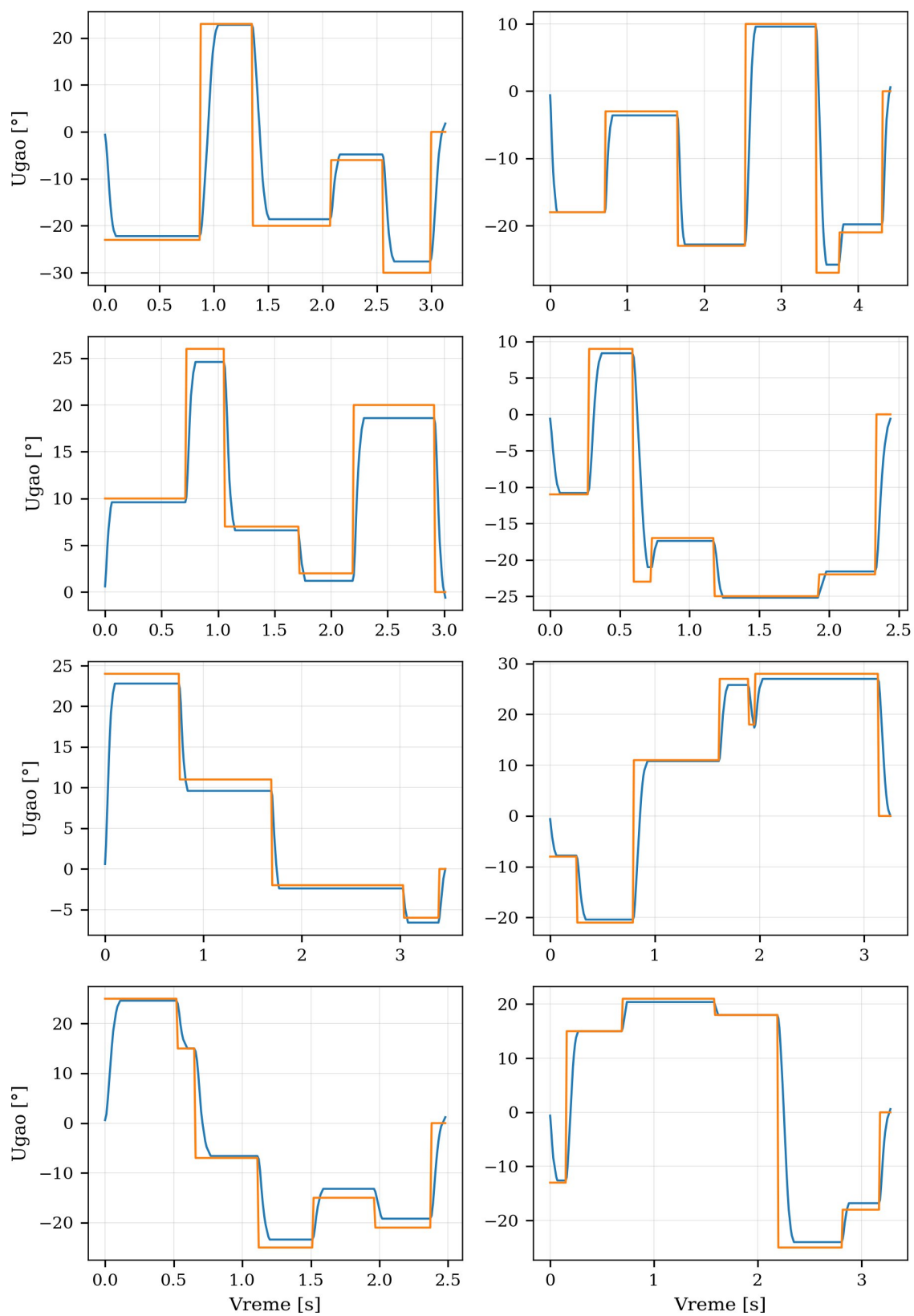
Сви обрађени подаци су графички приказани на слика у наставку.



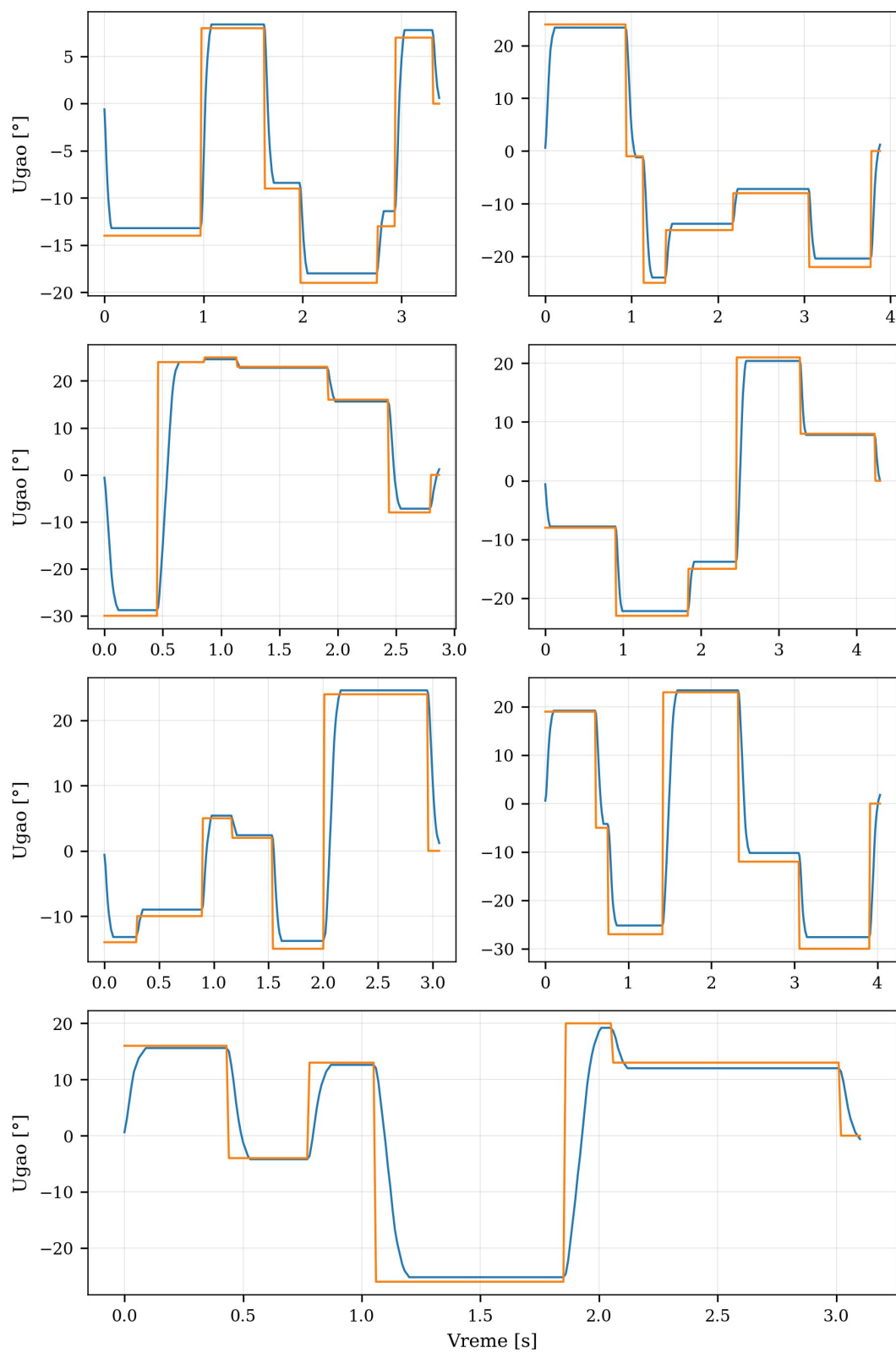
Графички приказ података прикупљених експериментом 1 (први део) – Слика 3.12



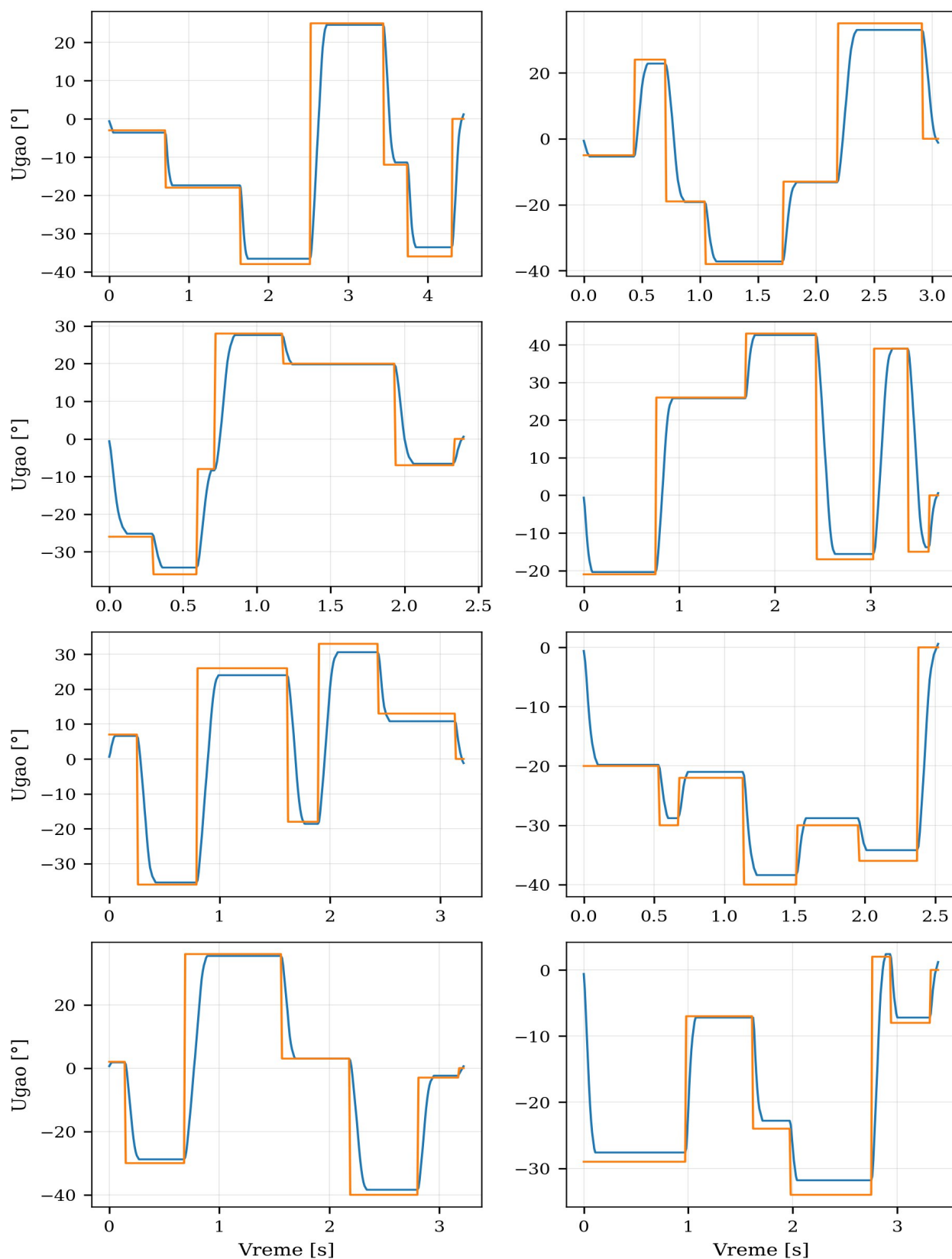
Графички приказ података прикупљених експериментом 1 (други део) – Слика 3.13



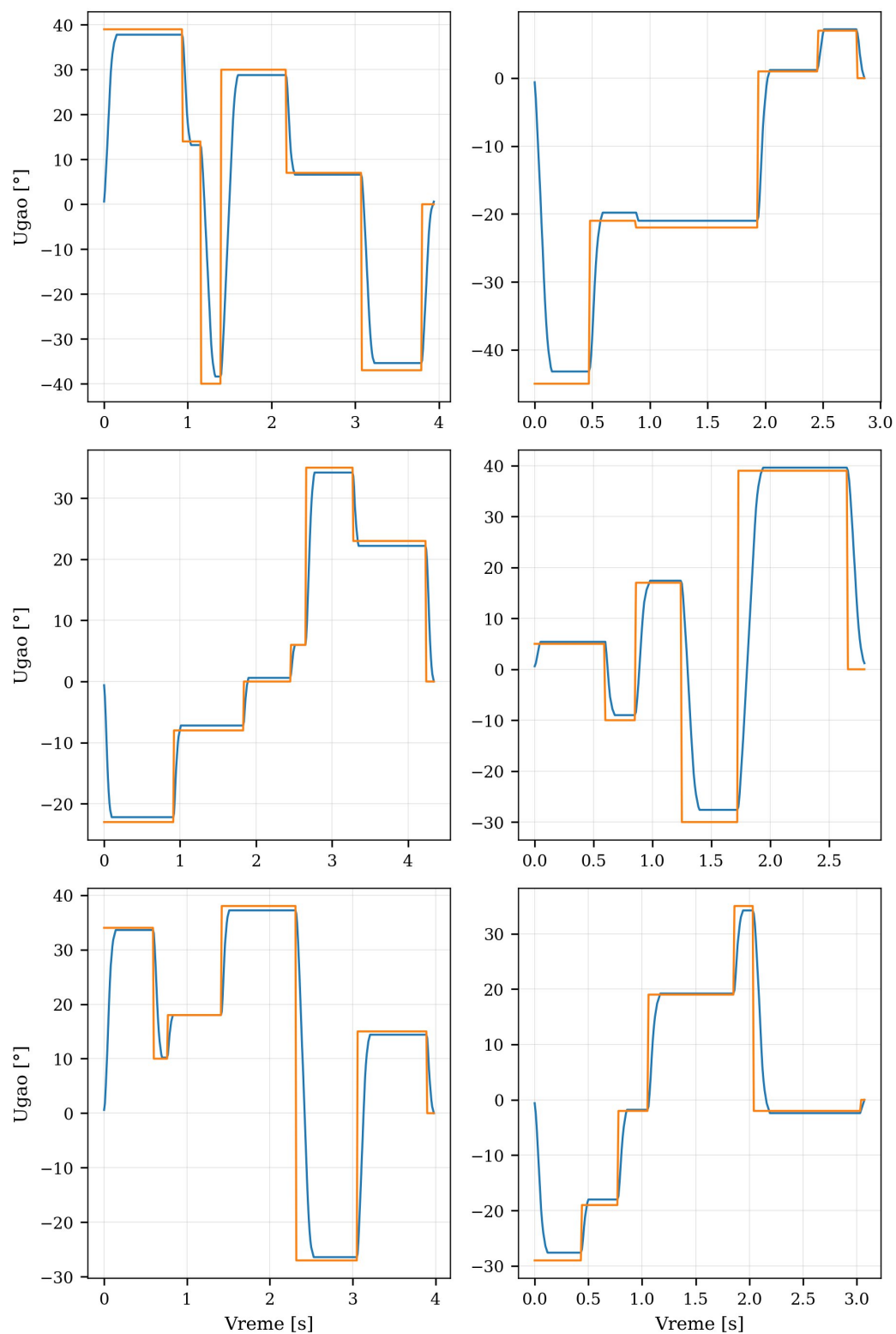
Графички приказ података прикупљених експериментом 3 (конфигурације 1, први део) –
Слика 3.14



Графички приказ података прикупљених експериментом 3 (конфигурације 1, други део)
– Слика 3.15



Графички приказ података прикупљених експериментом 3 (конфигурације 2, први део) –
Слика 3.16



Графички приказ података прикупљених експериментом 3 (конфигурације 2, други део)– Слика 3.17

3.2.2. Идентификација система методом сиве кутије

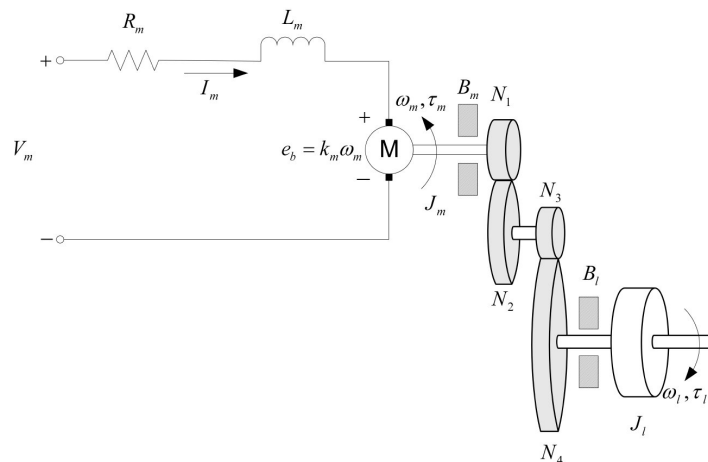
Серво мотор једносмерне струје се састоји од мотора једносмерне струје, који је директно спрегнут са редуктором, позиција излазног вратила се очитава помоћу потенциометра и сам серво мотор поседује свој контролер са управљачким алгоритмом.

На основу претходно реченог може се закључити да је систем затворен и да се до структуре модела можемо доћи налажењем општег математичког модела затвореног система електро мотора једносмерне струје са редуктором.

3.2.2.1. Општи математички модел електро мотора једносмерне струје са редуктором

Шема електро мотора једносмерне струје са редуктором је приказана на слици 3.18, где су:

- V_m – напон,
- I_m – струја,
- R_m – отпорност мотора,
- L_m – индуктивност мотора,
- k_m – константа електро моторне силе мотора,
- e_b – повратни напон електро мотора,
- ω_m – угаона брзина вратила мотора једносмерне струје,
- ω_l – угаона брзина излазног вратила мотора са редуктором,
- τ_l – обртни момент излазног вратила мотора са редуктором,
- J_x – момент инерције компоненте x , где је $x = m, l$,
- B_x – вискозно трење, где је $x = m, l$ и
- N_x – број зубаца на зупчанику x , где је $x = 1, 2, 3, 4$.



Шема спреге електро мотора једносмерне струје са редуктором – Слика 3.18

Моделовање електричног дела система

Кренувши од другог Кирховог закона који гласи “Сума свих напона у контури је једнака нули”, долази се до следеће једначине:

$$(3.2.1) \quad V_m(t) - R_m I_m - L_m \frac{dI_m(t)}{dt} - k_m \omega_m(t) = 0.$$

Узевши у обзир да је индуктивност мотора много мања од отпорности мотора, претходна једначина може се записати у следећем облику:

$$(3.2.2) \quad V_m(t) - R_m I_m - L_m \frac{dI_m(t)}{dt} - k_m \omega_m(t) = 0,$$

односно:

$$(3.2.3) \quad I_m(t) = (V_m(t) - k_m \omega_m(t)).$$

Моделовање механичког дела система

Кренувши од другог Њутновог закона који за ротационо кретање гласи “Угаоно убрзање које при кретању добије једно тело сразмерно је интензитету обртног момента који делује на њега, а обрнуто сразмерно моменту инерције тела”

$$(3.2.4) \quad J \alpha = \tau,$$

може се доћи до следећих једначина:

$$(3.2.5) \quad J_l \frac{d\omega_l(t)}{dt} + B_l \omega_l(t) = \tau_l(t),$$

$$(3.2.6) \quad J_m \frac{d\omega_m(t)}{dt} + B_m \omega_m(t) + \tau_{ml}(t) = \tau_m(t),$$

где је (3.2.4) једначина за обртни момент излазног вратила система, а (3.2.5) једначина обртног момента на излазном вратилу електро мотора. Једначина обртног момента излазног вратила система може се записати и у следећем облику:

$$(3.2.7) \quad \tau_l(t) = \eta_g K_g \tau_{ml}(t),$$

где су:

$$(3.2.8) \quad K_{gi} = \frac{N_2}{N_1}; K_{ge} = \frac{N_4}{N_3}; K_g = K_{ge} K_{gi}.$$

За потребе даљег решавања једначина (3.2.6) је усвојена у следећем облику:

$$(3.2.9) \quad \tau_{ml}(t) = \frac{\tau_l(t)}{\eta_g K_g}.$$

Зависност промене угла излазног вратила мотора у односу на промену угла излазног вратила система мотора са редуктором може се представити следећом једначином:

$$(3.2.10) \quad \theta_m(t) = K_g \theta_l(t),$$

одакле следи да зависност њихових угаоних брзина може бити исказана следећом једначином:

$$(3.2.11) \quad \omega_m(t) = K_g \omega_l(t).$$

односно на основу претходних релација може се доћи до следећег израза:

$$(3.2.12) \quad J_m K_g \frac{d\omega_l(t)}{dt} + B_m K_g \omega_l(t) + \frac{J_l \frac{d\omega_l(t)}{dt} + B_l \omega_l(t)}{\eta_g K_g} = \tau_m(t),$$

и ако се уведу смене:

$$(3.2.13) \quad J_{eq} = \eta_g K_g^2 J_m + J_l \text{ и}$$

$$(3.2.14) \quad B_{eq} = \eta_g K_g^2 B_m + B_l,$$

доћи ће се до следеће једначине:

$$(3.2.15) \quad J_{eq} \frac{d\omega_l(t)}{dt} + B_{eq} \omega_l(t) = \eta_g K_g \tau_m(t).$$

Комбиновање електричног и механичког система

Комбиновањем претходна два система доћи ће се до општег математичког модела електро мотора једносмерне струје са редуктора.

Обртни момент мотора може се представити и следећом једначином:

$$(3.2.16) \quad \tau_m(t) = \eta_m k_t I_m(t),$$

која се на основу једначине (3.2.2) можемо даље развити у облик:

$$(3.2.17) \quad \tau_m(t) = \frac{\eta_m k_t (V_m(t) - k_m \omega_m(t))}{R_m}$$

односно, на основу (3.2.10):

$$(3.2.18) \quad \tau_m(t) = \frac{\eta_m k_t (V_m(t) - k_m K_g \omega_l(t))}{R_m}.$$

Убацивањем једначине (3.2.14) у претходну добија се:

$$(3.2.19) \quad J_{eq} \frac{d\omega_l(t)}{dt} + B_{eq} \omega_l(t) = \frac{\eta_g K_g \eta_m k_t (V_m(t) - k_m K_g \omega_l(t))}{R_m}.$$

Док сређивањем једначине (3.2.18) се добија:

$$(3.2.20) \quad J_{eq} \frac{d}{dt} \omega_l(t) + \left(\frac{k_m K_g^2 k_t \eta_g \eta_m}{R_m} + B_{eq} \right) \omega_l(t) = \frac{\eta_g K_g \eta_m k_t}{R_m} V_m(t),$$

односно:

$$(3.2.21) \quad J_{eq} \frac{d}{dt} \omega_l(t) + B_{eq,v} \omega_l(t) = A_m V_m(t),$$

где је:

$$(3.2.22) \quad B_{eq,v} = \frac{k_m K_g^2 k_t \eta_g \eta_m}{R_m} + B_{eq} \text{ и}$$

$$(3.2.23) \quad A_m = \frac{\eta_g K_g \eta_m k_t}{R_m}.$$

Једначина (3.2.20) представља математички модел електро мотора једносмерне струје са редуктором и применом леве Лапласове трансформације може се свести на следећи облик:

$$(3.2.24) \quad J_{eq} \dot{\omega}_l(s) + B_{eq,v} \omega_l(s) = A_m V_m(s),$$

односно:

$$(3.2.25) \quad \frac{\omega(s)}{V_m(s)} = \frac{A_m}{J_{eq} s + B_{eq,v}}.$$

Узевши у обзир да се у свим експериментима управљало позицијом серво мотора, предходна једначина је интегралена, чиме је добијена преносна функција промене угла излазног вратила мотора једносмерне струје са редуктором у зависности од напона:

$$(3.2.26) \quad W(s) = \frac{\theta_l(s)}{V_m(s)} = \frac{A_m}{s(J_{eq}s + B_{eq,v})} = \frac{b_0}{s + a_0} \frac{1}{s}$$

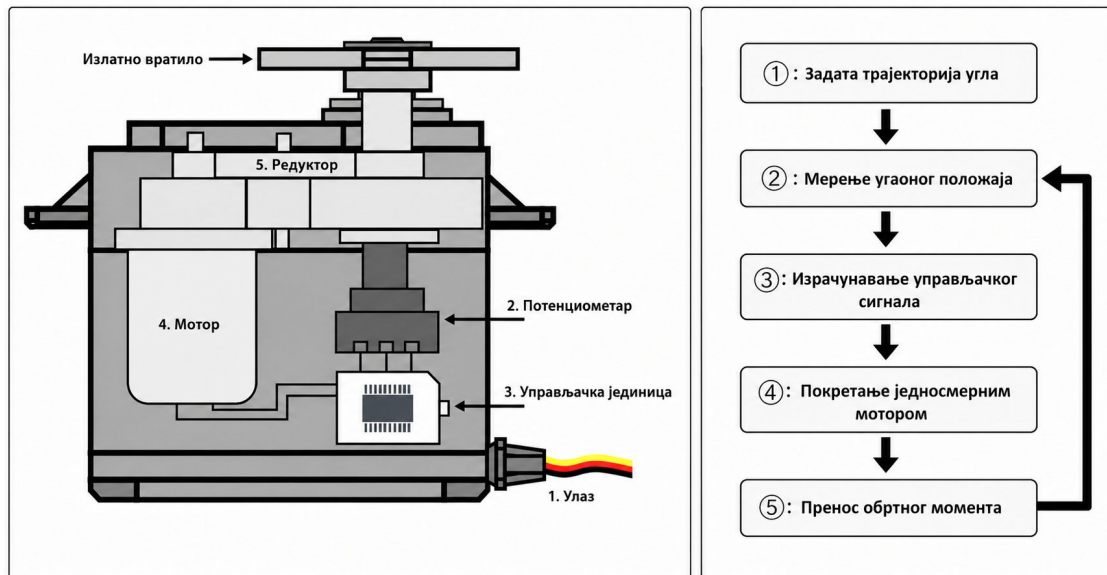
3.2.2.2. Одређивање математичког модела

Познато је да је серво мотор, сам по себи, један затворен систем. На улаз серво мотора је потребно довести одговарајући напон и он би на основу тог напона ротира излазно вратило за одређени угао. Узимајући у обзир да је за експерименте коришћена библиотека која омогућава да се за жељену вредност излаза зада жељена позиција излазног вратила серво мотора за преносну функцију је усвојен количник стварне и жељене позиције излазног вратила серво мотора:

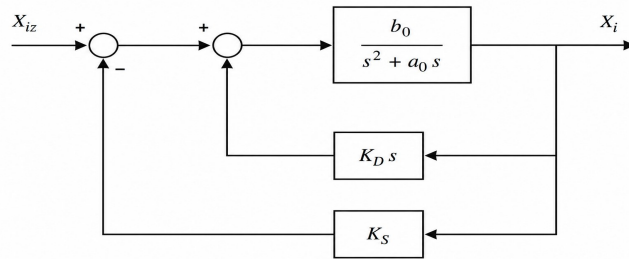
$$(3.2.27) \quad W(s) = \frac{\theta_l(s)}{X_{изж}(s)}.$$

Структура и принцип рада комерцијалног серво мотора једносмерне струје детаљно су приказани у истраживачком раду Yoonky Hawnga, Yuki Minami и Makato Ishikawe, под насловом Виртуални сензор обртног момента за јефтине серво моторе једносмерне струје засноване на идентификацији динамичког система са параметријским ограничењем (енг. Virtual Torque Sensor for Low-Cost RC Servo Motors Based on Dynamic System Identification Utilizing Parametric Constraints).

Алгоритам рада серво мотора и сликовита шема приказани су на слици 3.19, док је блок дијаграм серво мотора једносмерне струје приказан на слици 3.20.



Алгоритам рада серво мотора једносмерне струје и сликовита шема – Слика 3.19



Блок дијаграм серво мотора једносмерне струје – Слика 3.20

Сређивањем блок дијаграма са слике 3.20 долазимо до следеће преносне функције:

$$(3.2.28) \quad W(s) = \frac{\theta_l(s)}{X_{из}(s)} = \frac{b_0}{s^2 + s(a_0 - K_D b_0) + K_S b_0}.$$

Естимација параметара и први део валидације је одређен помоћу Matlab скрипте tfEstimation.m.

Скупина — tfEstimation.m

```
close all
clear all
clc

sysData = struct();
dir_path = "..\Ball_And_Beam\System_Identification\Data\Encoder_data\Test_";
data_path = "\Processed\Test_";

% Polovi i nule
np = 2; % broj polova
nz = 0; % broj nula
file_path = "..\Ball_And_Beam\System_Identification\Data\Estimation_data\GreyBox\Model_1.json";

% Estimacija prenosne funkcije
for idx = 1:2
    for jdx = 0:29
        try
            final_path = dir_path + int2str(idx) + data_path + int2str(jdx) + ".csv";
            T = readtable(final_path, 'ReadVariableNames', true);

            % Remove everything afeter t>= 2s and NaN values
            toDelete = T.Time_s >= 2.0;
            T(toDelete, :) = [];
            T = rmmissing(T);

            % Estimate tf and save num and den of sys
            data = iddata(T.Angles, T.PWM, 0.01);
            sys = tfest(data, np, nz);
            test_no = sprintf('test_%d_%d', idx, jdx);
```

```

sysData.(test_no).num = sys.Numerator;
sysData.(test_no).den = sys.Denominator;

% Simulate and show on plot for quick validation
simTime = 0:0.01:length(T.PWM)/100-0.01;
[y, tout] = lsim(sys, T.PWM, simTime);
figure(idx*100+jdx);
plot(tout, y, T.Time_s, T.Angles);
grid on;

catch ME
    if (strcmp(ME.identifier, 'MATLAB:readtable:OpenFailed'))
        ;
    else
        disp(ME)
    end
end
end
end
end

% Validate data
keys = fieldnames(sysData);
for idx = 1:length(keys)
    current_key = string(keys(idx));
    MSE_data = Validate(sysData.(current_key).num, sysData.(current_key).den);
    sysData.(current_key).MSE = MSE_data;
end
jsonData = jsonencode(sysData);

fileID = fopen(file_path, 'w');
if fileID == -1
    error('Cannot open file for writing: %s', file_path);
end
fprintf(fileID, '%s', jsonData);
fclose(fileID);

```

Функција — Validation.m

```

function [MSE] = Validate(num,den)
%VALIDATE Summary of this function goes here
% Detailed explanation goes here

dir_path = "..\Ball_And_Beam\System_Identification\Data\Encoder_data\Validation_data\Dir_";
errorData= struct();
sys = tf(num, den);

for i = 1:2
    for j = 0:8
        try
            final_path = dir_path + int2str(i) + '\Val_Data_' + int2str(j) + '.csv';
            T = readtable(final_path, 'ReadVariableNames', true);
            [y, ] = lsim(sys, T.PWM, T.Time_s);
            test_no = sprintf('mse_%d_%d', i, j);

```



```

        errorData.(test_no) = immse(T.Angles, y);

    catch ME
        if (strcmp(ME.identifier, 'MATLAB:readtable:OpenFailed'))
            ;
        else
            disp(ME)
        end
    end
end
end
end

MSE = errorData;

end

```

Ова скрипта може да се подели у неколико делова:

- иницијализација – део на почетку скрипте где се дефинишу бројеви полова и нула преносне функције (np и nz),
- естимација параметара преносне функције – део где се на основу података сваког случаја прве групе експеримента естимирају параметри преносне функције,
- валидација модела – циљ овог дела јесте поређење излаза модела са излазима сваког случаја једног дела друге групе експеримента (-30, 30). Излаз ове функције је низа средњих квадратних грешака.
- чување података – овај део скрипте је задужен за креирање датотеке json формата и чување естимираних вредности модела и низова средњих квадратних грешака у истој датотеци.

За даљу анализу претходно сачуване датотеке искоришћена је скрипта Data_visualization.py (код ове скрипте се може наћи у додатку Г). Скрипта за сваки модел рачуна просечну, медијану и најгору средњу квадратну грешку и рангира преносне функције. Такође, даје и графички приказ расподела средњих квадратних грешака (слика 3.21), са којег можемо закључити да су експериментални подаци задовољавајућег квалитета за идентификацију мотора и омогућава предселекцију кандидата за даљи рад. Као финални излаз ова скрипта даје и коефицијенте пет најбољих преносних функција (табела 3.1).

Преносна функција	Коефицијенти бројиоца	Коефицијенти имениоца
1	569.62	1, 38.54, 593.73
2	569.32	1, 38.51, 593.46
3	566.56	1, 38.71, 590.51
4	620.45	1, 39.94, 637.82
5	534.42	1, 37.08, 549.42

Табела пет најбољих преносних функција – Табела 3.1

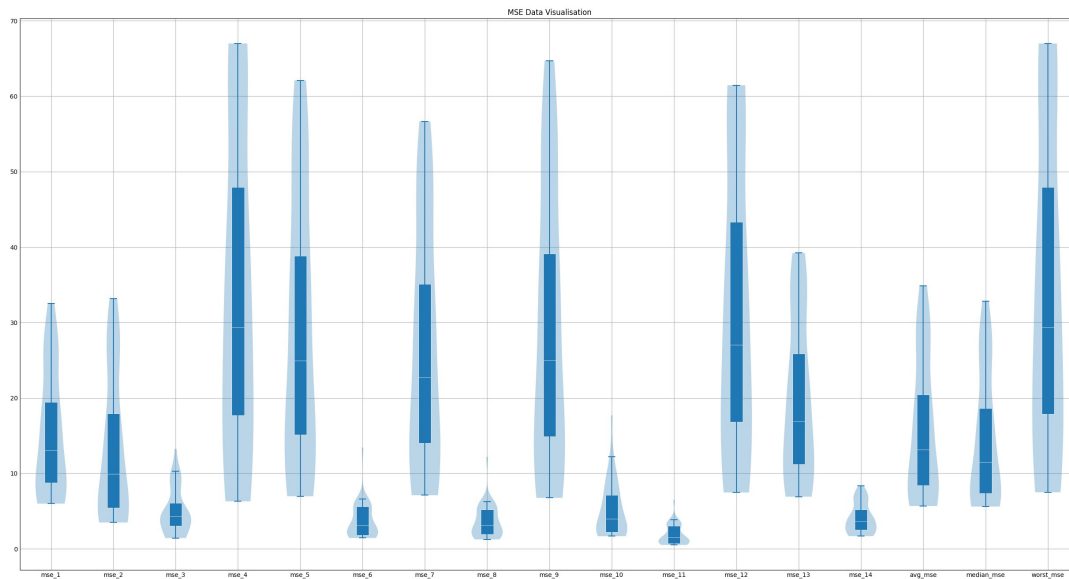
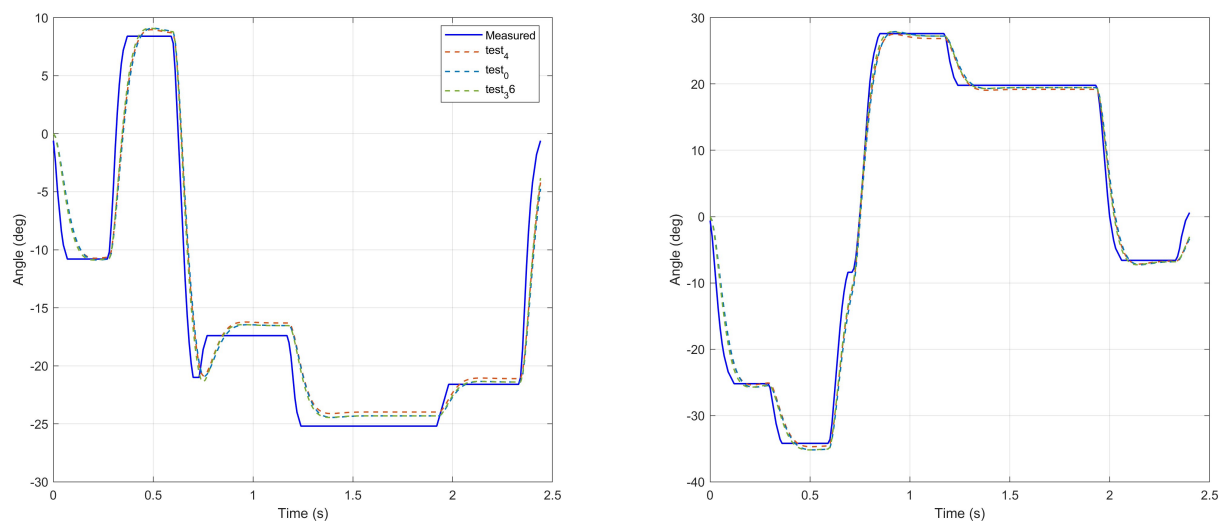
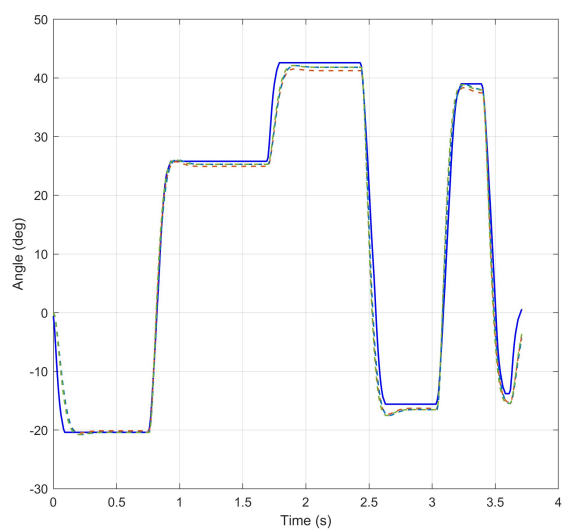
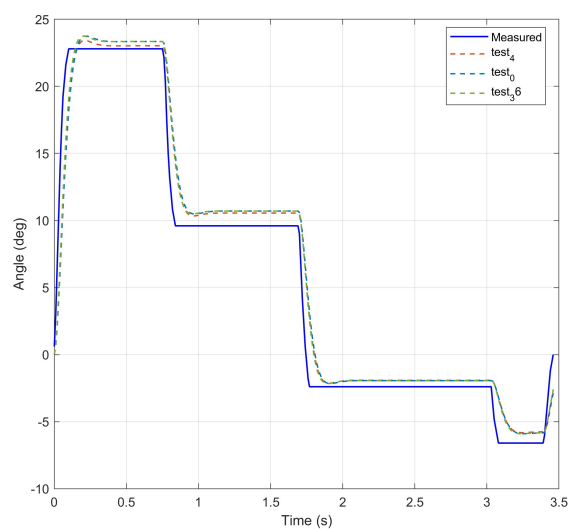


График расподеле средњих квадратних грешака по моделу – Слика 3.21

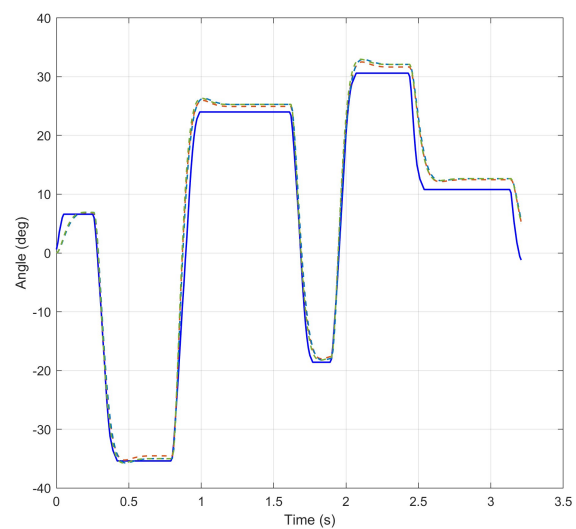
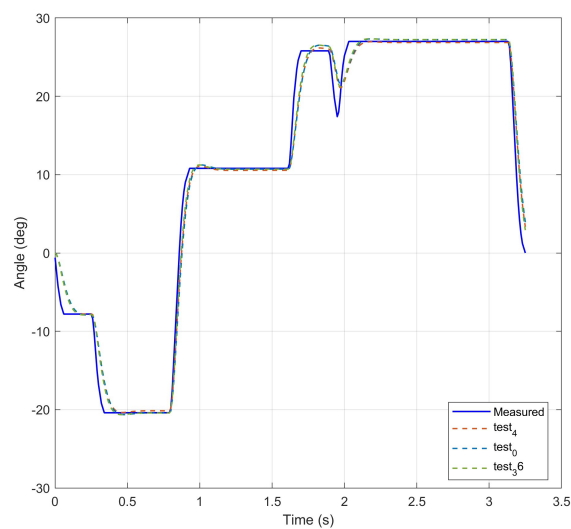
Три изабране преносне функције даље су анализиране скриптом `tfValidation.m` на скуповима података скупљених другом групом експеримента. За сваки случај симулиран је одзив модела и упоређен се мереним, стварним, одзивом. Резултати су приказани графицима упоређених одзива (слике 3.22а, 3.22б, 3.22ц) и дијаграмима резидуала (слике 3.23а, 3.23б, 3.23ц).



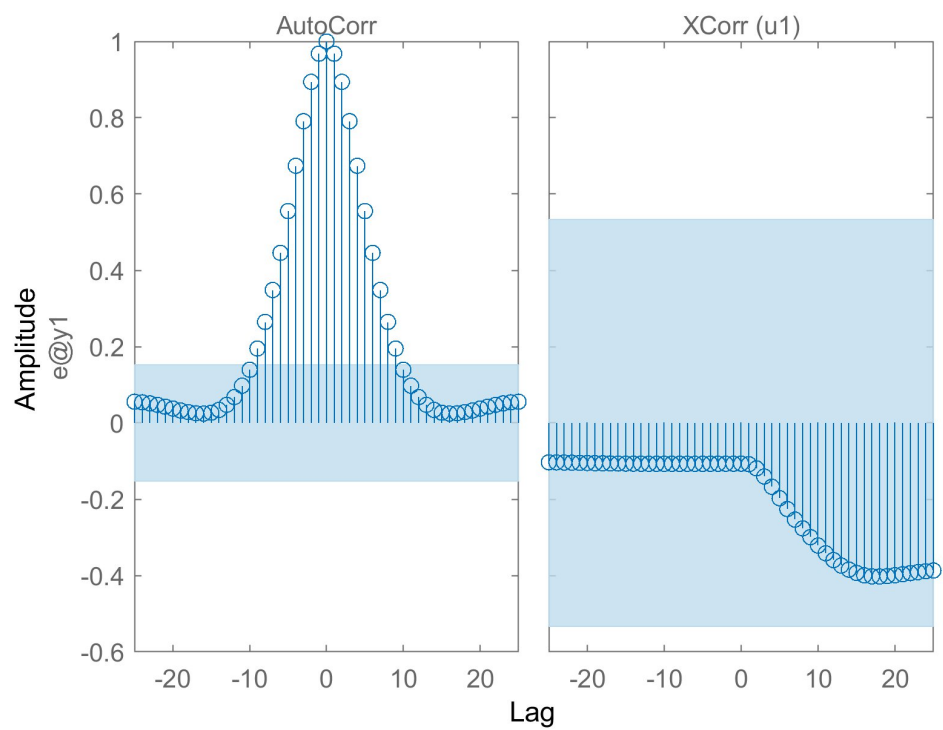
Графички приказ стварног одзива и одзива модела – Слика 3.22а



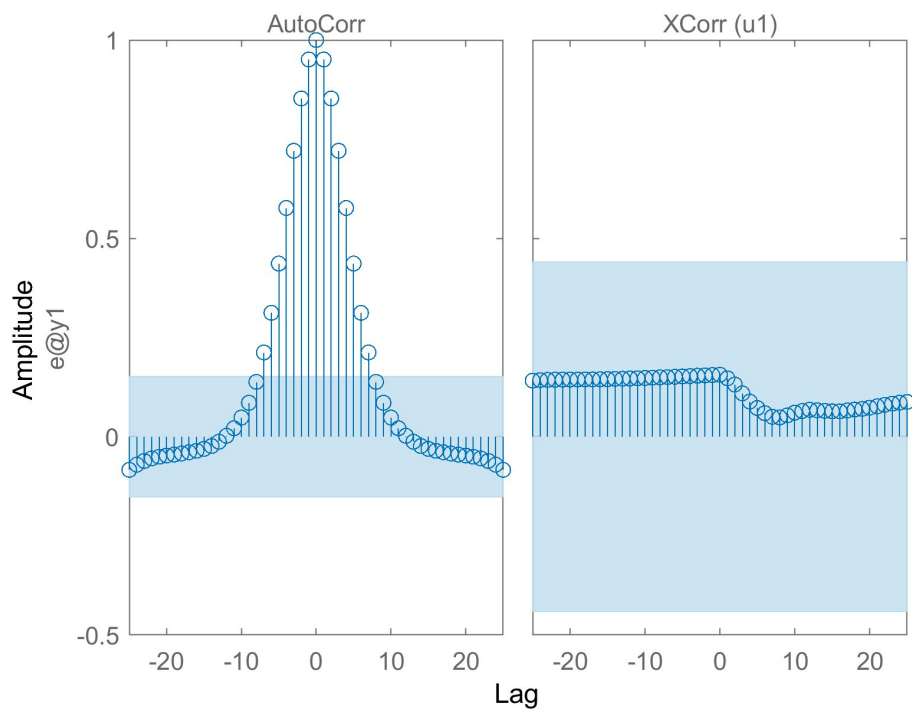
Графички приказ стварног одзива и одзива модела – Слика 3.22б



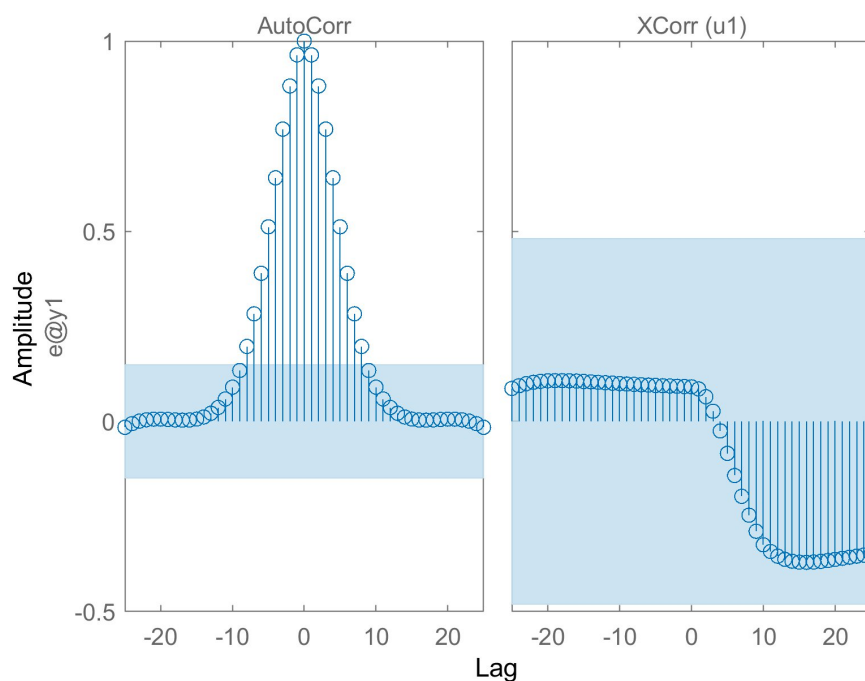
Графички приказ стварног одзива и одзива модела – Слика 3.22ц



Дијаграм резидуала – Слика 3.23а



Дијаграм резидуала – Слика 3.23б



Дијаграм резидуала – Слика 3.23ц

Са графика и дијаграма може се закључити да сва три модела су показала готово идентично праћење у временском домену и резидуалну статистику, што указује да је идентификована динамика робусна, а не осетљива на специфични скуп параметара.

За даљи рад изабрана је преносна функција 1 (3.2.29). Анализа резидуала тог модела показује да унакрсна корелација између резидуала и улазног сигнала референце остаје унутар интервала поверења на свим валидационим скуповима, чиме се потврђује да је улазно-излазна веза адекватно описана. Аутокорелација резидуала, међутим, систематски прелази границе интервала поверења, посебно током транзијената, што значи да модел другог реда не обухвата целокупну динамику излаза и да преостала грешка није потпуно случајна. И ако анализа резидуала указује на присуство динамике вишег реда или нелинеарних ефеката у систему серво мотора, прихватљиво је усвојити ову преносну функцију као математички модел и користити је у даљем раду

$$(3.2.29) \quad W(s) = \frac{569.62}{s^2 + 38.54s + 593.73}$$

3.2.3. Идентификација система методом црне кутије

У овом делу покушаћемо да одредимо математички модел серво мотора користећи метод црне кутије. Водећи се научним радовима:

- Deep Learning and System Identification, чији су аутори Lennart Ljung, Carl Andersson, Koen Tiels и Thomas B. Schon и
- Cascade Forward Neural Network for Time Series Prediction, чији су аутори Budi Warsito, Rukun Santoso, Suparti¹, Hasbi Yasin.

одлучено је да се за идентификацију система користи вишеслојна каскадна неуронска мрежа у комбинацији са нелинеарним ауторегресивним моделом са егзогеним улазима (NARX).

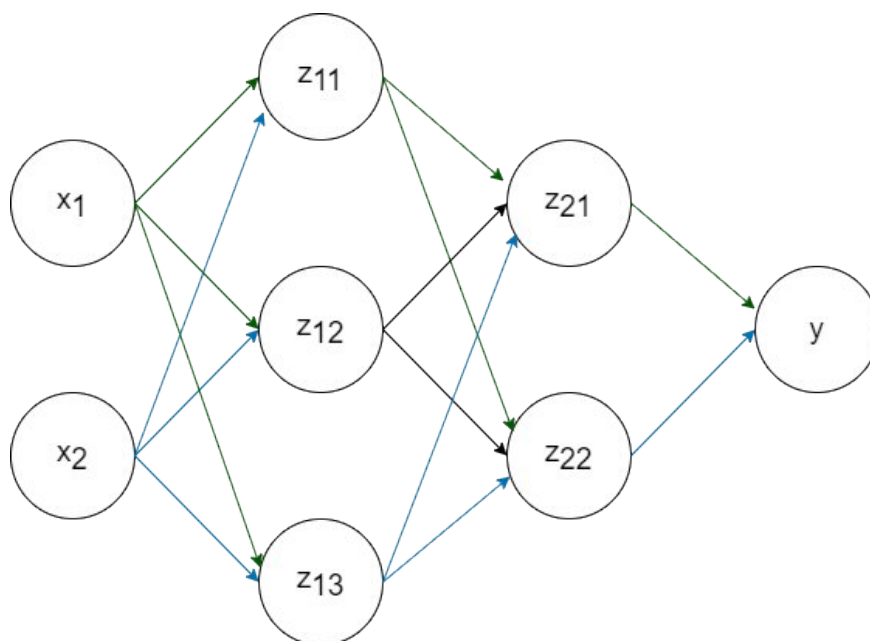
3.2.3.1. Вишеслојна каскадна неуронска мрежа без повратне спреге

Како би лакше објаснили вишеслојну каскадну неуронску мрежу без повратне спреге кренимо од “обичне” вишеслојне неуронске мреже без повратне спреге.

Због једноставности објашњења претпоставимо да имамо два улаза, један излаз и два скривена слоја како на слици 3.24, такође претпоставимо да сви неурони имају исту активациону функцију f и усвојимо следећу нотацију:

- x_p — p -ти улаз неуронске мреже,
- z_{ij} — излаз неурона у скривеном слоју i ,
- w_{jk}^i — тежина j -тог неурона у i -том скривеном слоју и
- y — излаз неуронске мреже,

где су $i, j, k, p = 1, 2, 3, 4, \dots, n$.



Вишеслојна неуронска мрежа без повратне спреге – Слика 3.24

Излаз једног неурална првог слоја је дат следећом једначином:

$$(3.2.30) \quad z_{1j} = f(w_{j1}^1 x_1 + w_{j2}^1 x_2) = f([w_{j1}^1 \ w_{j2}^1] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}).$$

Узимајући у обзир да први слој садржи три неурона можемо представити излаз целог слоја у следећем облику:

$$(3.2.31) \quad \mathbf{z}_1 = \begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix} = \mathbf{f}(\mathbf{w}^1 \mathbf{x}).$$

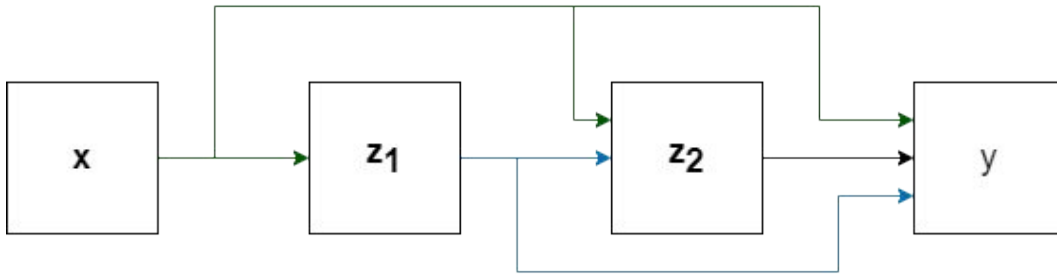
Други и излазни слој прате исти тренд и могу се представити следећим једначинама:

$$(3.2.32) \quad \mathbf{z}_2 = \begin{bmatrix} z_{21} \\ z_{22} \end{bmatrix} = \mathbf{f}(\mathbf{w}^2 \mathbf{z}_1) \text{ и}$$

$$(3.2.33) \quad y = \mathbf{f}(\mathbf{w}^3 \mathbf{z}_2).$$

Вишеслојна каскадна неуронска мрежа без повратне спреге се разликује од претходно објашњене мреже у начину на који су слојеви повезани. Код каскадне мреже слој $n+2$ ($n = 0, 1, 2, 3, \dots$) није повезан само са претходним слојем, већ са свим претходним слојевима.

Претпоставимо поново да наша мрежа има два улаза, један излаз и два скривена слоја као на слици 3.24, али усвојимо топологију каскадне неуронске мреже, слика 3.25. Такође, претпоставимо да сви неурони имају исту активациону функцију f и задржимо исту нотацију.



Вишеслојна каскадна неуронска мрежа без повратне спреге – Слика 3.25

Јасно је да је и у овом случају излаз једног неурона у првом слоју дат једначином (3.2.32), односно излаз целог слоја је дат изразом (3.2.33). Разлика од претходне мреже се може уочити у другом слоју, где је излаз једног неурона у овом слоју дат једначином:

$$(3.2.34) \quad z_{2j} = f(w_{j1}^2 x_1 + w_{j2}^2 x_2 + w_{j3}^2 z_{11} + w_{j4}^2 z_{12} + w_{j5}^2 z_{13}),$$

односно:

$$(3.2.35) \quad z_{2j} = f([w_{j1}^2 \ w_{j2}^2 \ w_{j3}^2 \ w_{j4}^2 \ w_{j5}^2] \begin{bmatrix} x_1 \\ x_2 \\ z_{11} \\ z_{12} \\ z_{13} \end{bmatrix}).$$

Узимајући у обзир да други слој садржи два неурона, излаз целог слоја можемо представити у следећем облику:

$$(3.2.36) \quad \mathbf{z}_2 = \begin{bmatrix} z_{21} \\ z_{22} \end{bmatrix} = f(\mathbf{w}^2 \begin{bmatrix} \mathbf{x} \\ \mathbf{z}_1 \end{bmatrix}).$$

Пратећи исти тренд јасно је да поред излаза другог слоја $\mathbf{z}_2$, утицај на излаз ове мреже имати ће и улаз $\mathbf{x}$ и излаз првог слоја $\mathbf{z}_1$ и да једначина излаза бити:

$$(3.2.37) \quad y = f(\mathbf{w}^3 \begin{bmatrix} \mathbf{x} \\ \mathbf{z}_1 \\ \mathbf{z}_2 \end{bmatrix}).$$

3.2.3.2. Дефиниције ауторегресивних модела

Ауторегресивни модели су основни алати за моделирање и анализу динамичких система, описујући како тренутни излаз система зависи од његових претходних излаза. Они играју кључну улогу у хватању унутрашње динамике система и често се проширују укључивањем спољних улаза, што доводи до свеобухватнијих модела. У области идентификације система, ауторегресивни модели се користе за креирање математичких представљања система заснованих на посматраним подацима. Ови модели су посебно ефикасни за системе где се понашање може предвидети на основу претходних стања, са фокусом на разумевање унутрашње динамике. Када су присутни спољни фактори, ауторегресивне компоненте се могу комбиновати са егзогеним улазима, формирајући напредније моделе као што су ARX или ARMAX, који укључују и унутрашњу динамику и спољне утицаје.

Дефиниција 9 Ауторегресивни (AR) модел је тип статичког модела који изражава тренутну вредност променљиве као линеарну комбинацију њених претходних вредности. Често се користи за предвиђање података који зависе од времена, претпостављајући да је тренутни излаз функција претходних излаза. Општи облик ауторегресивног модела p -тог реда дат је једначином:

$$(3.2.38) \quad y(t) = c + \sum_{i=1}^p \varphi_i y(t-i) + \varepsilon(t).$$

Где је:

- $y(t)$ — излаз,
- c — константа,
- φ_i — i -ти коефицијент модела,
- $\varepsilon(t)$ — грешка,
- p — ред модела, који показује број коришћених претходних вредности.

Дефиниција 10 Егзогена улазна величина представља све познате улазне величине система, односно објекта, које битно утичу на рад самог система.

Да сумирамо, егзогена улазна величина се односи и на све жељене вредности излазне величине и све мерене/познате величине поремећаја.

Дефиниција 11 Ауторегресивни модел са егзогеним улазом (ARX модел) је тип линеарног динамичког модела који се користи у идентификацији система и комбинује ауторегресивни део (AR) са егзогеним улазом (X). Општи облик ARX модела је:

$$(3.2.39) \quad y(t) = \sum_{i=1}^{n_a} a_i y(t-i) + \sum_{j=1}^{n_b} b_j u(t-j) + \varepsilon(t).$$

Где је:

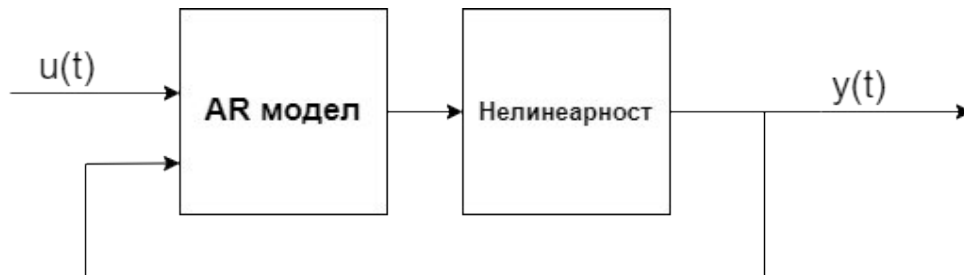
- $y(t)$ — излаз,
- $u(t-j)$ — егзогени улаз,
- a_i — коефицијент за ауторегресивне чланове, где је $i = 1, 2, \dots, n_a$,
- b_j — коефицијент за егзогене улазе, где је $j = 1, 2, \dots, n_b$,
- n_a — број претходних вредности излаза (ред ауторегресије),
- n_b — број претходних вредности улаза које се узимају у обзир,
- $\varepsilon(t)$ — грешка која са собом носи информације о свим не мереним улазима (углавном поремећајима).

Дефиниција 12 Нелинеарни ауторегресивни модел са егзогеним улазима (NARX) је врста динамичког нелинеарног модела који се обично користи за моделирање система са информацијама о прошлим улазима и прошлим излазима који утичу на будуће излазе. Овај модел је описан следећом једначином:

$$(3.2.40) \quad y(t) = f(y(t-1), y(t-2), \dots, y(t-n_y), u(t-1), u(t-2), \dots, u(t-n_u)) + \varepsilon(t),$$

где су:

- $y(t)$ — излаз,
- $u(t)$ — улаз,
- $f(\bullet)$ — функција нелинеарности и
- $\varepsilon(t)$ — грешка која са собом носи информације о свим не мереним улазима (углавном поремећајима).



Графички приказ нелинеарног ауторегресивног модела са егзогеним улазима —
Слика 3.26

3.2.3.3. Идентификација система

За потребе овог задатка искористили смо податке из другог експеримента, где смо насумично одабрали двадесет случајева и искористили за обучавање мреже и естимацију модела, а осталих десет смо искористили за валидацију.

За идентификацију и валидацију модела искористили смо скрипту Matlab скрипту `bbModelEstimation.m` где смо за конструкцију модела изабрали нелинеарни ауторегресивни модел са егзогеним улазом, а за излазну функцију, односно нелинеарност, изабрали вишеслојну каскадну неуронску мрежу без повратне спреге. Кроз пар итерација дошли смо до конфигурације каскадне мреже која има пет скривених слојева са следећим бројем неурона [2, 3, 4, 3, 2] и ауторегресивног модела са егзогеним улазом који се може описати следећом једначином:

$$(3.2.41) \quad y(t) = \sum_{i=1}^4 a_i y(t-i) + \sum_{j=1}^4 b_j u(t-j).$$

Скрипта — `bbModelEstimation.m`

```
close all
clear all
clc
```

```
training_data_path = "..\Ball_And_Beam\System_Identification\Matlab_Scripts\Black_Box\TestData.csv";
validatio_data_path =
"..\Ball_And_Beam\System_Identification\Matlab_Scripts\Black_Box\ValidationData.csv";
```

```
T = readtable(training_data_path, 'ReadVariableNames', true);
training_data = iddata(T.Angles, T.PWM, 0.01);
```

```
net = cascadeforwardnet([2,3,4,3,2]);
N2 = neuralnet(net);
sys = nlarx(training_data, [4 4 0], N2);
```

```
% Validate against training data
figure(1);
compare(training_data, sys);
```

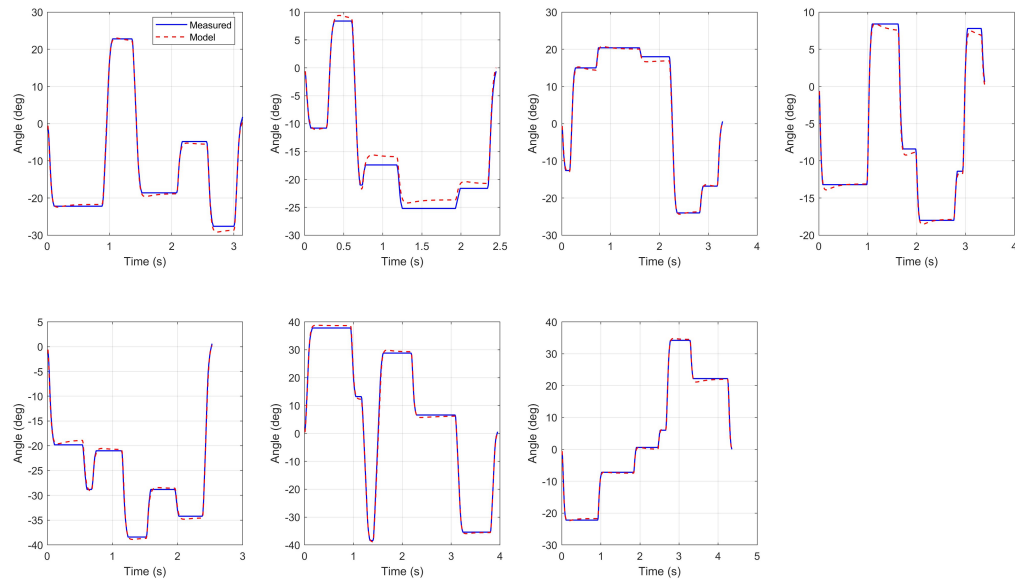
```
figure(2);
resid(training_data, sys);
```

```
% Validate against validation data
T = readtable(validatio_data_path, 'ReadVariableNames', true);
validatio_data = iddata(T.Angles, T.PWM, 0.01);
```

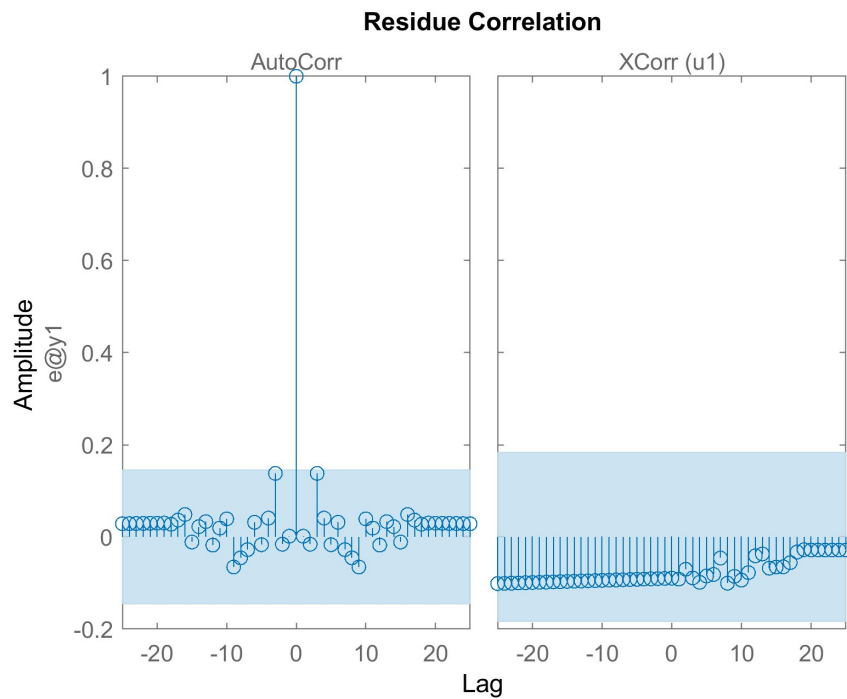
```
figure(3);
compare(validatio_data, sys);
```

```
figure(4);
resid(validatio_data, sys);
```

Након естимације, модел је проверен на независном скупу од седам валидационих експеримената.



Поређење измереног и симулираног одзива – Слика 3.27



Дијаграм резидуала – Слика 3.28

Слика 3.28b. Резидуали — експеримент 6 (validation_-_residuals_exp06.png)

На свим случајевима модел прати облик одзива, а степен преклапања је нешто већи од 93% и на скупу за обучавање и на валидационом скупу. У односу на модел сиве кутије, овде је преклапање веће (преко 93% насупрот око 87%), а преостала грешка модела нема значајне корелације ни са собом ни са улазним сигналом. NARX модел не захтева унапред задату структуру мотора, већ директно учи улазно-излазну везу из података, па имплицитно обухвата нелинеарности и ефекте унутрашњег серво регулатора које линеарни модел не обухвата.

4. Пројектовање и имплементација регулатора

4.1. Анализа и испитивање система

Пре пројектовања и имплементације регулатора потребно је испитати и анализирати цео систем. У нашем случају систем се састоји из два дела, лопте на шини и серво мотора. Чије смо математичке моделе одредили и анализирали у претходном делу.

У овом делу упоредићемо линеаризовани и нелинеарни модел лопте на шини са стварним системом. И на основу тог поређења одабрати модел који ћемо касније искористити за пројектовање регулатора.

4.1.1 *Нелинеарни, линеаризовани и стварни систем лопте на шини*

Као што је напоменуто у уводу, математички модел лопте на шини је одређен у претходном поглављу, прецизније у поглављу 3.1 *Једначина кретање лопте на шини*. У овом поглављу једначина је упрошћена и дата у облику преносне функције, изразом 3.1.19. Међутим, кретање лопте на шини може се представити и једначинама 3.1.6 и 3.1.12.

Користећи софтверски алат Simulink за даље потребе мастер рада, анализирали смо:

- нелинеарни систем лопте на шини без серво мотора,
- линеаризовани систем лопте на шини без серво мотора,
- нелинеарни систем лопте на шини са преносном функцијом серво мотора,
- линеаризовани систем лопте на шини са преносном функцијом серво мотора,
- нелинеаризовани систем лопте на шини са нелинеарним моделом серво мотора и
- линеаризовани систем лопте на шини са нелинеарним моделом серво мотора.

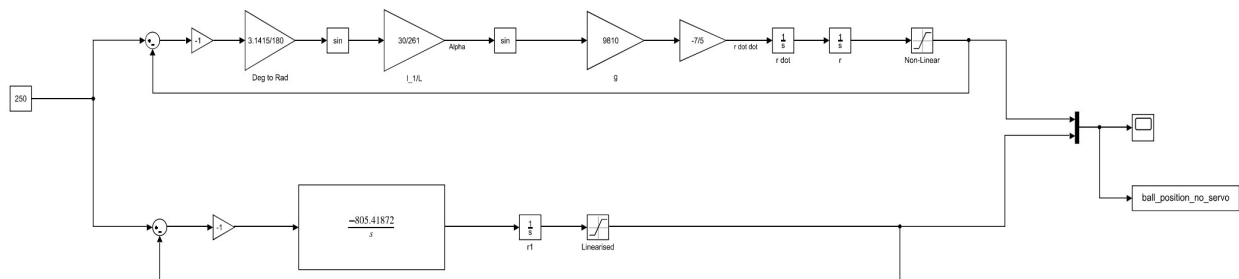
У обзир су узети и отворени и затворени систем. При анализи отвореног система посматран је само одзив система у временском домену, док су при анализи затвореног система разматрани и одзив система у временском домену и промена угла серво мотора.

Приликом креирања блок дијаграма у програмском пакету Simulink усвојено су следеће вредности параметара:

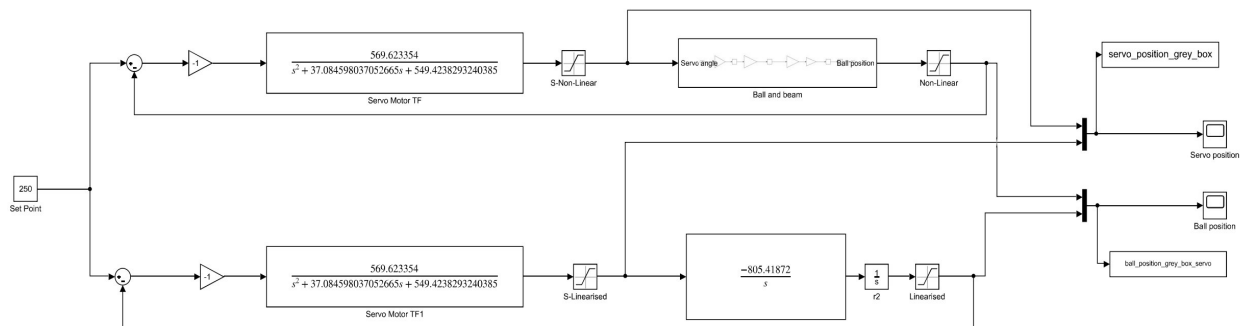
- опсег угла серво мотора: од -45° до 45° ,
- опсег умерене позиције лопте: од 0mm до 400mm,
- почетна позиција лопте: 100mm и
- референта (жељена) позиција лопте: 250mm.

Блок дијаграми затворених система су приказани на наредним сликама и то редом:

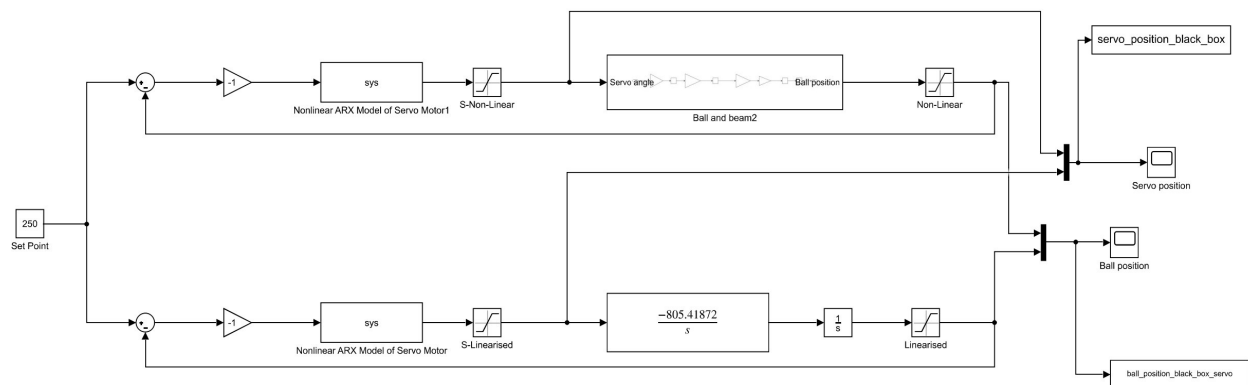
- Нелинеарни и линеаризовани модели лопте на шини – слика 4.1,
- Нелинеарни и линеаризовани модели лопте на шини у спрези са математичким моделом серво мотора одређеним методом сиве кутије – слика 4.2 и
- Нелинеарни и линеаризовани модели лопте на шини у спрези са математичким моделом серво мотора одређеним методом црне кутије – слика 4.3.



Нелинеарни и линеаризовани модели лопте на шини – Слика 4.1



Нелинеарни и линеаризовани модели лопте на шини у спрези са математичким моделом серво мотора одређеног методом сиве кутије – Слика 4.2



Нелинеарни и линеаризовани модели лопте на шини у спрези са математичким моделом серво мотора одређеног методом црне кутије – Слика 4.3

Док су у наставку приказани одзиви система и промена угла серво мотора за различите конфигурације модела, и то следећим редоследом:

- отворени систем (слика 4.4);
- затворени систем без модела серво мотора (слика 4.5);
- затворени систем са математичким моделом серво мотора добијеним методом сиве кутије (слике 4.6 и 4.7);
- затворени систем са математичким моделом серво мотора добијеним методом црне кутије (слике 4.8 и 4.9).

Као и одзив система и промена угла серво мотора стварног система (слика 4.10).

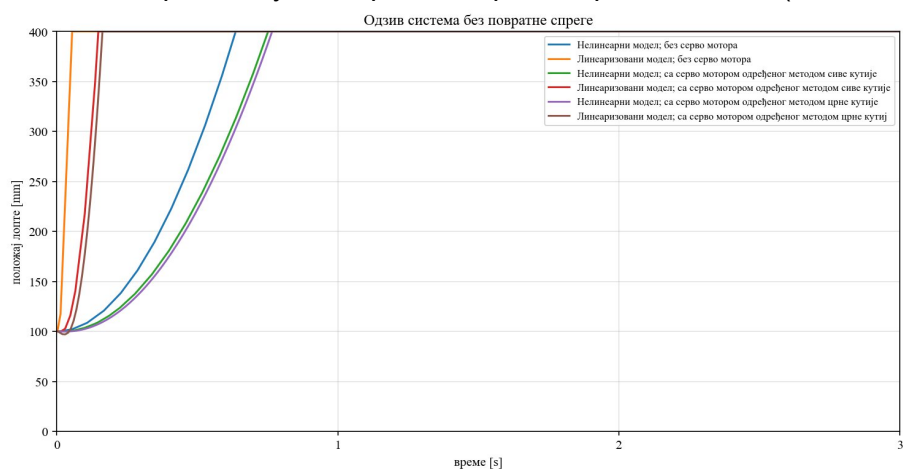


График одзива отворених система – слика 4.4

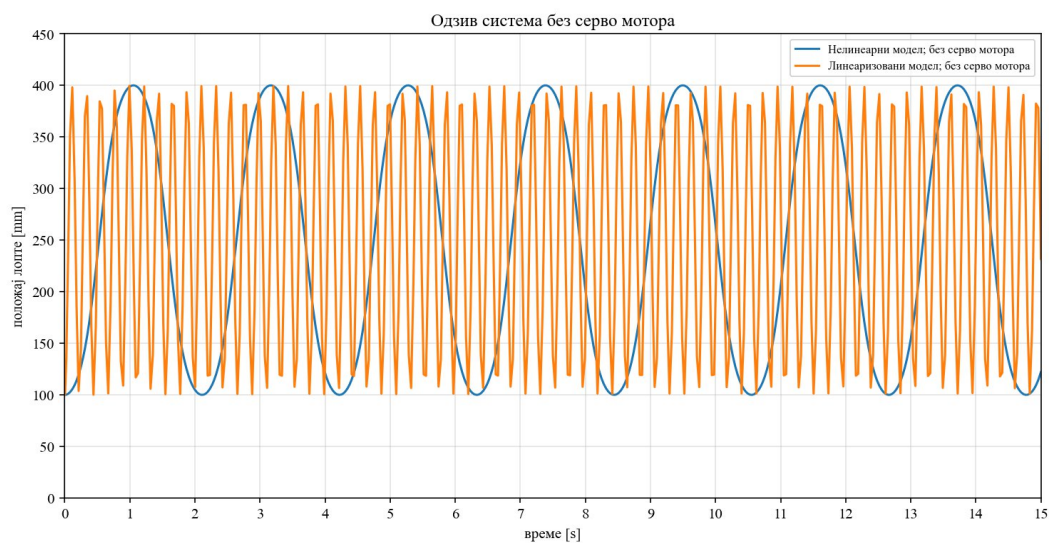


График одзива система без серво мотора – слика 4.5

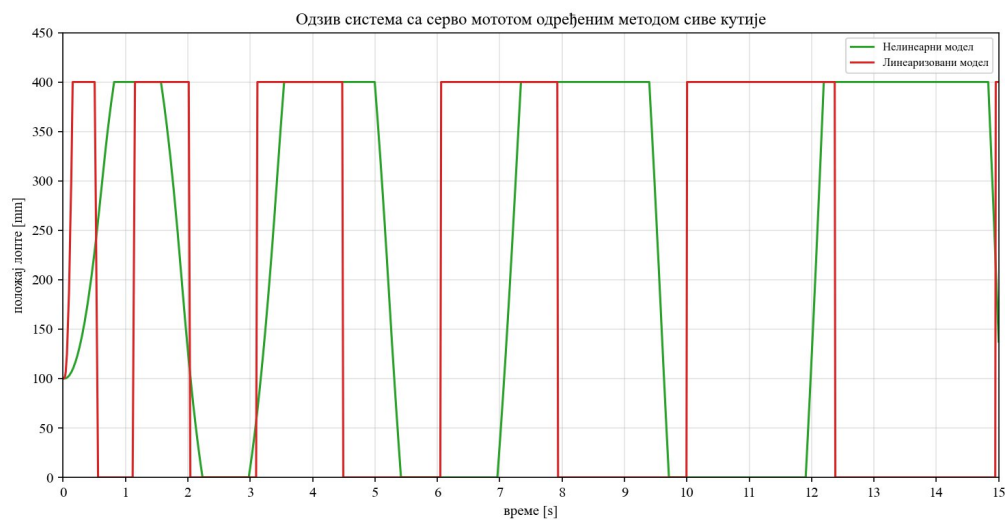


График одзива система са моделом серво мотора одређеним методом сиве кутије –
Слика 4.6

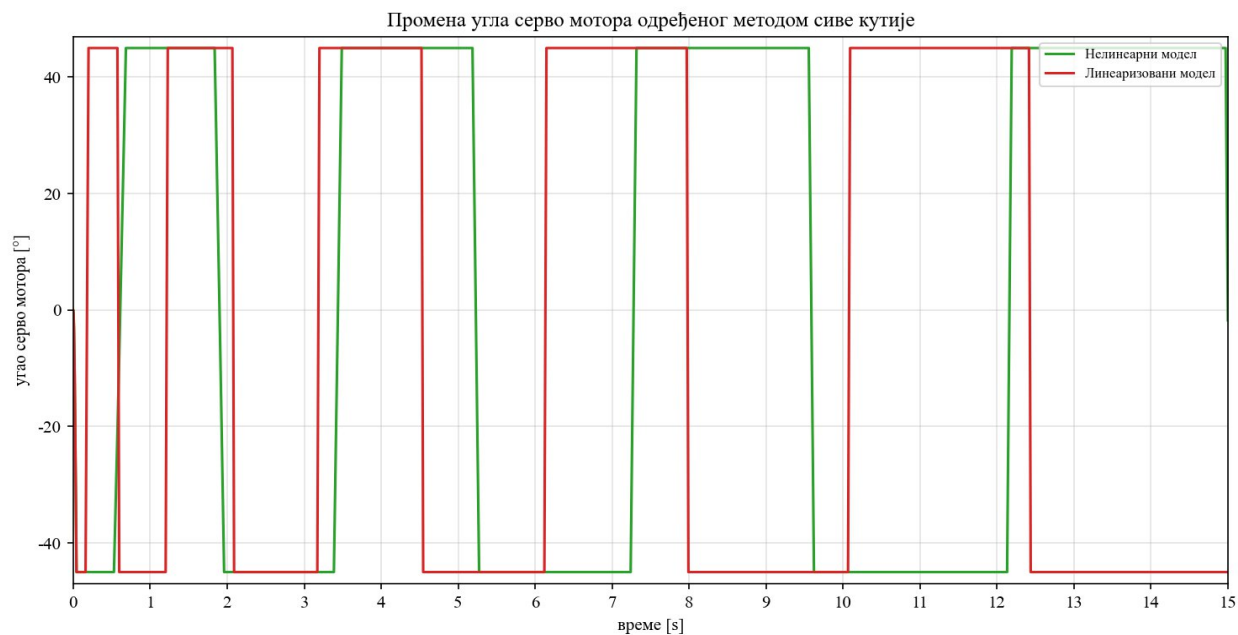


График промене угла модела серво мотора одређеним методом сиве кутије – Слика 4.7

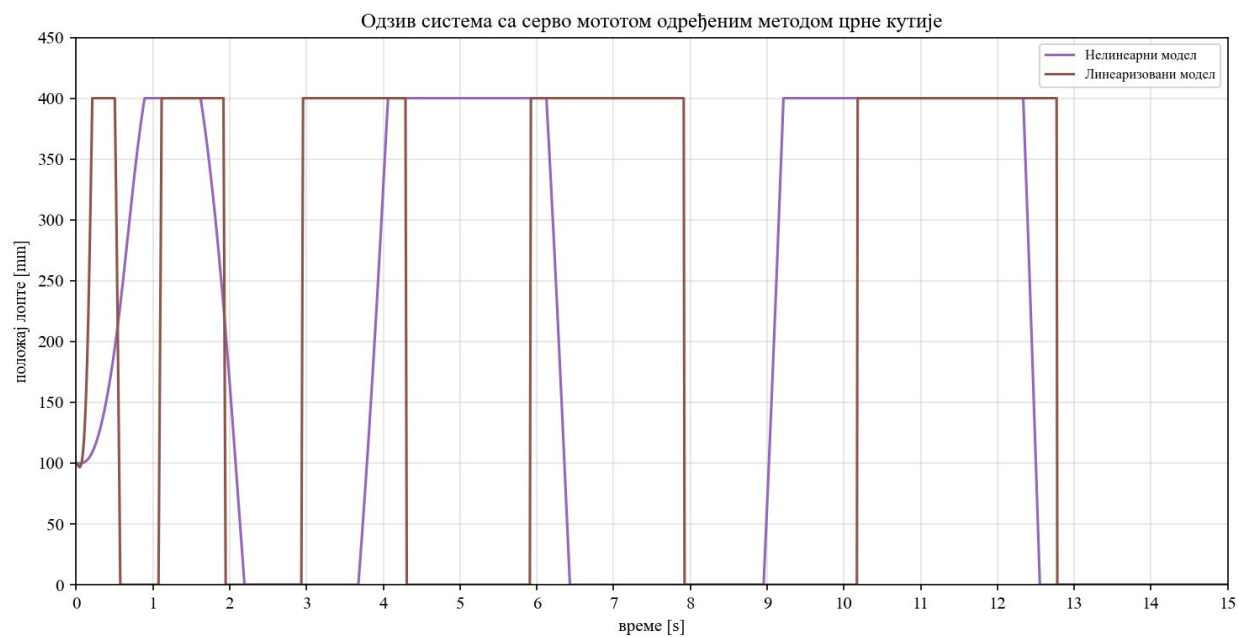


График одзива система са моделом серво мотора одређеним методом црне кутије –
Слика 4.8

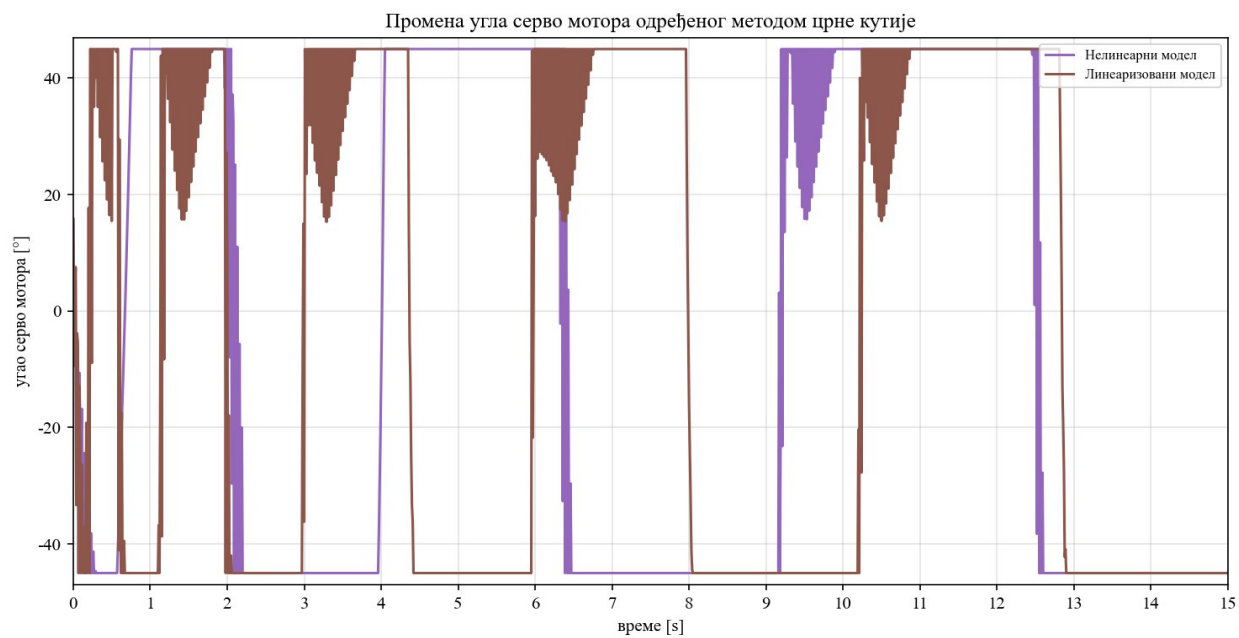
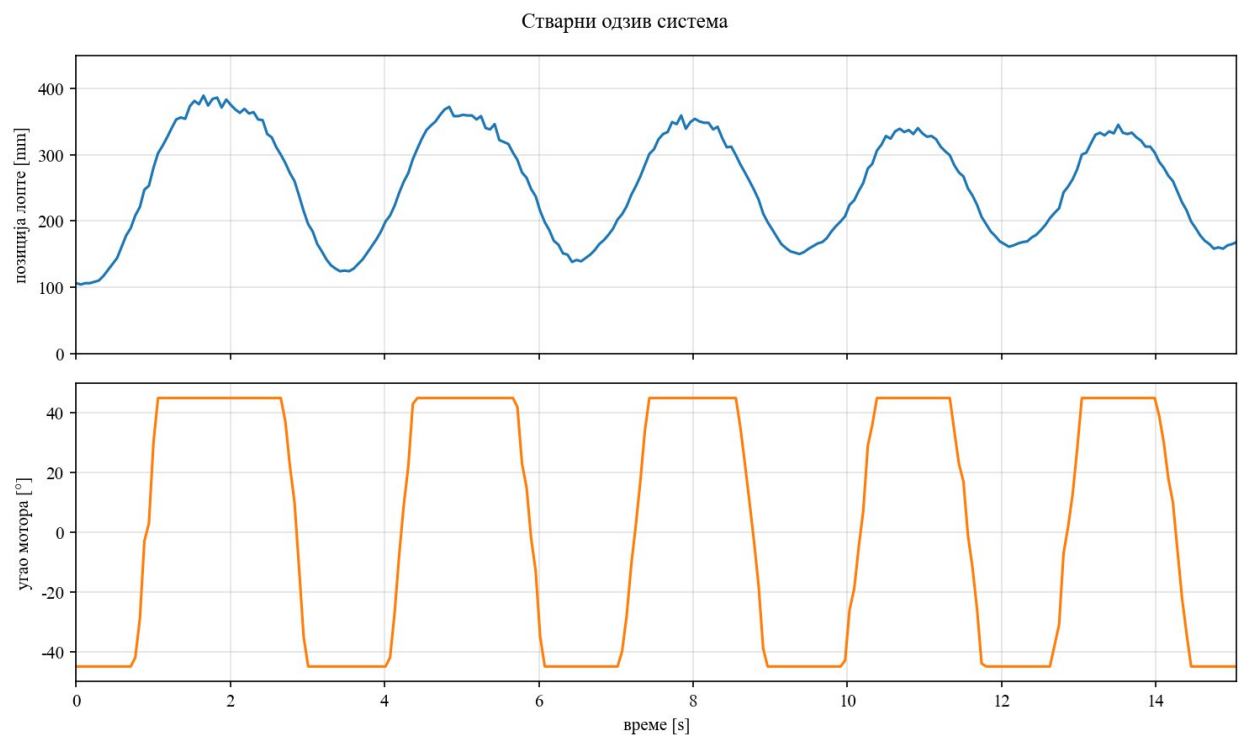


График промене угла модела серво мотора одређеним методом црне кутије – Слика 4.9



Графици одзива и промене угла серво мотора стварног система – Слика 4.10

На основу анализе одзива система и промене угла серво мотора можемо закључити:

- да је систем брз;
- да нелинеарни модел даје глаткији и физички прихватљивији одзив од линеаризованог, који показује брзе, нереалистичне осцилације;
- да су одзиви система са моделом серво мотора одређеним методом сиве кутије и методом црне кутије међусобно доста слични, што се уочава на свим графицима; али је неопходно напоменути да је модел црне кутије је нешто спорији;
- да су промене угла серво мотора готово исте код сиве кутије, црне кутије и стварног система: угао се задржава на граничним вредностима $\pm 45^\circ \pm 45^\circ$;
- да стварни одзив има мању амплитуду осцилација у односу на симулацију, док је фреквенција истог реда величине.

На основу наведеног, за даљи рад усваја се нелинеарни модел кретања лопте по шини. Док због сличног понашања оба модела серво мотора оправдано је пројектовати исти PID регулатор.

4.2. Одабир регулатора

Регулатор представља кључни део система аутоматског управљања. Без њега систем ради у отвореној спрези: сваки нагиб шине различит од хоризонталног равнотежног положаја ствара компоненту гравитационе силе која убрзава лопту дуж шине, па се лопта удаљава од жељене позиције. Затварањем повратне спреге регулатор непрекидно пореди измерену позицију лопте са референцом и на основу те разлике формира управљачки сигнал за серво мотор. Тиме регулатор омогућава стабилизацију нестабилног објекта, као и регулацију — задржавање лопте у близини задате позиције уз присуство поремећаја, несигурности модела и ограничења актуатора.

Избор одговарајуће структуре регулатора није ствар произвољног избора, већ треба да проистекне из особина самог система, расположивих сензора и актуатора, квалитета математичког модела и практичних ограничења имплементације. Да би се одабрала погодна структура регулатора за овај систем, потребно је одговорити на следећа питања.

1. Динамика и физика система

- Да ли је систем линеаран или нелинеаран?

Систем је нелинеаран. Кретање лопте зависи од нагибног угла шине преко тригонометријских релација, а серво мотор је ограничен опсегом угла. На основу анализе из поглавља 4.1, за даљи рад усвојен је нелинеарни модел кретања лопте по шини; линеаризовани модел служи само као упрошћени опис, а не као основни модел за пројектовање регулатора.

- Да ли је систем отворене спреге стабилан или нестабилан?

Систем у отвореној спрези је нестабилан: при константном углу шине различитом од равнотежног положаја лопта сама од себе не мирује на жељеној позицији, већ се убрзава дуж шине. Због тога је неопходна непрекидна повратна спрега. Затварање спреге само по себи није довољно. При испитивању затвореног система са чисто пропорционалним регулатором ($K_p = 1$, $K_r = 1$) одзив је осцилаторан и не смирује се око референце.

- Да ли је систем SISO или MIMO?

По конструкцији систем је SISO: управљачка величина је жељени угао серво мотора, а мерена излазна величина је позиција лопте са ласерског сензора удаљености.

- Колико је брза динамика система?

На основу симулација из поглавља 4.1 закључено је да је систем брз. Идентификовани серво мотор се смирује реда десетине секунде, док се кретање лопте одвија на упоредивом, нешто споријем временском нивоу. Петља управљања на микроконтролеру извршава се са периодом узорковања од 50 ms, што је прихватљиво за овај механички систем.

2. Хардверска и рачунарска ограничења

- Која су ограничења процесора?

Регулатор се имплементира на микроконтролеру Arduino Uno. Овај контролер извршава једну главну петљу, има ограничена меморијска и рачунарска средства и није погодан за сложене алгоритме са великим бројем операција у реалном времену.

- Који сензори и актуатори су на располагању?

Позиција лопте мери се ласерским сензором удаљености VL53L0X. Извршни орган је комерцијални серво мотор непознатог произвођача, који се побуђује ширинско-модулисаним сигналом. Брзина лопте се не мери директно.

- Да ли актуатор има сопствени унутрашњи регулатор?

Да. Комерцијални серво мотор већ садржи унутрашњу повратну спрегу по углу. Спољни регулатор стога задаје жељени угао, а не момент или напон, што природно води ка каскадној структури: унутрашња петља серво мотора и спољна петља која стабилизира позицију лопте.

3. Модел и познавање параметара

- Да ли постоји довољно тачан математички модел?

Да, за потребе симулационог пројектовања. На располагању је нелинеарни модел кретања лопте по шини, као и модели серво мотора добијени методом сиве и методом црне кутије. Ови модели су довољно добри за анализу затворене спреге и подешавање параметара регулатора, али нису савршени за агресивну инверзију модела или сложено нелинеарно управљање.

- Колика је очекивана варијација параметара?

Очекује се умерена варијација параметара. Она потиче од разлика у идентификованим моделима серво мотора, трења и зазора у 3D штампаној механици, заосталог нагиба шине, шума сензора и промене ефективног појачања при већим угловима. Због тога регулатор треба да буде једноставан и довољно робустан.

4. Критеријуми перформанси

Основни циљ је стабилизација система отворене спреге и регулација позиције лопте на задату референцу (у симулацијама 250 mm), уз ограничење угла серво мотора на опсег $\pm 45^\circ$, прихватљиво пререгулисање и смиривање одзива. Детаљнији квантитативни критеријуми проверавају се симулацијом на нелинеарном моделу, а затим на стварном систему.

На основу претходне идентификације система, анализе из поглавља 4.1 и чињенице да је реч о лабораторијском („уради сам“) систему са Arduino Uno контролером, серво мотором непознатог произвођача са унутрашњим регулатором и 3D штампаном механиком, одлучује се да се пројектује PID регулатор или нека његова редукована варијанта (нпр. PD или PI). Ова фамилија регулатора одговара расположивим мерењима, рачунарским ограничењима микроконтролера и усвојеном нелинеарном моделу за пројектовање. Која ће се тачно комбинација чланова (P, I и/или D) задржати утврђује се у наставку, кроз подешавање параметара и проверу на симулацији и стварном систему.

4.3. Пројектовање и подешавање регулатора

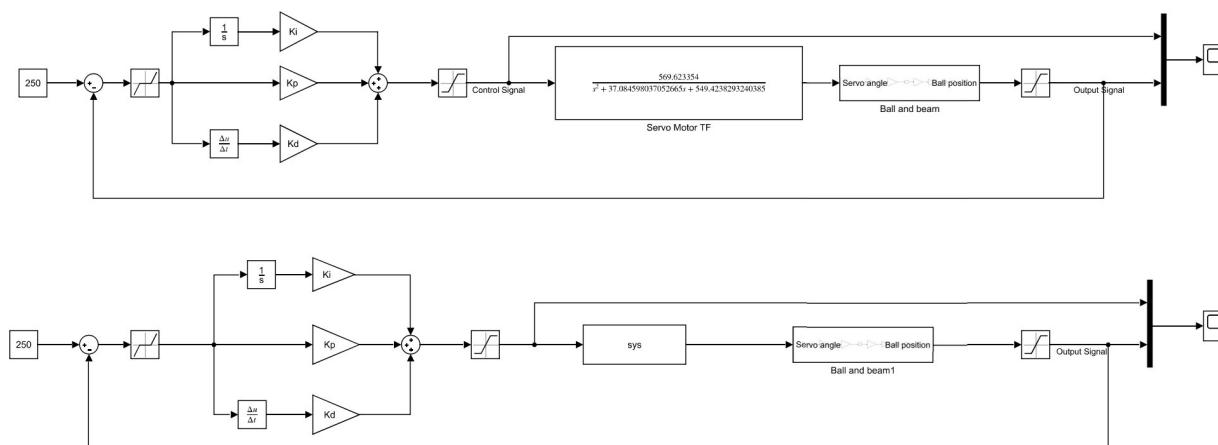
Како је у претходном делу одлучено да се за управљање системом користи PID регулатор, у овом делу приступа се његовом пројектовању и подешавању параметара. Рад започиње једноставним паралелним PID регулатором. Уколико се током подешавања и испитивања покаже да овај облик није задовољавајућ, размотриће се и друге познате конфигурације PID регулатора (нпр. серијска / интерактивна форма или облик са филтрираним диференцијалним чланом).

Подешавање параметара спроводи се итеративно, поступним повећањем вредности појачања са неједнаким корацима. Након сваке пробе посматра се одзив затвореног система (и, по потреби, промена угла серво мотора), па се бира вредност која даје најповољније понашање према усвојеним критеријумима перформанси.

Неопходно је напоменути да је приликом склапања и подешавања система уочено да ласерски сензор удаљености за исту позицију лопте не даје увек исту измерену вредност, већ да читавања осцилују у опсегу од приближно $\pm 5mm$. Да би се избегло непрекидно корекционо дејство регулатора услед ове варијације мерења када је лопта већ у близини референце, у управљање је уведена **мртва зона**: ако је апсолутна вредност грешке мања или једнака 5mm, сматра се да је лопта довољно близу жељене

позиције и управљачки сигнал се не коригује додатно. Иста мртва зона унета је и у Simulink модел затвореног система, како би симулација верније пратила понашање стварне имплементације.

На слици 4.11 приказан је блок дијаграм комплетног затвореног система са паралелним PID регулатором, пре подешавања параметара K_p , K_i и K_d . У горњем делу приказан је систем са моделом серво мотора добијеним методом сиве кутије, а у доњем делу систем са моделом добијеним методом црне кутије. Структура регулатора, мртва зона и ограничења актуатора исте су у оба случаја; разликује се само модел серво мотора.



Блок дијаграм затвореног система са паралелним PID регулатором: горе модел серво мотора методом сиве кутије, доле методом црне кутије – Слика 4.11

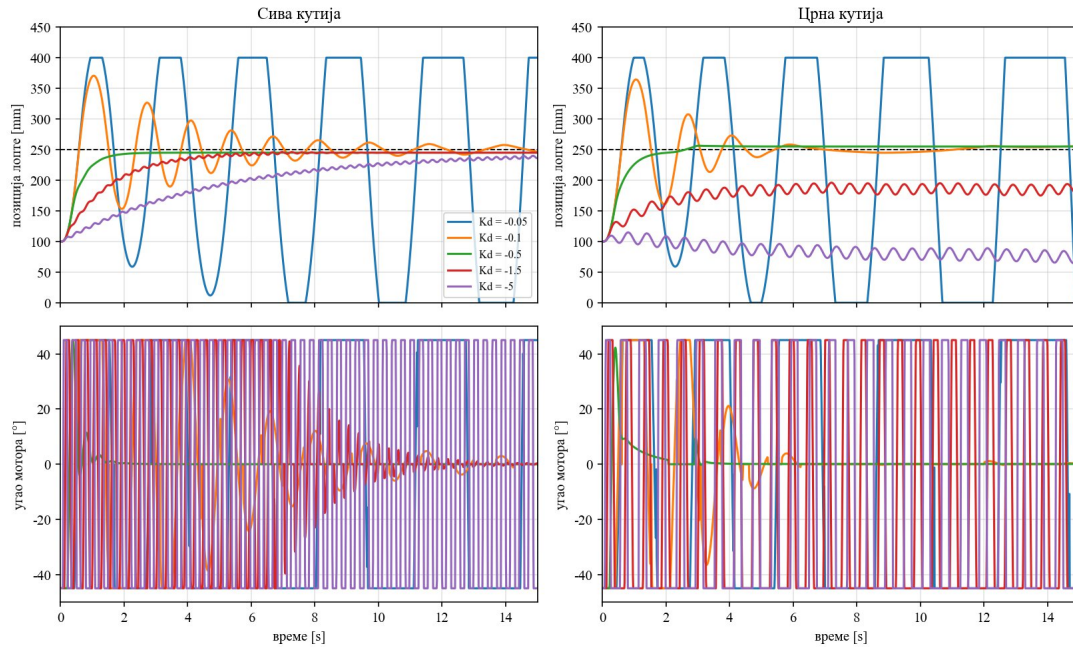
Подешавање диференцијалног појачања K_d

Подешавање је започето диференцијалним појачањем. На основу претходне анализе, задржано је $K_p = -1$ и $K_i = 0$, док је K_d мењано неравномерно инкрементално. Циљ овог корака био је да се унесе довољно пригушења како би се смањиле осцилације затвореног система.

У првој итерацији испитан је шири опсег вредности K_d . Репрезентативни одзиви приказани су на слици 4.12. Уочено је следеће:

- при малим вредностима $|K_d|$ (нпр. $K_d = -0,05$) систем остаје јако осцилаторан и не смирује се око референце;
 - око $K_d = -0,5$ одзив постаје прихватљив на оба модела серво мотора;
- при превеликом $|K_d|$ (нпр. $K_d = -1,5$ и $K_d = -5$) долази до погоршања, посебно код модела црне кутије, уз изражено засићење угла серво мотора.
- На основу прве итерације закључено је да се повољно подручје налази око $K_d \approx -0,5$, па је у другој итерацији извршено финије подешавање у том опсегу.

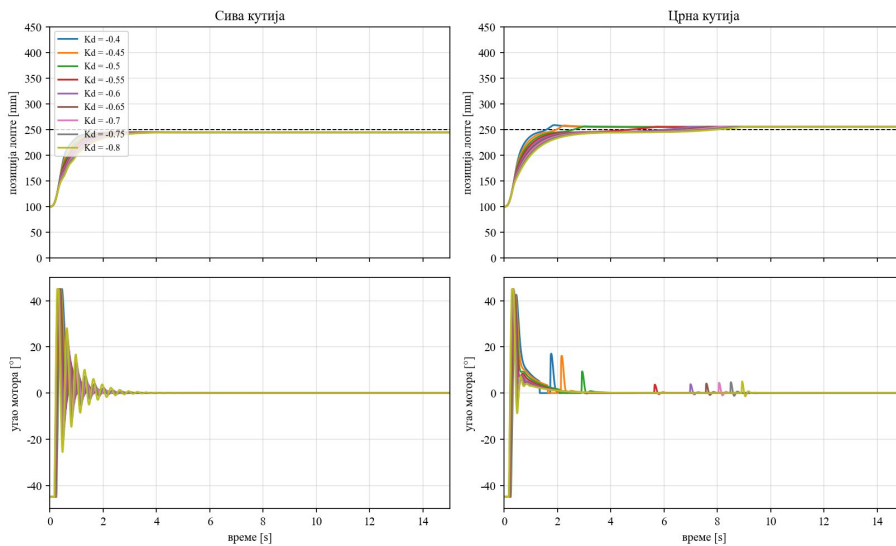
Симулација PID, промена K_d ($K_p = -1$, $K_i = 0$)



Одзив система при промени K_d ($K_p = -1$, $K_i = 0$): позиција лопте и угао серво мотора –
Слика 4.12

У другој итерацији K_d је мењано у ужем опсегу око $-0,5$ (од $-0,4$ до $-0,8$), и даље при $K_p = -1$ и $K_i = 0$. Одзиви су приказани на слици 4.13, а основни показатељи перформанси у табели X.X. Време смиривања T_s дефинисано је као тренутак након којег грешка остаје унутар $\pm 10\text{mm}$.

Симулација PID, промена K_d ($K_p = -1$, $K_i = 0$)



Фино подешавање K_d : позиција лопте и угао серво мотора (сива и црна кутија) –
Слика 4.13

K_d	T_s сива [s]	T_s црна [s]	Π сива [%]	Π црна [%]
-0,4	1,26	1,08	0	1,5
-0,5	1,63	1,47	0	0,5
-0,6	1,99	1,90	0	0,1
-0,7	2,34	2,28	0	0,1
-0,8	2,68	2,63	0	0,1

Утицај K_d (друга итерација; $K_p = -1$, $K_i = 0$) – Табела 4.1

Све испитане вредности у овом опсегу дају стабилан одзив. Мање $|K_d|$ даје брже смиривање, али нешто већи прескок код модела црне кутије. Веће $|K_d|$ даје јаче пригушење и спорији одзив. За даљи рад усваја се: $K_d = -0,7$.

Ова вредност није најбржа у табели, али оставља резерву пригушења пре следећег корака — подешавања K_p . Очекивано је да повећање пропорционалног појачања убрза систем и може повећати пререгулисање, па је намерно задржано нешто јаче диференцијално дејство.

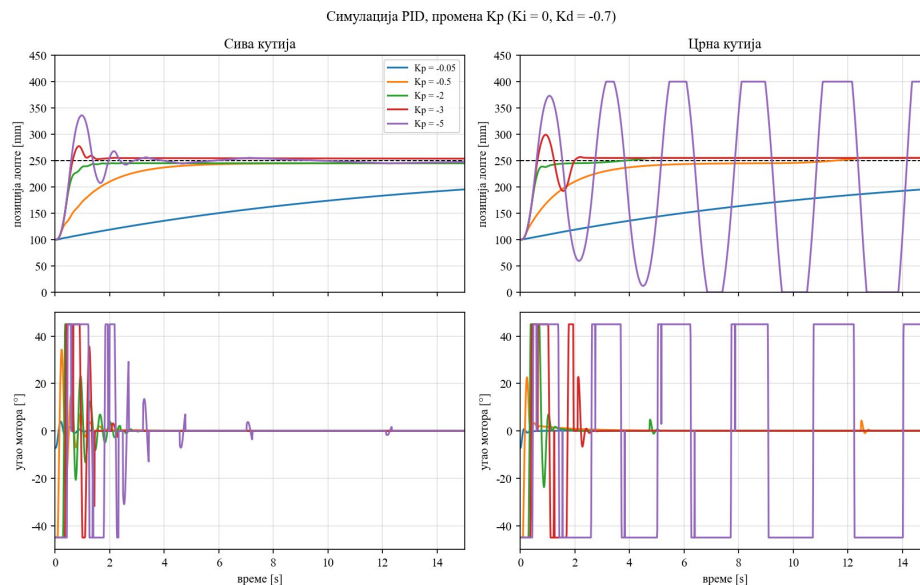
Подешавање пропорционалног појачања K_p

Након усвајања $K_d = -0,7$, подешено је пропорционално појачање при $K_i = 0$.

У првој итерацији испитан је шири опсег K_p (слика 4.14). Уочено је:

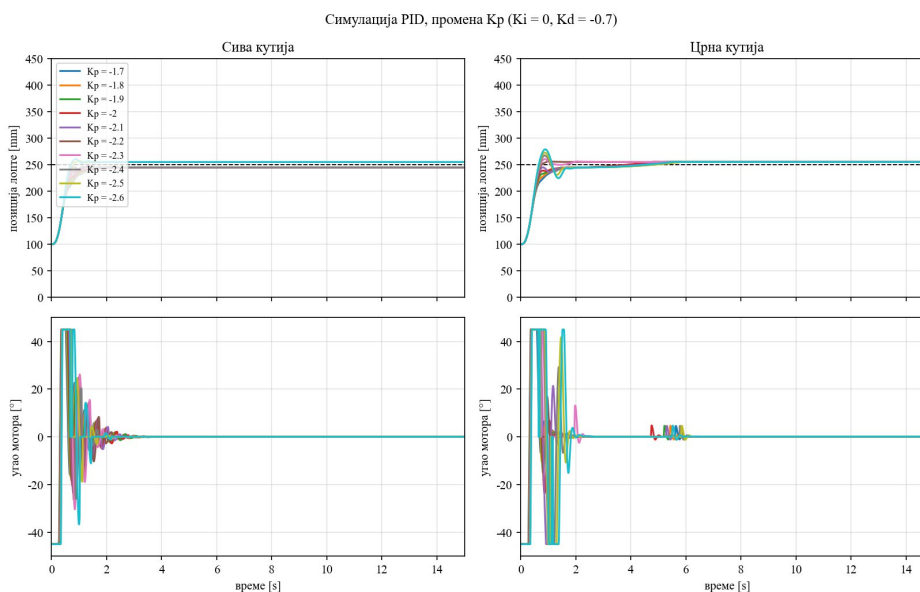
- при премалим вредностима $|K_p|$ (нпр. $-0,05$) систем не достиже референцу у посматраном времену;
- око $K_p = -2$ добија се добар компромис брзине и пререгулисања на оба модела;
- при превеликом $|K_p|$ (нпр. -5) долази до осцилација и засићења, посебно код модела црне кутије.

Због тога је друга итерација урађена у ужем опсегу око $K_p = -2$.



Одзив система при промени K_p ($K_d = -0.7$, $K_i = 0$): позиција лопте и угао серво мотора – Слика 4.14

Резултати финијег подешавања ($K_d = -0.7$, $K_i = 0$) дати су на слици 4.15 и у табели 4.2.



Фино подешавање K_p : позиција лопте и угао серво мотора (сива и црна кутија) – Слика 4.15

K_p	T_s сива [s]	T_s црна [s]	Π сива [%]	Π црна [%]
-------	----------------	----------------	----------------	----------------

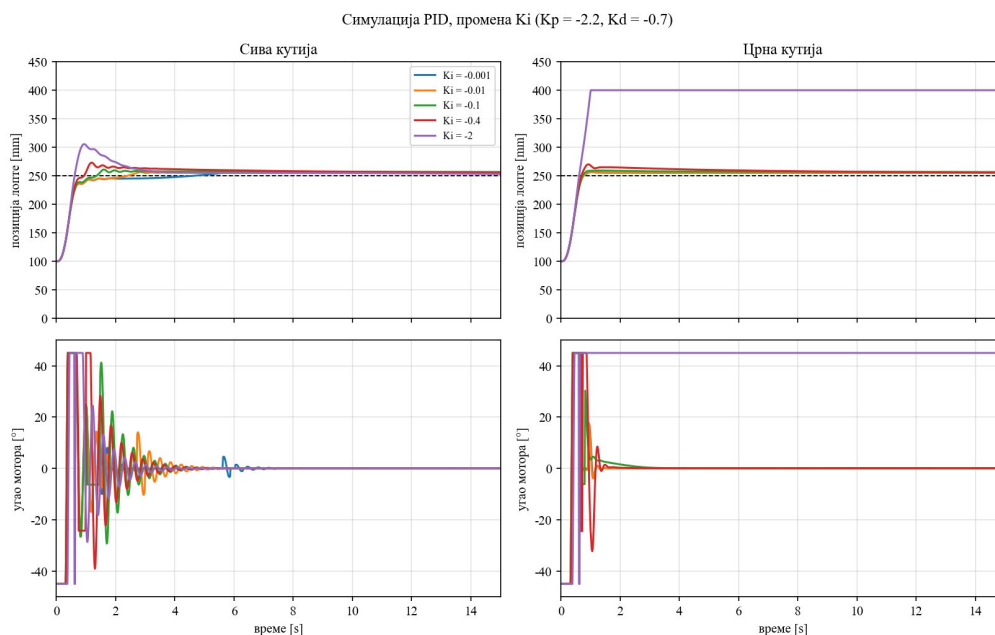
-1,8	1,28	1,23	0	0,1
-2,0	1,22	1,04	0	0,1
-2,2	0,99	0,68	0,2	0,6
-2,4	0,68	1,35	0,5	4,8
-2,6	0,94	1,58	2,4	9,4

Утицај K_p (друга итерација; $K_d = -0,7$, $K_i = 0$) – Табела 4.2

Вредности око $-2,0$ до $-2,2$ дају брз одзив уз мало пререгулисање. Даљим повећањем $|K_p|$ пререгулисање код црне кутије нагло расте. За даљи рад усваја се: $K_p = -2,2$. Тиме је задржана добра брзина одзива, уз прихватљиво пререгулисање на оба модела серво мотора.

Подешавање интегралног појачања K_i

На крају је, при усвојеним $K_p = -2,2$ и $K_d = -0,7$, испитан утицај интегралног члана. Репрезентативни одзиви приказани су на слици 4.16, а показатељи у табели 4.3.



Одзив система при промени K_i ($K_p = -2,2$, $K_d = -0,7$): позиција лопте и угао серво мотора – Слика 4.15

K_i	T_s сива [s]	T_s црна [s]	Π сива [%]	Π црна [%]
-0,001	0,99	0,68	0,2	0,6

-0,01	0,98	0,68	0,6	0,6
-0,1	1,66	0,67	1,7	1,0
-0,4	5,31	5,27	6,7	5,5
-2	2,88	—	20,5	нестабилно

Утицај K_i ($K_p = -2,2$, $K_d = -0,7$) – Табела 4.3

При веома малом $|K_i|$ понашање је готово исто као код PD регулатора. Повећањем $|K_i|$ расту прескоци и време смиривања, а код већих вредности модел црне кутије постаје нестабилан. Поред тога, у систему је присутна мртва зона од $\pm 5mm$. Унутар те зоне грешка се третира као нула, па интегрални члан ионако не може значајно да уклони преосталу грешку реда величине мртве зоне. Због тога се закључује да интегрални члан у овом случају није потребан. Усваја се: $K_i = 0$.

На основу итеративног подешавања на нелинеарном моделу, са моделима серво мотора добијеним методом сиве и методом црне кутије, усваја се следећи регулатор:

$$K_p = -2,2, K_i = 0, K_d = -0,7.$$

За управљање системом користи се једноставан паралелни PD регулатор. Овај регулатор даје задовољавајуће смиривање и регулацију око референце од $250mm$, уз ограничење угла серво мотора и уз мртву зону сензора. Исти закон управљања преноси се на стварни систем (имплементација на Arduino Uno) ради експерименталне провере.

4.4. Имплементација регулатора

Након подешавања параметара PD регулатора у програмском пакету Simulink, исти закон управљања пренет је на стварни систем. Имплементација је урађена на микроконтролеру Arduino Uno, у наменском развојном окружењу Arduino IDE, при чему су коришћене библиотеке Servo.h за управљање серво мотором и Adafruit_VL53L0X за рад са ласерским сензором удаљености.

Основна идеја имплементације јесте затворена повратна спрега која се извршава у реалном времену. У сваком кораку главне петље микроконтролер:

- очитава тренутну позицију лопте помоћу ласерског сензора удаљености,
- израчунава грешку и вредност управљачког сигнала применом PD закона управљања,
- шаље израчунату вредност серво мотору у облику ширинско-модулисаног сигнала.

Сензор даје информацију о стварној позицији лопте, микроконтролер врши потребна израчунавања, а извршни орган прима жељени угао шине преко ширинско-модулисаног сигнала. Такође је битно напоменути да за рад:

- безбедног постављања лопте на шину пре почетка мерења,
- могућности прекида експеримента без искључивања напајања,
- једноставне контроле током испитивања и прикупљања података,

у извршни код је додата и петља којом можемо да контролишемо покретање и заустављање система. Регулација се не покреће аутоматски, већ тек након команде G послате преко серијске комуникације, док команда S зауставља систем и враћа серво мотор у неутрални положај.

Алгоритам програма може се поделити у три целине, приказане на слици 4.16:

- Иницијализација: Покреће се серијска комуникација, серво мотор се повезује на одговарајући пин и поставља у неутрални положај, а затим се иницијализује ласерски сензор удаљености и покреће се његов континуални режим мерења.
- Провера команди: У главној петљи проверава се да ли је преко серијске комуникације стигла команда. Командом G систем прелази у режим рада, док се командом S регулација зауставља и серво мотор враћа у неутрални положај.
- Радна петља: Када је систем активан, извршавају се следећи кораци: очитавање позиције лопте, формирање грешке, израчунавање управљачког угла применом PD закона, слање команде серво мотору, слање података на рачунар и пауза до следећег очитавања сензора.



Алгоритам рада програма за управљање системом – Слика 4.16

Периода узроковања

У симулацијама је коришћен фиксни корак интеграције од $10ms$, док је на стварном систему период главне петље $50ms$. Ова разлика није узета у обзир приликом подешавања параметара регулатора у симулацији. И поред тога, стварни систем са усвојеним параметрима ради задовољавајуће.

Израчунавање управљачког сигнала и претварање у ширинско-импулсне модулације

Дискретни PD закон управљања реализован је у следећем облику:

$$u(k) = K_p e(k) + K_d \frac{e(k) - e(k-1)}{\Delta t}, e \leq |5|$$

где су усвојена појачања $K_p = -2,2$ и $K_d = -0,7$. Уколико је апсолутна вредност грешке мања или једнака $5mm$, активира се мртва зона, при чему је излаз регулатора једнак нули, а претходна вредност грешке се ресетује. Излаз регулатора представља жељени угао шине и ограничен је на опсег $\pm 45^\circ$, чиме је у имплементацији поновљено засићење актуатора усвојено у симулационом моделу.

Добијени угао се затим линеарно пресликава у трајање импулса ширинско-модулисаног сигнала према изразу:

$$PWM = 500 + \frac{\alpha + 135}{270} \cdot 2000[\mu s],$$

што одговара радном опсегу коришћеног серво мотора, односно трајању импулса од 500 до $2500\mu s$ за угао од -135° до 135° (неутрална позиција 180°).

Негативан предзнак појачања K_p и K_d последица је оријентације сензора и смера дејства серво мотора у конкретном склопу, а не другачијег облика регулатора у односу на симулациони модел.

Слање података преко серијске комуникације

Током рада микроконтролер шаље податке у формату *време [ms]*, *позиција лопте [mm]*, *угао серво мотора [°]*, што омогућава касније поређење експерименталног одзива са резултатима симулације. Ова операција се извршава након сваког корака регулације и незнатно продужава трајање главне петље, али не мења структуру алгоритма нити битно утиче на понашање система.

Комплетан програм за управљање системом за балансирање лопте на шини дат је у наставку. Програм обухвата дефиниције параметара, иницијализацију хардвера, реализацију регулатора са мртвом зоном, претварање угла у ширинско-модулисани сигнал, као и логику за покретање и заустављање система преко серијске комуникације.

Током имплементације и тестирања на реалном систему параметри регулатора су незнатно кориговани у односу на вредности добијене у фази пројектовања: пропорционално појачање је снижено на $K_p = -1$, а уведено је и мало интегрално дејство

$K_i = -0,01$. Разлози за ове измене и анализа понашања реалног система приказани су у наредном поглављу.

```
#include <Servo.h>
#include "Adafruit_VL53L0X.h"

const int SERVO_PIN = 5;
const int ZERO_PWM = map(0, -135, 135, 500, 2500);

const float BEAM_MIN = -45.0f;
const float BEAM_MAX = 45.0f;

Servo beamServo;
Adafruit_VL53L0X lox;

const float SETPOINT_MM = 250.0f;
const float TOLERANCE_MM = 5.0f;
const unsigned long LOOP_MS = 50;

float Kp = -1.2f;
float Ki = -0.01f;
float Kd = -0.7f;

bool running = false;
float distance_mm = 0.0f;
float beam_deg = 0.0f;
float integral = 0.0f;
float prev_error = 0.0f;
unsigned long t0_ms = 0;

void setBeamAngle(float angle_deg) {
    float pwm_f = 500.0f + (angle_deg + 135.0f) * (2000.0f / 270.0f);
    int pwm = (int)lround(pwm_f);
    beamServo.writeMicroseconds(pwm);
}

void holdNeutral() {
    beam_deg = 0.0f;
    beamServo.writeMicroseconds(ZERO_PWM);
}

void readDistance() {
    if (!lox.isRangeComplete()) {
        return;
    }

    uint16_t raw = lox.readRange();
    if (raw != 65535) {
        distance_mm = (float)raw;
    }
}

float pidStep(float error) {
    if (fabs(error) <= TOLERANCE_MM) {
```

```

    integral = 0.0f;
    prev_error = 0.0f;
    return 0.0f;
}

const float dt = LOOP_MS / 1000.0f;

integral += error * dt;
float D = Kd * (error - prev_error) / dt;
prev_error = error;

beam_deg = constrain(Kp * error + Ki * integral + D, BEAM_MIN, BEAM_MAX);
return beam_deg;
}

void logData() {
    Serial.print(millis() - t0_ms);
    Serial.print(',');
    Serial.print((int)lround(distance_mm));
    Serial.print(',');
    Serial.println(beam_deg, 2);
}

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(1);
    }

    beamServo.attach(5);
    holdNeutral();

    if (!lox.begin()) {
        Serial.println(F("VL53L0X init failed"));
        while (1) {
            delay(100);
        }
    }

    lox.startRangeContinuous();

    Serial.println(F("Ball and Beam v1"));
    Serial.println(F("G = start, S = stop"));
    Serial.println(F("time_ms,distance_mm,beam_deg"));
}

void loop() {
    if (Serial.available()) {
        char cmd = Serial.read();
        while (Serial.available()) {
            Serial.read();
        }

        if (cmd == 'G' || cmd == 'g') {
            running = true;
            t0_ms = millis();

```

```

    integral = 0.0f;
    prev_error = 0.0f;
    holdNeutral();
    Serial.println(F("RUN"));
} else if (cmd == 'S' || cmd == 's') {
    running = false;
    holdNeutral();
    Serial.println(F("STOP"));
}
}

if (!running) {
    return;
}

readDistance();

float error = SETPOINT_MM - distance_mm;
setBeamAngle(pidStep(error));
logData();

delay(LOOP_MS);
}

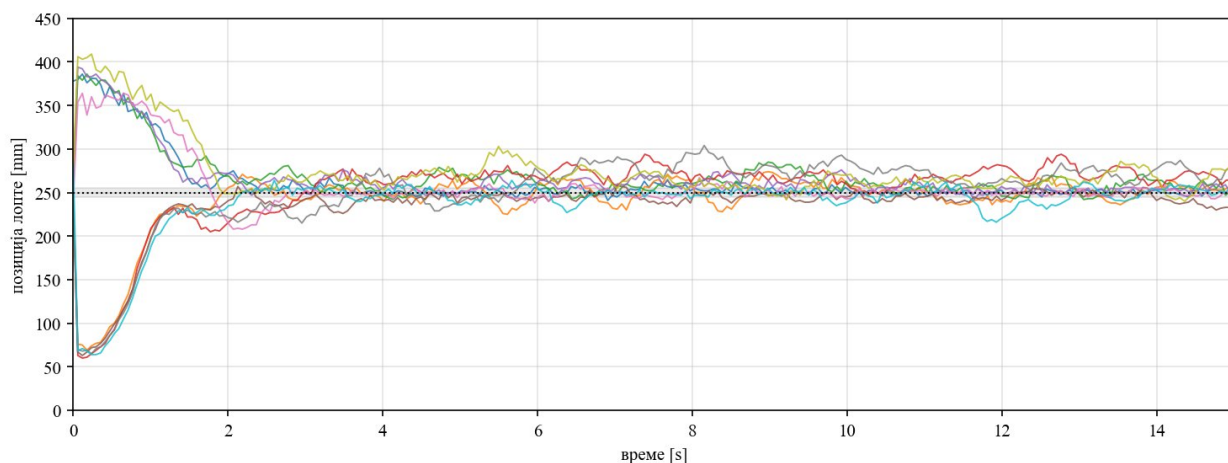
```

4.5. Поређење математичких модела са стварним СИСТЕМОМ

Пре самог упоређивања математичког модела са стварним спроведено је десет узаостопних експеримената под истим условима на основу којих је урађена анализа и уочено следеће:

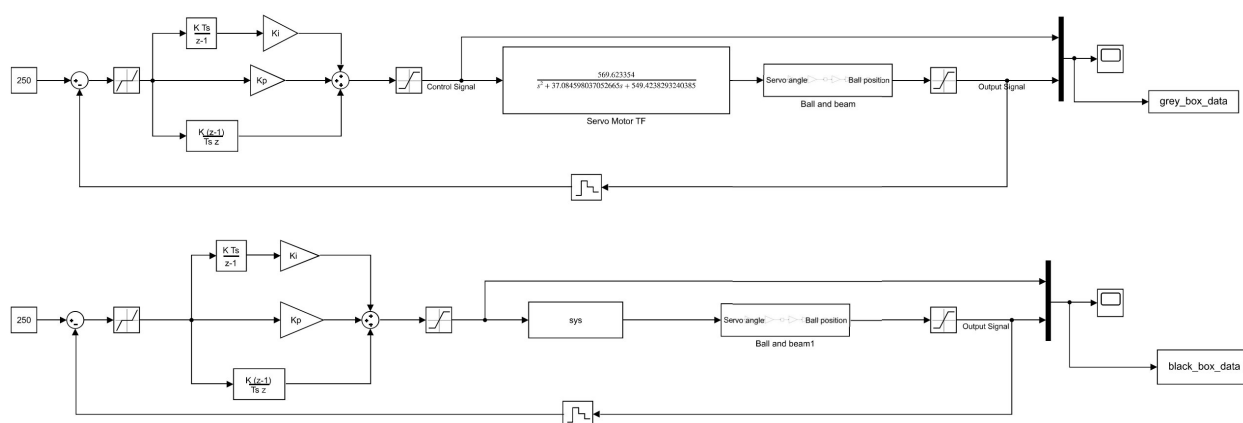
- Прелазни режим је изразито понављив. Лопта долази у околину референтне вредности за $2,6 \pm 0,8s$, уз прескок од 12 до 19mm у зависности од стране са које полази, што показује да је одзив приближно симетричан.
- У устаљеном стању лопта не мирује, већ се одржава ситно таласање око референце са стандардном девијацијом од 7,9mm. Лопта се налази унутар зоне од $\pm 5mm$ у 40% времена, а унутар зоне од $\pm 20mm$ у око 90% времена.
- Стварни период одабирања износи 59ms уместо номиналних 50ms, а стандардна девијација шума мерења положаја износи приближно 3mm, при чему расте са растојањем: од 1,8mm на блиским растојањима до 5,8mm преко 350mm.
- Јавља се статичка грешка: лопта се у просеку не смирује на референтној позицији, већ 7,0mm даље од ње, при чему се појединачни експерименти крећу у распону од -5,3 до + 24,4mm.

Понављивост: десет узастопних експеримената



Стварни одзив система (десет експеримената) – Слика 4.17

Да би поређење са мерењима било смислено, Simulink модел је најпре ажуриран (слика 4.18) параметрима регулатора коришћеним на реалном систему. Такође поред је измењен и начин на који је регулатор реализован у моделу. Уместо континуалног регулатора, у повратну спрегу је уведено коло задршке нултог реда са периодом одабирања од $59ms$, диференцијално дејство реализовано је дискретним блоком, а интегрално дејство дискретним интегратором.



Финални Simulink модел – Слика 4.18

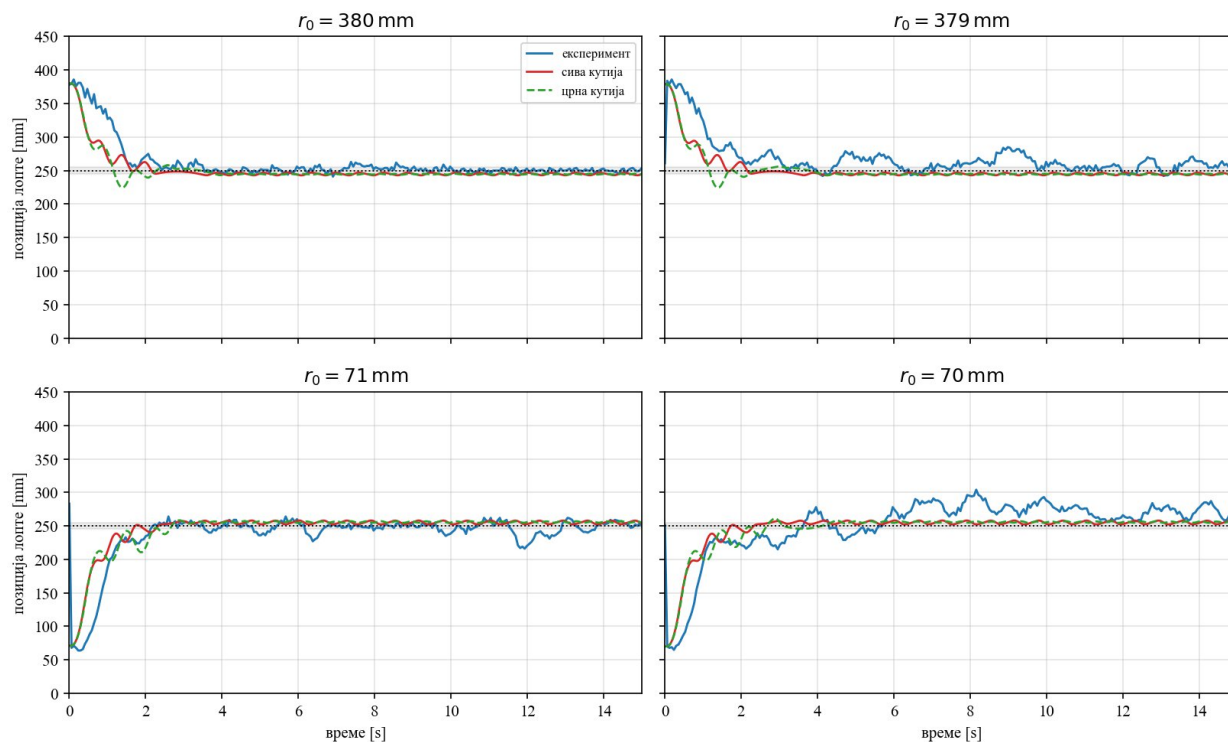
За проверу верности модела изабрана су четири насумична експеримента — два са горње и два са доње стране шине — са почетним положајима $r_0 \in \{380, 379, 71, 70\} \text{ mm}$. За сваки експеримент изведена је симулација са истим почетним условом, посебно за модел са серво мотором одређеним методом сиве кутије и посебно за модел са серво мотором одређеним методом црне кутије. Динамика лопте, регулатор, мртва зона и период одабирања идентични су у обе симулације; једина разлика је модел серво мотора. Тиме се поређење своди на питање који од два модела боље репродукује понашање реалног система у затвореној спрези.

Степен преклапања изражен је у процентима као:

$$100 \left(1 - \frac{\|y - \hat{y}\|}{\|y - \bar{y}\|} \right),$$

где је y измерени положај лопте, $\hat{y}$ одзив симулације, а $\bar{y}$ средња вредност мерења. Вредност од 100% означава потпуно преклапање, док вредност близу нуле значи да модел није бољи од пуког предвиђања средње вредности.

Поређење симулације и експеримента за четири почетна услова



Поређење симулације и експеримента за четири почетна услова – Слика 4.19

На основу слике и израчунатог преклапања уочава се следећ:

- Оба модела дају стабилан одзив и прате смер кретања лопте у свим четири случајевима. Просечно преклапање на целој трајекторији износи 35,5% за модел сиве кутије и 31,5% за модел црне кутије. У прелазном режиму (0–4s) разлике су израженије: 32,6% наспрам 27,0%. Модел сиве кутије, дакле, нешто боље преклапа мерење, али разлика није велика.
- При поласку одозго ($r_0 = 380mm$ и $r_0 = 379mm$) оба модела су бржа од експеримента: у зону од $\pm 10mm$ улазе за око 2s, док експеримент за то треба 3,4s у најбољем случају. Преклапање је највеће код експеримента са $r_0 = 380mm$ (45% сива кутија, 36% црна кутија). Код $r_0 = 379mm$ преклапање пада (16% односно 11%), јер експеримент у устаљеном стању показује израженије осциловање и статичку грешку које симулација не репродукује.
- При поласку одоздо ($r_0 = 71mm$ и $r_0 = 70mm$) модели су међусобно скоро изједначени, са преклапањем од око 40%. Модел црне кутије овде достиже зону $\pm 10mm$ приближно у исто време као експеримент ($\approx 2,6s$), док је модел сиве кутије и даље бржи ($\approx 1,6s$).

Ни један модел не репродукује статичку грешку реалног система. У симулацији се лопта зауставља на ивици мртве зоне (око 245mm), док на поставци средња вредност често лежи изнад референце. Разлог је што модел не садржи механички нагиб нултог положаја греде. Осциловање у устаљеном стању на реалном систему такође је веће него у симулацији, јер симулација не садржи шум мерења који се на поставци појачава диференцијалним дејством.

Поређење у затвореној спрези показује да модел са серво мотором одређеним методом сиве кутије нешто верније репродукује понашање управљаног система од модела одређеног методом црне кутије, при истим параметрима регулатора и истим почетним условима. Разлика у преклапању је умерена. Модел црне кутије и даље даје стабилан и употребљив одзив, посебно при поласку одоздо, али за пројектовање регулатора поузданији опис постројења пружа модел сиве кутије.

5. Закључак

Циљ овог мастер рада био је пројектовање и израда система за балансирање лопте на шини, одређивање његовог математичког модела — са посебним акцентом на идентификацију серво мотора методом црне кутије — као и пројектовање, имплементација и испитивање регулатора на реалној поставци.

У првом делу рада систем је пројектован и израђен. Изабрана је конфигурација са непокретним ослонцем на једном крају шине и погоном преко система полуга на другом. Положај лопте мерен је ласерским сензором растојања, као извршни орган коришћен је серво мотор, а управљање је реализовано на микроконтролеру Arduino Uno. Нестандардни делови израђени су адитивном технологијом, а систем је склопљен и ожичен у складу са пројектованом шемом.

У другом делу одређен је математички модел система. Динамика лопте на шини описана је на основу закона механике. За серво мотор, чија унутрашња структура није у потпуности позната, спроведена је идентификација на основу експериментално прикупљених улазно-излазних података. Модел је одређен на два начина: методом сиве кутије, полазећи од општег модела електромотора једносмерне струје са редуктором и естимацијом параметара преносне функције, и методом црне кутије, применом неуронске мреже / NARX структуре. Оба модела валидирана су на независном скупу података и уграђена у Simulink окружење, заједно са нелинеарним моделом кретања лопте.

У трећем делу пројектован је и имплементиран регулатор. На основу карактеристика објекта, ограничења хардвера и захтева квалитета регулације усвојен је PID регулатор. Параметри су подешени итеративно у симулацији, а затим пренети на микроконтролер. При имплементацији уведена је мртва зона ради смањења утицаја шума мерења у близини референце, а током испитивања на реалном систему додато је интегрално дејство и кориговано пропорционално појачање. Регулатор успешно стабилизује лопту око жељене позиције од 250mm.

На крају је извршено поређење симулације и експеримента за четири почетна услова. Оба модела серво мотора дају стабилан одзив и прате смер кретања лопте. Просечно преклапање са мерењем износи око 35,5% за модел одређен методом сиве кутије и око 31,5% за модел одређен методом црне кутије. Модел сиве кутије стога нешто верније описује понашање управљаног система у затвореној спреси, док модел црне кутије остаје употребљив и даје блиске резултате, посебно при поласку лопте са доње стране шине. Разлике између симулације и експеримента — бржи одзив модела, одсуство статичке грешке и слабије осциловање у устаљеном стању — последица су тога што модел кретања лопте по шини није потпун и што у њега нису урачунати разни физички фактори и несавршености реалног система.

На основу свега изнетог може се закључити да је циљ рада остварен: израђен је функционалан систем за балансирање лопте на шини, идентификовани су модели серво мотора методама сиве и црне кутије, пројектован и имплементиран је регулатор који стабилно управља системом, а валидацијом у затвореној спрези показано је да оба модела могу да се користе за анализу и пројектовање, уз благу предност модела сиве кутије у погледу преклапања са мерењем на реалној поставци.

6. Додатак

6.1. Додатак А – Техничка документација стандардних компоненти

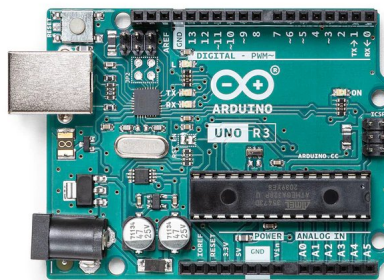
6.1.1. Микроконтролер – Arduino Uno

У данашње време је немогуће замислити аутоматски систем без присуства неког типа рачунара. Рачунарске јединице попут PLC-а се данас користе у различитим индустријама где је потребно управљати робусним стварима попут производних линија и котловима за прављење полуфабриката од различитих метала, док се рачунари попут микроконтролере и микропроцесоре користе у авионима, аутомобилима, али и многим другим индустријама за обраду сигнала и фино управљање актуатора.

У системима аутоматског управљања рачунари на улаз добијају сигнал жељеног динамичког понашања система (мада то често може бити и записано у његовој меморији), као и мерени одзив система, односно стварно динамичко понашање система. На основу ових улазних величина и одговарајућег алгорита управљања он израчунава вредност управљања.

За потребе овог система изабрана је Arduino Uno (слика 2.3) развојна плоча са Atmel-овим микроконтролером. Arduino Uno је платформа отвореног кода коју је развио Arduino тим. Ова платформа се састоји од хардверског и софтверског дела, а користи се за стварање и контролисање интерактивних електронских пројеката.

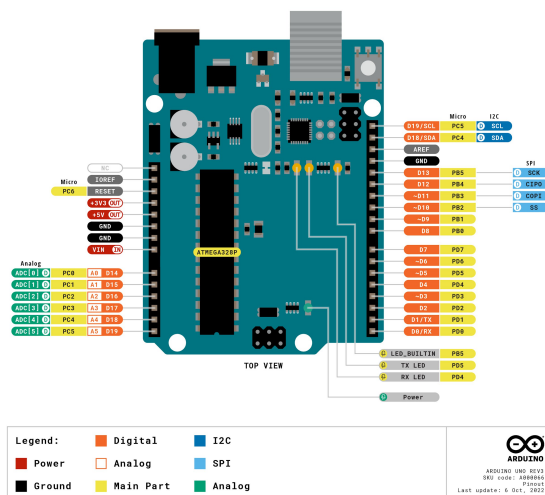
Техничке карактеристике и шема улазно/излазних пинова овог микроконтролера су приказани у табели 7.1 и слици 7.1.



Arduino Uno – Слика 7.1

Микроконтролер	ATmega328P
Радни напон	5 V
Улатни напон	7-12 V
Дигитални У/И пинови	14
PWM дигитални У/И пинови	6
Аналогни улазни пинови	6
Једносмерна струја по У/И пину	20 mA
Флеш меморија	32 KB (ATmega328P)
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Радни такт	16 MHz
Дужина	68.6 mm
Ширина	53.4 mm
Маса	25 g

Техничке карактеристике микроконтролера Arduino Uno – Табела 7.1



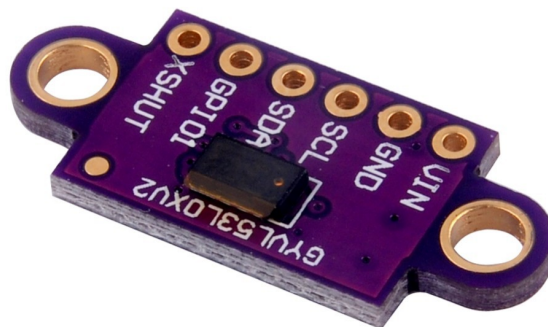
Шема улазно/излазних пинова – Слика 7.2

6.1.2. Сензор растојања – VL53L0X

VL53L0X је сензор за мерење растојања заснован на **Time-of-Flight (ToF)** технологији. Сензор користи инфрацрвени VCSEL емитер таласне дужине 940 nm и SPAD пријемну матрицу за детекцију одбијеног сигнала. На основу времена проласка светлосног сигнала до објекта и назад, интерна електроника сензора одређује растојање. Сензор омогућава мерење апсолутног растојања до 2 m. Принцип рада сензора може се описати кроз следеће кораке:

1. **Емитовање сигнала:** Сензор емитује кратке импулсе инфрацрвеног зрачења помоћу VCSEL емитера.
2. **Одбијање сигнала:** Емитовано зрачење се одбија од површине објекта.
3. **Пријем сигнала:** SPAD пријемна матрица региструје повратни сигнал.
4. **Израчунавање растојања:** Уграђена електроника одређује време проласка сигнала и на основу њега израчунава растојање до објекта.

Комуникација са микроконтролером остварује се путем **I²C интерфејса**, док се сигнал GPIO1 може користити за генерисање прекида након завршетка мерења. Сензор такође поседује XSHUT улаз који се користи за ресетовање и искључивање сензора. За потребе овог система коришћен је **High Accuracy режим мерења**, са временским буџетом од **200 ms**, чиме је приоритет дат стабилности и прецизности мерења растојања. Техничке карактеристике VL53L0X сензора приказане су у табели 7.2.



Сензор растојања VL53L0X – Слика 7.3

Техничка карактеристика	Вредност
Тип сензора	Time-of-Flight (ToF)
Инфрацрвени емитер	VCSEL, 940 nm
Максимално растојање	до 2 m
Радни напон	2,6 – 3,5 V
Радна температура	–20 °C до 70 °C
Комуникациони интерфејс	I ² C
Димензије сензора	4,4 × 2,4 × 1,0 mm
Временски буџет мерења	200 ms

Техничке карактеристике сензора VL53L0X – Табела 7.2

6.1.3. Серво мотор

Серво мотори једносмерне струје су специјализовани типови електромотора дизајнирани за прецизну контролу угла или линеарне позиције, брзине и убрзања. Широко се користе у апликацијама где је прецизна контрола кретања критична, као што су роботи, CNC машине и аутоматизациони системи.

За овај пројекат изабран је серво мотор непознатог произвођача, који је широко коришћен модел, погодан за различите апликације, укључујући хоби роботiku, возила на даљинско управљање (RC) и основне задатке аутоматизације. Пружа добар баланс између перформанси и цене, чинећи га популарним избором како за почетнике тако и за искусне програмере. Техничке карактеристике овог мотора можете наћи у табели 7.3.

Тип мотора	Стандардни серво мотор једносмерне струје
Радни напон	6V до 8.3V
Стални обртни момент	6V: 58 kg.cm, 8.3V: 70 kg.cm
Радна брзина	6V: 0.17 сек/60°, 8.3V: 0.13 сек/60°
Тип зупчаника	Метални зупчаници
Контролни систем	PWM (Ширина импулса модулација)
Опсег ширине импулса	500-2500 μ s
Опсег ротације	270°
Тежина	158g
Димензије	40.7mm x 19.7mm x 42.9mm
Тип конектора	тропински JR
Тип лежаја	Једноредни куглични лежај
Температурни опсег	-15°C до +70°C
Материјал кућишта	Алуминијум

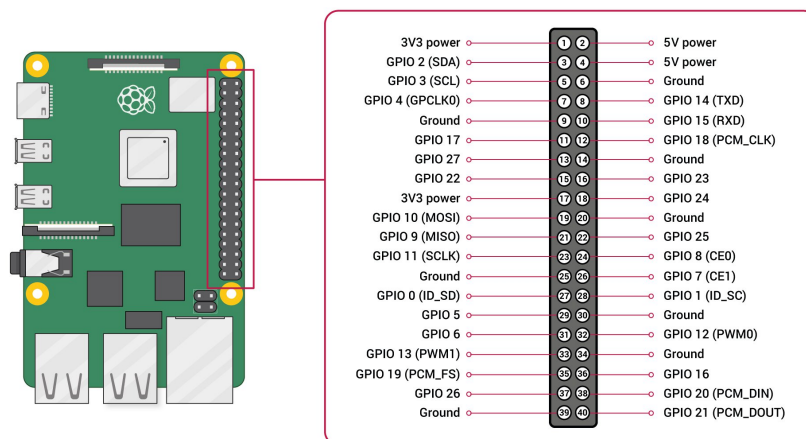
Техничке карактеристике серво мотора – Табела 7.3



Серво мотора – Слика 7.4

6.1.4. Микропроцесор - Raspberry Pi 4

Raspberry Pi 4 је моћна и свестрана развојна плоча са микропроцесором, дизајнирана за широк спектар апликација, од образовања до напредних уграђених система. Са својим четворојезгарним ARM Cortex -A72 процесором, вишеструким опцијама меморије и обимним могућностима повезивања, Raspberry Pi обезбеђује процесорску снагу и флексибилност неопходну за једноставне прототипе, као и за сложене уграђене пројекте. Поддржава два 4К дисплеја, брзе USB 3.0 везе и има робусне могућности умрежавања, што га чини идеалним за мултимедијалне апликације, IoT уређаје и много више. Било да развијате хоби пројекат малог обима или индустријску апликацију у пуном обиму, Raspberry Pi је поуздана и богата функцијама платформа. Техничке карактеристике овог сензора се могу пронаћи у табели 7.4.



Raspberry Pi 4 и шема улазно/излазних пинова – Слика 7.5

Процесор	Broadcom BCM2711, Quad-core Cortex-A72 (ARM v8) 64-bit, 1.5 GHz
Графичка карта	VideoCore VI, подржава OpenGL ES 3.0, Vulkan, H.265 (4Kp60 декодирање)
Меморија (RAM)	8GB LPDDR4 SDRAM
Меморија за складиштење	MicroSD картица (подржава велике брзине UHS-I), USB boot support
Мрежа	Gigabit Ethernet, Dual-band 802.11ac Wi-Fi, Bluetooth 5.0
USB портови	2x USB 3.0, 2x USB 2.0
Видео излаз	2x Micro-HDMI (подржава резолуцију до 4Kp60)
Аудио излаз	четворо полни стерео излаз и композитни видео, HDMI аудио, I ² S аудио
Улазно/Излазни Пинови	40 пинова, 3.3V логика
Интерфејс за камеру	MIPI CSI
Интерфејс за дисплеј	MIPI DSI (Display Serial Interface)
Напајање	5V/3A преко USB-C конектора
Оперативни системи	Raspberry Pi OS, Ubuntu, Manjaro, LibreELEC, RetroPie
Димензије	85.6mm x 56.5mm x 17mm
Маса	46g
Типови комуникације	2x UART, 2x SPI, 5x I ² C, 27x GPIO, 1x PCM/I ² S, 1x SDIO
Потрошња енергије	У стању мировања 2.7W, приликом максималног оптерећења 7W

Техничке карактеристике микропроцесора Raspberry Pi 4 – Табела 7.4

6.1.5. Ротациони енкодер – LDP3806–600BM–5G –24C

Ротациони енкодер је електромеханички уређај који конвертује угаону позицију или кретање осовине у аналогни или дигитални сигнал. Ови сигнали су кључни за одређивање позиције, брзине и правца ротационог кретања. Ротациони енкодери се широко користе у апликацијама које захтевају прецизну контролу ротационог кретања, као што су системи за контролу мотора, роботика, CNC машине и индустријска аутоматизација.

LDP3806-600BM-G5-24C је инкрементални ротациони енкодер познат по својој високој резолуцији и робусном дизајну, што га чини погодним за апликације које захтевају прецизну контролу кретања. Идеалан је за примене које захтевају повратне информације високе резолуције у изазовним окружењима, као што су прецизна контрола мотора, роботика и индустријска аутоматизација. Његова робусна конструкција обезбеђује поуздан рад у захтевним условима.

Тип енкодера	Инкрементални ротациони енкодер
Резолуција	600 импулса по револуцији
Пречник осовине	6mm
Радни напон	5V до 24V DC
Тип излаза	Квадратни (А и Б фаза сигнали)
Излазни напон	Високо: $\geq 2.4V$ (TTL-компатибилан), Ниско: $\leq 0.4V$
Максимална фреквенција одговора	100 kHz
Максимална брзина ротације	5000 ротација у минути
Излазна струја	30 mA макс по каналу
Радна температура	-10°C до +70°C
Конекција	Кабл са штитом, 5-пински конектор
Димензије	Пречник 38mm x дужина 44mm
Тежина	150g

Техничке карактеристике ротационог енкодера LDP3806-600BM-G5-24C – Табела 7.5



Ротационог енкодера LDP3806-600BM-G5-24C – Слика 7.6

6.2. Додатак Б – Помоћне скрипте

6.2.1. data_processing_01.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def show_plot(time_s, angle, pwm, idx):
    plt.figure(idx)
    plt.plot(time_s, angle, time_s, pwm)
    plt.grid()
    plt.title("Test " + str(idx))
    plt.xlabel("Time [s]")
    plt.ylabel("Angle")
    plt.show()

def get_brake_point(df):
    for i in range(len(df["Time_s"])):
        if df["Time_s"][i + 1] - df["Time_s"][i] > 0.03:
            return i + 1
    return False

path_1 = "./Data/Encoder_data/Test_1/Raw/"
processed_path_1 = "./Data/Encoder_data/Test_1/Processed/"

exp_value_1 = [-45, -30, -20, -10, 10, 20, 30, 45]
j = -1

for i in range(40):
    if not i%5:
        j = j + 1
    try:
        # Load data
        input_data_path = path_1 + "Test_" + str(i) + ".csv"
        df = pd.read_csv(input_data_path)

        brake_poit = get_brake_point(df)

        df_2 = df[:brake_poit]
        df_3 = df[brake_poit:]

        miss_value = int(df_3["Time_s"].iloc[0] * 100) - int(df_2["Time_s"].iloc[-1] * 100)
        missed_angle = np.full(miss_value, df_2["Angles"].iloc[-1])
        missed_time = np.round(np.linspace(df_2["Time_s"].iloc[-1], df_3["Time_s"].iloc[0], miss_value), 2)
        missed_time = missed_time.iloc[-1] + 0.01, df_3["Time_s"].iloc[0] - 0.01, miss_value), 2)

        df_23 = pd.DataFrame()
        df_23["Time_s"] = missed_time
        df_23["Angles"] = missed_angle

        df_2["PWM"] = np.full(len(df_2["Angles"]), exp_value_1[j])
        df_23["PWM"] = np.full(len(df_23["Angles"]), exp_value_1[j])
        df_3["PWM"] = np.full(len(df_3["Angles"]), 0)
        df_final = pd.concat([df_2, df_23, df_3], ignore_index=True)
```

```

df_final.to_csv(processed_path_1 + "Test_" + str(i) + ".csv")
show_plot(time_s=df_final["Time_s"], angle=df_final["Angles"], idx=i, pwm=df_final["PWM"])
except:
    print("Data not found or relevant")

```

6.2.2. data_processing_02.py

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

```

def show_plot(time_s, angle, pwm, idx):
    plt.figure(idx)
    plt.plot(time_s, angle, time_s, pwm)
    plt.grid()
    plt.title("Test " + str(idx))
    plt.xlabel("Time [s]")
    plt.ylabel("Angle")
    fig_manager = plt.get_current_fig_manager()
    fig_manager.full_screen_toggle()
    plt.show()

```

```

def split_contiguous_pwm_blocks(df):
    blocks = []
    start = 0
    for i in range(1, len(df)):
        if df["PWM"].iloc[i] != df["PWM"].iloc[i - 1]:
            blocks.append(df.iloc[start:i].copy())
            start = i
    blocks.append(df.iloc[start:].copy())
    return blocks

```

```

def add_missing_data(data_1, data_2):
    missing_time = np.arange(
        start=data_1["Time_s"].iloc[-1] + 0.01,
        stop=data_2["Time_s"].iloc[0],
        step=0.01,
    )
    if len(missing_time) == 0:
        return pd.DataFrame(columns=["Angles", "Time_s", "PWM"])

    if np.equal(np.round(missing_time[-1], 2), data_2["Time_s"].iloc[0]):
        missing_time = np.delete(missing_time, -1)
    missing_time = np.round(missing_time, 2)
    missing_angle = np.full(shape=np.shape(missing_time), fill_value=data_1["Angles"].iloc[-1])
    missing_pwm = np.full(shape=np.shape(missing_time), fill_value=data_1["PWM"].iloc[0])

    df_tmp = pd.DataFrame()
    df_tmp["Angles"] = missing_angle
    df_tmp["Time_s"] = missing_time
    df_tmp["PWM"] = missing_pwm

    return df_tmp

```

```

path_1 = "./Data/Encoder_data/Test_2/Test_45/Raw_json/"
processed_path_1 = "./Data/Encoder_data/Test_2/Test_45/Processed/"

for i in range(15):
    try:
        # Load data
        output_data_path = path_1 + "Test_" + str(i) + ".csv"
        df = pd.read_csv(output_data_path)
        blocks = split_contiguous_pwm_blocks(df)

        new_data = {}
        for idx, block in enumerate(blocks):
            new_data[idx * 2] = block
            if idx < len(blocks) - 1:
                new_data[idx * 2 + 1] = add_missing_data(data_1=block, data_2=blocks[idx + 1])

        # Create new data by merging missing data and recorded data
        df_new = pd.DataFrame()
        for idx in range(len(new_data)):
            df_new = pd.concat([df_new, new_data[idx]], ignore_index=True)

        df_new.to_csv(processed_path_1 + "Test_" + str(i) + ".csv")
        show_plot(time_s=df_new["Time_s"], angle=df_new["Angles"], idx=i, pwm=df_new["PWM"])

    except Exception as error:
        print("Failed to process test " + str(i) + ": " + str(error))

```


6.2.3. data_visualization.py

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
matlab_data_path = "./Data/Estimation_data/GreyBox/Model.json"
py_data_path = "./Data/Estimation_data/GreyBox/Model.csv"
```

```
def workaround():
```

```
    """
```

```
    Convert MATLAB JSON export to CSV for pandas analysis.
```

```
    Manual JSON cleanup is required before this works reliably:
```

1. Open matlab_data_path
2. Format JSON (e.g. online formatter)
3. Replace ``"MSE": {`` with a blank line
4. Replace ``},`` with ``},``,```
5. Remove the trailing ``}```
6. Format JSON again
7. Run this function to write Model.csv
8. Rename the first column to ``test\_no`` if it has no header

```
    Writes:
```

```
    Model.csv at py_data_path
```

```
    """
```

```
    df = pd.read_json(matlab_data_path)
    df = df.T
    df.to_csv(py_data_path)
```

```
def show_plot():
```

```
    df = pd.read_csv(py_data_path)
    df = df.drop(columns=['test_no', 'num', 'den'])
```

```
    plt.style.use('_mpl-gallery')
```

```
    _, ax = plt.subplots()
    ax.set_title("MSE Data Visualisation")
    ax.boxplot(df, widths=0.2, patch_artist=True,
               showmeans=False, showfliers=False,
               medianprops={"color": "white", "linewidth": 0.5},
               boxprops={"facecolor": "C0", "edgecolor": "white",
                          "linewidth": 0.5},
               whiskerprops={"color": "C0", "linewidth": 1.5},
               capprops={"color": "C0", "linewidth": 1.5})
```

```
    ax.violinplot(df, widths=0.5, showmeans=False, showmedians=False, showextrema=False)
    ax.set_xticklabels(df.columns)
```

```
    plt.show()
```

```

def prepare_ranked_models(columns_to_drop=None):
    columns_to_drop = columns_to_drop or []

    df = pd.read_csv(py_data_path)
    mse_cols = [
        col for col in df.columns
        if col.startswith("mse_") and col not in columns_to_drop
    ]
    if not mse_cols:
        raise ValueError("No MSE columns left after dropping requested columns.")

    df["avg_mse"] = df[mse_cols].mean(axis=1)
    df["median_mse"] = df[mse_cols].median(axis=1)
    df["worst_mse"] = df[mse_cols].max(axis=1)

    df["tf_key"] = df["num"].astype(str) + df["den"].astype(str)
    unique = df.drop_duplicates("tf_key").sort_values("avg_mse")

    rank_cols = ["test_no", "num", "den", "avg_mse", "median_mse", "worst_mse"]
    return df, unique, mse_cols, rank_cols

def print_ranking_summary(unique, mse_cols, columns_to_drop=None, total_models=None):
    columns_to_drop = columns_to_drop or []
    total_models = total_models if total_models is not None else len(unique)

    print(f"Using {len(mse_cols)} validation columns: {' '.join(mse_cols)}")
    if columns_to_drop:
        print(f"Dropped: {' '.join(columns_to_drop)}")
    print(f"{total_models} models -> {len(unique)} unique transfer functions\n")

def print_top_models(unique, rank_cols, top_n=5):
    top_models = unique[rank_cols].head(top_n).reset_index(drop=True)
    top_models.index = top_models.index + 1
    top_models.index.name = "rank"

    print(f"=== Top {top_n} models (lowest avg_mse) ===")
    print(top_models.to_string())
    print()

    return top_models

def save_ranked_table(df):
    df.drop(columns="tf_key").to_csv(py_data_path, index=False)

def make_new_table(columns_to_drop=None, top_n=5):
    df, unique, mse_cols, rank_cols = prepare_ranked_models(columns_to_drop)
    print_ranking_summary(unique, mse_cols, columns_to_drop, total_models=len(df))
    top_models = print_top_models(unique, rank_cols, top_n=top_n)
    # save_ranked_table(df) # comment this out if you want to print the top N models only
    return top_models

if __name__ == "__main__":
    # workaround()
    # show_plot()
    make_new_table()

```

7. Литература

- [1] Mustafa Saad, Mohammad Khalallah, „Design and Implementation of an Embedded Ball-beam Controller Using PID Algorithm“, *Universal Journal of Control and Automation*, 5(4): 63–70, 2017. DOI: 10.13189/ujca.2017.050402
- [2] Yoonkyu Hwang, Yuki Minami, Masato Ishikawa, „Virtual Torque Sensor for Low-Cost RC Servo Motors Based on Dynamic System Identification Utilizing Parametric Constraints“, *Sensors*, 18(11): 3856, 2018. DOI: 10.3390/s18113856
- [3] Lennart Ljung, Carl Andersson, Koen Tiels, Thomas B. Schön, „Deep Learning and System Identification“, *IFAC-PapersOnLine*, 53(2): 1175–1181, 2020. DOI: 10.1016/j.ifacol.2020.12.1329
- [4] Budi Warsito, Rukun Santoso, Suparti, Hasbi Yasin, „Cascade Forward Neural Network for Time Series Prediction“, *Journal of Physics: Conference Series*, 1025(1): 012097, 2018. DOI: 10.1088/1742-6596/1025/1/012097
- [5] Douglas, B., *Linear System Identification | System Identification, Part 2*, YouTube канал MATLAB, 7. 12. 2021., доступно на: https://www.youtube.com/watch?v=qC_C04SEV1E (приступљено: 24. 8. 2026.)